

VOLUME 05 / RUST BACKEND MASTERY

Rustバックエンド — async・並行処理・ジョブ・運用

Future の仕組みから、タスク・channel・timeout・再試行・ログまで、たくさんの仕事を同時に安全にさばくサーバーの土台を身に付ける

第5巻: Rustバックエンド — async・並行処理・ジョブ・運用

この巻で身に付けること

Web アプリのサーバーには、たくさんの人から同時にリクエストが届きます。ノートアプリ RustNote でも、Aさんがノートを保存している間に、Bさんは一覧を開き、Cさんはログインしようとしています。

1つのリクエストの処理時間のほとんどは、実は**待ち時間**です。DB の返事を待つ、外の API の返事を待つ、ネットワークの向こうからデータが届くのを待つ。待っている間、CPU はほとんど何もしていません。その待ち時間に**ほかの人の仕事を進める**のが、この巻で学ぶ **async**（エイシンク：非同期処理）です。

ただし、たくさんの仕事を同時に進めると、1つずつ順番に進めるときには起きなかった事故が起きます。

- 1件ずつ順番に処理していて、10人が同時に来ると、10人目は**10倍待たされる**
- パスワードのハッシュのような重い計算をしている間、**ほかの全員の応答が止まる**
- 外の API が返事をしないまま、**いつまでも待ち続ける**
- 仕事が一気に押し寄せて、受け付けた仕事でメモリがあふれ、**サーバーごと落ちる**
- 鍵（Mutex）を持ったまま待ってしまい、**全員が鍵を待ったまま動かなくなる**
- サーバーを止めた瞬間に、**保存の途中だった仕事が切れる**
- エラーが起きても、どのリクエストで何が起きたのかを**ログからたどれない**

この巻のテーマは、こうした事故を、**Future**（フューチャー）**・タスク**・**channel**（チャネル）・**Semaphore**（セマフォ）・**timeout**（タイムアウト）・**spawn_blocking**（スポーン・ブロッキング）・**再試行**・**tracing**（トレーシング）という仕組みで防ぐことです。Rust では、多くの間違いを**コンパイルの時点で**止めてくれます。この巻では、そのエラーメッセージを読み解く力も身に付けます。

到達目標

この巻を終えると、次のことができるようになります。

- `async fn` が返す **Future** の正体（「まだ始まっていない仕事の札」で、`poll` されて少しずつ進む状態機械）を説明できる
- `tokio::join!` と `tokio::try_join!` で、複数の待ち時間を重ねて、全体を速くできる
- Tokio の**ランタイム**の役割を説明し、`tokio::spawn` でタスクを作って結果を受け取れる。タスクに `'static` と `Send` が要る理由が分かる
- `JoinSet` と `select!` で、子のタスクをまとめて待ち、最初に終わったものを選べる。同時に動かす数に上限を付けられる

- channel でタスクの間に値を渡し、容量の決まった channel で**多すぎる仕事に「待った」をかけられる**（backpressure：バックプレッシャー）
- `Mutex`・`RwLock`・`Semaphore` で共有するデータと同時に入れる数を守り、「鍵を持ったまま `.await` しない」を守れる
- `timeout` で待つ時間に上限を付け、Ctrl+C を受けたら処理中の仕事を片付けてから止まれる（graceful shutdown：グレースフル・シャットダウン）
- 重い計算を `spawn_blocking` に逃がし、ほかのリクエストを止めない
- HTTP クライアントで外の API を呼び、一時的な失敗だけを間隔をあけて再試行できる。同じ操作を2回送っても困らない形（冪等性：べきとうせい）を説明できる
- tracing で、リクエストごとの情報（何の処理か・誰の処理か・何ミリ秒かかったか）を付けたログを出せる
- 画面の外で動く仕事（バックグラウンドジョブ）を、サーバーが落ちても続けられる形で設計できる

学習地図

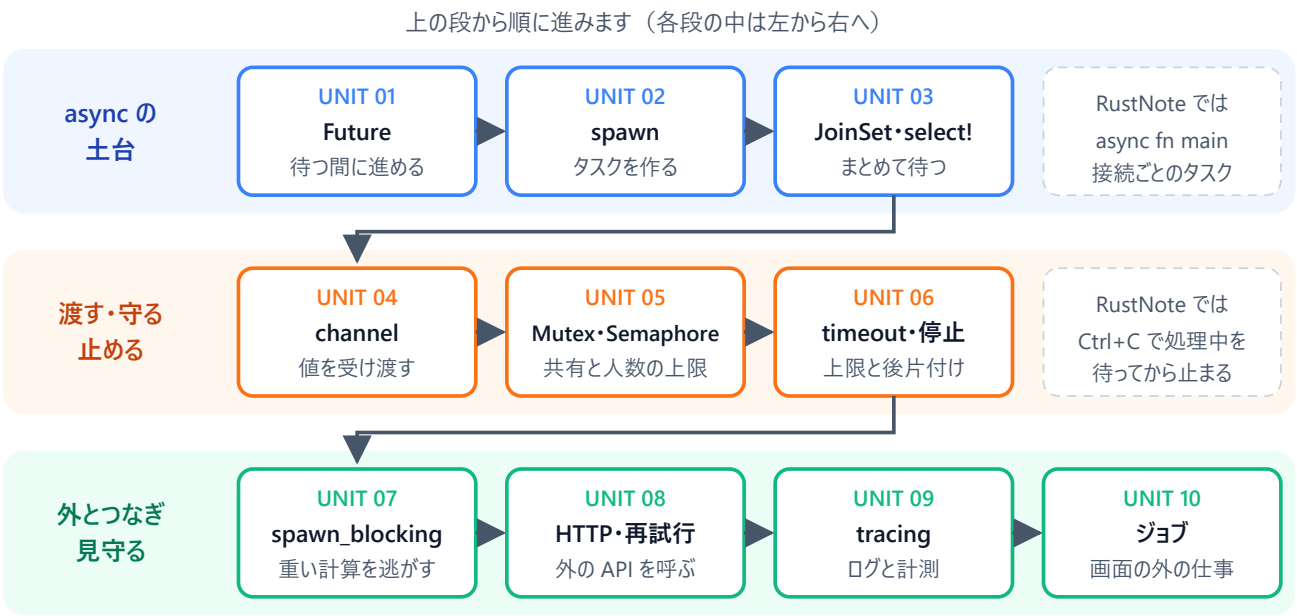


図5-1: 第5巻の学習地図。async の土台から始めて、タスクの間の受け渡し・守り方・止め方、外とのやり取りと見守り方へ進む

単元	テーマ	ひとことで言うと
UNIT 01	async・Future・await の正体	「待つ間に、ほかの仕事を進める」仕組みを、中からのぞく
UNIT 02	Tokio のランタイム・spawn・JoinHandle	仕事（タスク）を作り、何人もの作業員で分け合う
UNIT 03	JoinSet・select!・同時に動かす数の管理	子のタスクをまとめて待ち、最初に終わったものを選ぶ
UNIT 04	channel・backpressure・所有権の移動	タスクの間で値を受け渡し、多すぎる仕事に「待った」をかける
UNIT 05	Mutex・RwLock・Semaphore と .await	共有するデータと、同時に入れる数を守る
UNIT 06	timeout・キャンセル・graceful shutdown（準備中）	待つ時間に上限を付け、止めるときは仕事を片付けてから止める
UNIT 07	spawn_blocking と重い計算（準備中）	パスワードのハッシュのような重い仕事を、別の場所に逃がす
UNIT 08	HTTP クライアント・再試行・冪等性（準備中）	外の API を呼び、失敗したら賢くやり直す
UNIT 09	tracing・構造化ログ・メトリクス（準備中）	何が起きたかを、あとからたどれるように記録する
UNIT 10	バックグラウンドジョブと運用（準備中）	画面の外で動く仕事を、落ちても続けられる形にする
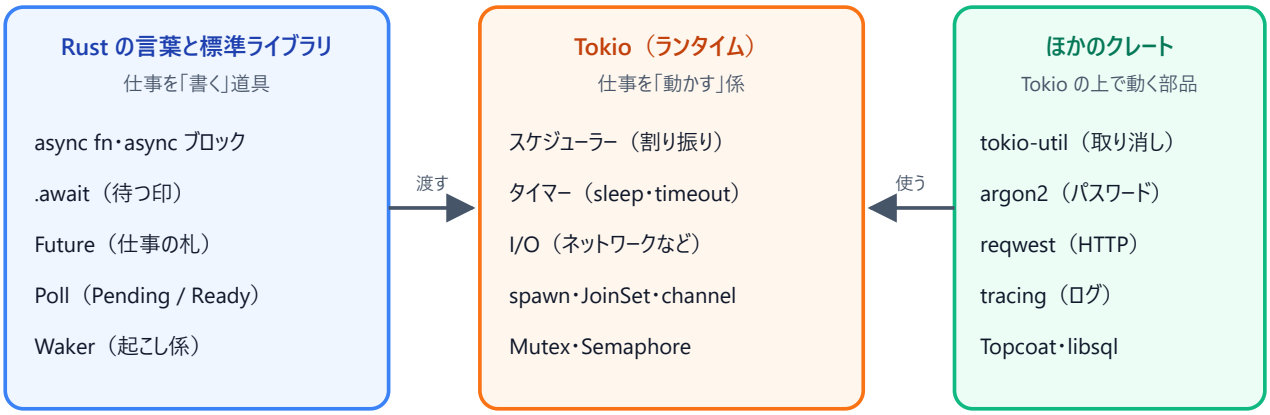
> **お知らせ:** この版には UNIT 01〜05 を収録しています。UNIT 06〜10 は準備中で、次の更新から順に追加します。本文の中の「UNIT 06 で学びます」のような予告は、これから追加する単元を指しています。

この巻に出てくる名前の整理

この巻には、Rust の言葉そのものと、クレート（部品）の名前が混ざって出てきます。最初に、どれが何なのかを分けておきましょう。

- `async` / `.await` / `Future`: Rust の言葉と標準ライブラリにある仕組みです。「待ちながら進む仕事」を**書く**ための道具で、どの Rust のプログラムでも使えます。
- **Tokio**（トーキョー）: その仕事を**実際に動かす**ためのクレートです。Rust の標準ライブラリには「Future を動かす係」が入っていないので、Tokio のような**ランタイム**を足して使います。RustNote が使う Topcoat も、第7巻の libsql も、Tokio の上で動きます。

- **そのほかのクレート**: Tokio の上で動く、目的別の部品です。この巻では、取り消しの合図に tokio-util、パスワードに argon2、HTTP クライアントに reqwest（リクエスト）、ログに tracing を使います。



async fn が作った Future を Tokio が動かし、ほかのクレートも Tokio の力を借りて動く

図5-2: この巻に出てくる名前の関係。仕事を「書く」のは Rust の言葉、「動かす」のは Tokio、目的別の部品は Tokio の上で動く

> **ヒント:** 第7巻（Turso・libSQL）では、`#[tokio::main]` を「`main` の中で `.await` を使えるようにするおまじない」、`.await` を「返事が来るまで待つ印」として使いました。その「おまじない」の中身を、UNIT 01 と UNIT 02 で開けて見ていきます。第7巻をまだ読んでいなくても、この巻は読み進められます。

前提知識（先に読んでおく巻）

- **第1巻（環境構築）**: Rust・VS Code・rust-analyzer がインストール済みで、`cargo new` と `cargo run` ができること。
- **第2巻（Rust 基礎）**: `struct` / `enum` / `match` / `Option` / `Result` / `?` / `Vec` / `String` / クロージャの基本。
- **第3巻（所有権・借用）**: 所有権の移動（move）と借用。`'static`（第3巻 UNIT 06）。そして特に、**第3巻 UNIT 10 の** `Arc`・`Mutex`・`Send`・`Sync` は、この巻で何度も使います。スレッドを作る `std::thread::spawn` の話を覚えていると、UNIT 02 の `tokio::spawn` がすっと入ります。
- **第4巻（設計）**: trait と generics、エラー型（thiserror と anyhow）、`Cargo.toml` の版の書き方（第4巻 UNIT 07）。

> **注意:** この巻では、スレッド・`Arc`・`Mutex` を「第3巻で学んだもの」として使います。`Send` と `Sync` の意味があいまいなときは、先に第3巻 UNIT 10 の「図で理解する」と「よくあるエラー」だけでも読み返しておくと、UNIT 02 のエラーメッセージがずっと読みやすくなります。

この巻の進め方

各単元では、次の順番を1サイクルとして進めてください。async は、頭の中で考えるだけでは、どの順番で何が起きるのかが想像しにくい分野です。**表示の順番を予想してから動かす**のが、いちばんの近道です。

1. **予想する:** 実行する前に、どの行がどの順番で表示されるか、全体で何秒かかるかを予想します。
2. **そのまま動かす:** 教材のコードをそのまま実行し、結果が教材と同じか比べます。
3. **1か所だけ変える:** 待ち時間や `.await` の位置を1つだけ変えて、表示の順番と時間がどう変わるかを観察します。「よくあるエラー」の節のコードも実際にビルドして、エラーメッセージを自分の目で読みます。
4. **整える:** 単元の終わりには、次の3つのコマンドを実行する習慣をつけます。

```
# コードの見た目を Rust の標準の形に整える
cargo fmt
# よくある書き間違い・もったいない書き方を指摘してもらう
cargo clippy
# テストを実行する
cargo test
```

> **ヒント:** この巻の実行結果には、かかった時間（ `0.5 秒` など）や、スレッドの番号（ `ThreadId(13)` など）が出てきます。時間はパソコンの速さやその時の混み具合で少し変わり、スレッドの番号は実行するたびに変わります。表示の順番が教材と入れ替わることもあります。どういうときに入れ替わるのかは、UNIT 02 で説明します。

使うツールと版

この巻のコードは、すべて次の環境で実際にコンパイル・実行し、実行結果とエラーメッセージは実際の出力をそのまま載せています（パソコンのユーザー名などを含む部分だけ `...` に置き換えています）。

項目	版・内容
OS	Windows 11（ARM64）。コマンドは PowerShell を主に、WSL / macOS / Linux は補足で示します
Rust	**1.98 以上**（本書シリーズの基準）。動作確認は rustc 1.99.0 / cargo 1.99.0（`aarch64-pc-windows-msvc`）
edition	2024（`cargo new` の既定値）
tokio	1.53.3（`full` の機能つき。この巻の全単元で使う）
tokio-util	0.7.20（UNIT 06。取り消しの合図と、タスクの見張り）
argon2	0.6.0（UNIT 07。パスワードのハッシュ）
reqwest	0.13.5（UNIT 08。HTTP クライアント。ARM64 版の Windows では、ビルドに clang が必要）
serde / serde_json	1.0.229 / 1.0.154（UNIT 08・09。JSON との変換）
tracing / tracing-subscriber	0.1.44 / 0.3.23（UNIT 09。ログ）

UNIT 01〜05 で使うクレートは tokio だけです。ほかのクレートは、使う単元で `cargo add` します。

練習用プロジェクトを用意する

この巻では、練習用のプロジェクト `async_lab` を1つ作り、小さなプログラムを `src/bin/` に1つずつ足していきます。

```
cargo new async_lab
cd async_lab
mkdir src/bin
cargo add tokio@=1.53.3 --features full
```

`cargo new` と `cargo add` の実際の出力は次のとおりです。

```
    Creating binary (application) `async_lab` package
note: see more `Cargo.toml` keys and their definitions at https://doc.rust-
lang.org/cargo/reference/manifest.html
    Updating crates.io index
      Adding tokio =1.53.3 to dependencies
             Features:
             + bytes
             + fs
             + full
             + io-std
             + io-util
             + libc
             + macros
             + net
             + parking_lot
             + process
             + rt
             + rt-multi-thread
             + signal
             + signal-hook-registry
             + socket2
             + sync
             + time
             + tokio-macros
             - io-uring
             - mio
             - schedule-latency
             - taskdump
             - test-util
             - tracing
             - windows-sys
    Updating crates.io index
     Locking 24 packages to highest Rust 1.99.0 compatible versions
      Adding tokio v1.53.3 (available: v1.54.0)
```

- `tokio@=1.53.3` の `=` は「ちょうど 1.53.3」という意味です（第4巻 UNIT 07）。最後の行の `available: v1.54.0` は「もっと新しい 1.54.0 も出ています」というお知らせです。`=` を付けずに `tokio@1.53.3` と書くと、Cargo は 1.x の中でいちばん新しい版を選ぶので、2026年10月22日の時点では 1.54.0 が入ります。この巻の出力と同じ結果になるように、版を固定しておきます。
- `--features full` は、Tokio の機能を**全部** ON にする指定です。`+` が付いた機能が ON になりました。`time`（待つ・時間の上限）、`sync`（channel や Mutex）、`macros`（`#[tokio::main]`）、`rt-multi-thread`（何本ものスレッドで動かす）、`signal`（Ctrl+C を受け取る）など、この巻で使う機能が全部入っています。
- 本番のアプリでは、使う機能だけを選ぶのがふつうです。RustNote の `Cargo.toml` は `features = ["rt-multi-thread", "macros"]` の2つだけです。機能を絞ると、ビルドが速くなり、使わない部品が入りません。

最後に、`Cargo.toml` の `[package]` の中、`edition = "2024"` の次の行に `rust-version = "1.98"` を手で1行足します。できあがった `Cargo.toml` は次のとおりです。

```
[package]
name = "async_lab"
version = "0.1.0"
edition = "2024"
rust-version = "1.98"

[dependencies]
tokio = { version = "=1.53.3", features = ["full"] }
```

一度 `cargo build` しておきましょう。最初のビルドでは Tokio とその部品をコンパイルするので、少し時間がかかります（この PC では約12秒でした）。

```
   Compiling tokio v1.53.3
   Compiling async_lab v0.1.0 (...\async_lab)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 11.89s
```

src/bin/ に置いたファイルは、それぞれが1つのプログラム

`src/bin/` フォルダに置いた `.rs` ファイルは、1つずつが別のプログラムになります。たとえば `src/bin/u01_lazy.rs` を作ったら、次のように名前（拡張子なし）を指定して実行します。

```
cargo run -q --bin u01_lazy
```

- `--bin u01_lazy` は「`src/bin/u01_lazy.rs` のプログラムを実行して」という指定です。
- `-q`（quiet：静かに）を付けると、`Compiling ...` や `Running ...` のような Cargo の表示が消えて、プログラムが表示した内容だけが残ります。この巻の実行結果は、特に断りがなければ `-q` を付けて実行したものです。
- コンパイルエラーの例は、エラーの全文を見るために `-q` を付けずに `cargo build --bin 名前` や `cargo run --bin 名前` で実行しています。

> **ヒント:** VS Code で `src/bin/` のファイルを開くと、`fn main` の上に「▶ Run」の文字が出ます。これをクリックしても同じように実行できます（rust-analyzer の機能です）。

UNIT 01: async・Future・await の正体

この単元のゴール

- `async fn` を呼んだだけでは中身が動かず、**Future**（フューチャー）という「仕事の札」が返ってくること、`.await` して初めて動き出すことを、実行結果で説明できる
- `.await` を順番に書いたときと、`tokio::join!` で同時に進めたときの時間の違いを、図で説明できる
- `tokio::try_join!` で、どれか1つが失敗したら、残りを待たずにすぐ失敗を返す処理を書ける
- Future が **poll**（ポール）されて、`Pending`（まだです）と `Ready`（できました）を返しながら少しずつ進む仕組みを、小さな実行係を自分で動かして確かめられる
- `async` や `.await` の付け忘れで出るエラーと警告を読んで、直せる

たとえ話でつかむ

ラーメン屋の店員にたとえます。麺をゆでるには3分かかります。

- **順番に待つ店員**は、麺を鍋に入れたら、ゆで上がるまでの3分間、鍋の前にじっと立っています。その間に来たお客さんは、注文すらできずに待たされます。
- **待ち時間をうまく使う店員**は、麺を鍋に入れたらタイマーを3分にセットして、次のお客さんの注文を取りに行きます。タイマーが鳴ったら、鍋に戻って麺を上げます。

店員は1人のままなのに、後の店員のほうが、ずっとたくさんのお客さんをさばけます。待ち時間が重なっているからです。これが **async**（非同期処理）の考え方です。この店のしくみを、Rust の言葉に置き換えると次のようになります。

ラーメン屋	Rust の async
注文の伝票	**Future**。書いた（作った）だけでは、料理は始まらない
店員	**ランタイム**（実行係）。伝票を手に取って、進められるところまで進める
伝票を手に取って進めること	**poll**（ポール）
「まだゆでている」	`Poll::Pending`。店員は伝票を置いて、ほかの伝票を進める
「できあがり」	`Poll::Ready(値)`
キッチンタイマー	**Waker**（ウェイカー：起こし係）。鳴ったら「あの伝票をもう一度見て」と店員に知らせる
伝票に書かれた「ここで待つ」の印	`.await`。待つ間、店員はほかの伝票を進めてよい

> **用語:** 非同期処理（async：エイシンク。待ち時間の間に、ほかの仕事を進められるように書いた処理。Rust では `async fn` や `async { }` で書く）

> **用語:** Future（フューチャー：「まだ終わっていない仕事」を表す値。`async fn` を呼ぶと返ってくる。`.await` されるまで、中身は動かない）

> **用語:** poll（ポール：Future に「進められるところまで進めて」と頼むこと。答えは `Poll::Pending`（まだです）か `Poll::Ready(値)`（できました）のどちらか）

> **用語:** ランタイム（runtime：Future を poll して動かす係。executor（エグゼキューター：実行係）とも呼ぶ。Rust の標準ライブラリには入っていないので、この巻では Tokio を使う）

図で理解する

まず、「順番に待つ」と「待ち時間を重ねる」で、全体の時間がどう変わるかを見ます。0.5 秒かかる DB への問い合わせを2つ行う場合です。この後の例1-2で、実際に測ります。

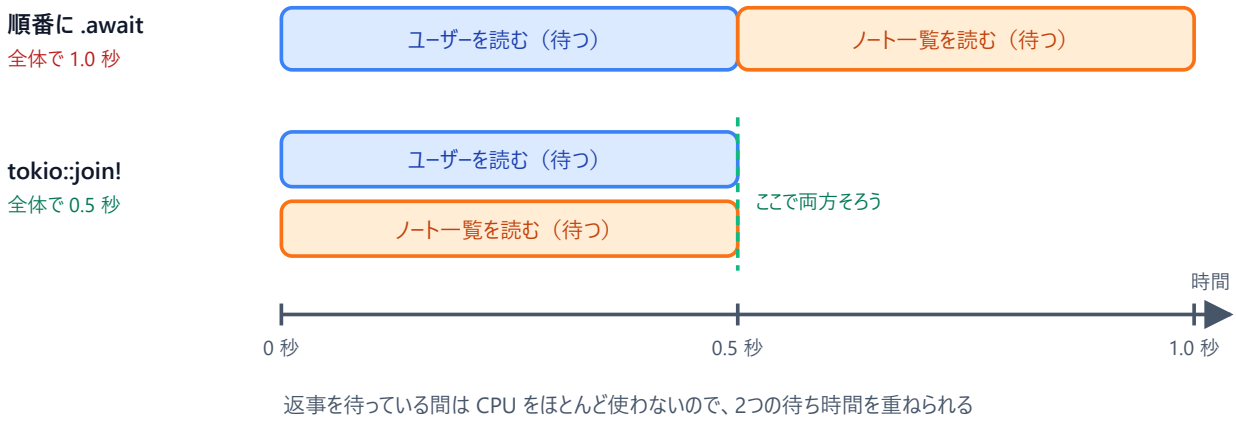


図5-3: 0.5 秒の問い合わせを2つ行うときの時間。順番に .await すると足し算で 1.0 秒、tokio::join! で同時に進めると、長いほうの 0.5 秒で終わる

- 順番に `.await` すると、1つ目の返事が来るまで、2つ目の問い合わせは**始まりもしません**。全体の時間は足し算になります。
- `tokio::join!` を使うと、2つの問い合わせを両方とも始めてから、両方の返事を待ちます。全体の時間は、**いちばん長いもの**で決まります。

> **用語:** 並行と並列（並行 concurrency：コンカレンシー は、複数の仕事を「切り替えながら」少しずつ進めること。並列 parallelism：パラレリズム は、複数の CPU で「本当に同時に」動かすこと。`tokio::join!` は並行で、CPU は1つしか使わない）

次に、Future が中でどう動いているかです。ランタイムは Future を poll し、Future は「まだです」か「できました」を答えます。

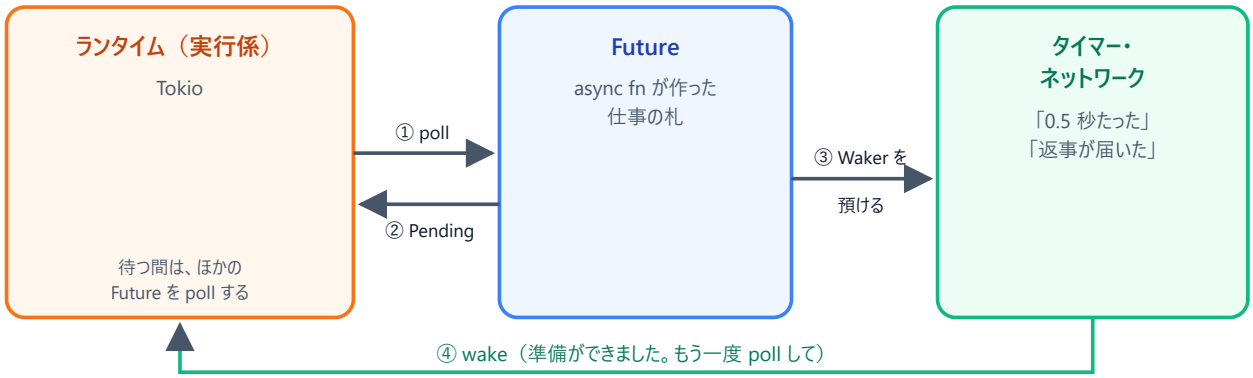


図5-4: Future が進むしくみ。poll されて「まだ」なら Waker（起こし係）を預けて Pending を返し、準備ができたら起こしてもらって、もう一度 poll される

- ① ランタイムが Future を poll します。② 返事がまだなら、Future は `Pending` を返します。③ そのとき、「準備ができたら起こしてください」と **Waker** をタイマーやネットワークの係に預けておきます。
- ランタイムは、その Future のことはいったん忘れて、ほかの Future を poll します。待っている Future のために、CPU を使い続けることはありません。
- ④ 時間がたったり返事が届いたりすると、Waker が「もう一度 poll して」とランタイムに知らせます。⑤ もう一度 poll すると、今度は `Ready(値)` が返ります。

> **ヒント:** ふだんのプログラムでは、自分で poll を呼ぶことはありません。`.await` と書けば、ここまでの流れを Rust とランタイムが全部やってくれます。例1-4で、この流れを一度だけ自分の目で確かめます。

まず動かす最小コード

例1-1: async fn を呼んだだけでは、中身は動かない

> **▼ このコードがやること（先に日本語で）:** 「お湯を沸かす」非同期関数 `boil_water` を作ります。`main` から呼んだ直後と、`.await` した後に、それぞれ表示を出して、関数の中身が**いつ**動いたのかを確かめます。

```rust
use std::time::Duration;

// 「お湯を沸かす」非同期関数。呼ぶと Future（まだ始まっていない仕事の札）が返る
async fn boil_water() -> u32 {
    println!("  [boil_water] 火をつけました");
    // 0.3 秒待つ（この間、ほかの仕事に席を譲る）
    tokio::time::sleep(Duration::from_millis(300)).await;
    println!("  [boil_water] 沸きました");
    100
}

#[tokio::main]
async fn main() {
    println!("1. boil_water() を呼びます");
    let future = boil_water();
    println!("2. 呼び終わりました（まだ火はついていません）");
    let temperature = future.await;
    println!("3. .await が終わりました: {temperature} 度");
}
```

`src/bin/u01_lazy.rs` として保存して実行します。

```
cargo run -q --bin u01_lazy
```

▼ 実行結果

```
1. boil_water() を呼びます
2. 呼び終わりました（まだ火はついていません）
  [boil_water] 火をつけました
  [boil_water] 沸きました
3. .await が終わりました: 100 度
```

`boil_water()` を呼んだのは「1.」と「2.」の間なのに、「火をつけました」は「2.」の**後**に表示されています。

▼ コードを1つずつ分解して解説

解説1: async fn を呼ぶと、戻り値の代わりに Future が返る

- `async fn boil_water() -> u32` は「`u32` を返す関数」ではなく、「**いつか `u32` ができあがる Future を返す関数**」です。
- `let future = boil_water();` の時点では、関数の中身は1行も動いていません。手元に「お湯を沸かす」と書かれた伝票ができただけです。
- だから「2.」の表示が先に出ます。これを、Future は**怠け者**（lazy：レイジー）だ、と言います。

解説2: .await して初めて、中身が動き出す

- `future.await` と書いたところで、ランタイムが伝票を手に取り、中身を進め始めます。「火をつけました」はここで表示されます。
- 中身の `sleep(...).await` で 0.3 秒待つ間、`main` も一緒に止まって待ちます。中身が最後まで進んで `100` を返すと、`future.await` 全体が `100` という値になり、`temperature` に入ります。

> **ヒント:** JavaScript の `async` 関数は、呼んだ瞬間に中身が動き始めます（Promise）。Rust の Future は `.await` するまで動きません。JavaScript を知っている人ほど、ここでつまずきやすいので注意しましょう。

解説3: tokio::time::sleep は「席を譲って」待つ

- `tokio::time::sleep(時間)` は、指定した時間だけ待つ Future を作ります。`.await` すると、待っている間、**ほかの Future に席を譲ります。**
- 第3巻で使った `std::thread::sleep` は、スレッドそのものを眠らせます。async の中でこれを使うと、そのスレッドで動いているほかの Future も全部止まってしまいます。

> **注意:** `async fn` の中では、`std::thread::sleep` を使わず、`tokio::time::sleep(...).await` を使います。同じように、時間のかかる計算やファイルの読み書きも、そのまま書くとほかの仕事を止めてしまいます。その避け方は UNIT 07 で学びます。

解説4: #[tokio::main] は、main の外側にランタイムを用意する

- Rust の `main` 関数は、ふつうは `async` にできません（「よくあるエラー1」で確かめます）。`main` を呼ぶのは OS で、OS は Future の poll のしかたを知らないからです。
- `#[tokio::main]` を付けると、「Tokio のランタイムを作って、その上で `async fn main` の Future を最後まで動かす」ふつうの `main` に書き換えてくれます。中で何をしているかは、UNIT 02 で自分の手で書いて確かめます。

例1-2: 順番に待つか、同時に待つか

> **▼ このコードがやること（先に日本語で）:** 0.5 秒かかる「DB への問い合わせ」の代わりの関数 `query` を2回使います。①では1つずつ順番に `.await` し、②では `tokio::join!` で2つを同時に進めて、それぞれ全体で何秒かかったかを表示します。

```rust
use std::time::{Duration, Instant};

// DB への問い合わせの代わり: 指定した時間だけ待ってから結果を返す
async fn query(name: &str, millis: u64) -> String {
    tokio::time::sleep(Duration::from_millis(millis)).await;
    format!("{name} の結果")
}

#[tokio::main]
async fn main() {
    // ① 順番に .await する（1つ終わってから次を始める）
    let start = Instant::now();
    let user = query("ユーザー", 500).await;
    let notes = query("ノート一覧", 500).await;
    println!(
        "順番に:   {user} / {notes} / {:.1} 秒",
        start.elapsed().as_secs_f64()
    );

    // ② tokio::join! で同時に進める（両方の待ち時間が重なる）
    let start = Instant::now();
    let (user, notes) = tokio::join!(query("ユーザー", 500), query("ノート一覧", 500));
    println!(
        "同時に:   {user} / {notes} / {:.1} 秒",
        start.elapsed().as_secs_f64()
    );
}
```

`src/bin/u01_join.rs` として保存して実行します。

```
cargo run -q --bin u01_join
```

▼ 実行結果

```
順番に:   ユーザー の結果 / ノート一覧 の結果 / 1.0 秒
同時に:   ユーザー の結果 / ノート一覧 の結果 / 0.5 秒
```

▼ コードを1つずつ分解して解説

解説1: 順番に .await すると、時間は足し算になる

- `query("ユーザー", 500).await` は、0.5 秒待って結果を受け取るまで、次の行へ進みません。2つ目の `query` は、1つ目が終わってから**作られ**、そこから 0.5 秒待ちます。
- だから全体で 0.5 + 0.5 = 1.0 秒です。

- `Instant::now()` は「今の時刻」、`start.elapsed()` は「`start` からたった時間」です。`as_secs_f64()` で秒の小数にし、`{:.1}` で小数第1位まで表示しています。

解説2: tokio::join! は、全部の Future を同時に進めて、全部を待つ

- `tokio::join!(A, B)` は、A と B の両方を進め、**両方が終わったら**、結果を `(Aの結果, Bの結果)` のタプルで返します。
- 中では、A を poll して `Pending` なら B を poll し、どちらかが起こされたらまた poll し…を繰り返しています。2つの 0.5 秒の待ちが重なるので、全体で 0.5 秒です。
- 3つ以上でも `tokio::join!(A, B, C)` のように並べられます（練習問題2）。

解説3: join! は「並行」。CPU を2つ使うわけではない

- `tokio::join!` は、1つの流れ（1つのタスク）の中で、A と B を**切り替えながら**進めます。CPU は1つしか使っていません。
- 速くなったのは、2つとも「待っているだけ」だったからです。もし A と B が、待ち時間のない重い計算だったら、`join!` にしても速くなりません。計算を本当に同時に（並列に）動かす方法は、UNIT 02 と UNIT 07 で学びます。

解説4: 前の結果を使うときは、順番に待つしかない

- 「ユーザーを読んでから、**そのユーザーの id で**ノートを読む」のように、2つ目が1つ目の結果を使うなら、`join!` にはできません。1つ目が終わらないと、2つ目を始められないからです。
- `join!` が使えるのは、**お互いに関係のない待ち**だけです。コードを読むときは「この2つは、どちらかの結果を使っているか？」を確かめましょう。

もう一歩: 実務に近い形

例1-3: 1つのリクエストで、2つの問い合わせを同時に進める（try_join!）

> **▼ このコードがやること（先に日本語で）**：画面の上に「hanako さんのノートは 3 件」と出すダッシュボードを考えます。ユーザーを読む問い合わせ（0.3 秒）と、ノートを数える問い合わせ（0.5 秒）を同時に進めます。どちらかが失敗したら、もう片方を待たずに、すぐ失敗を返します。いるユーザー（1）と、いないユーザー（2）で試します。

```rust
use std::time::{Duration, Instant};

#[derive(Debug)]
struct User {
    name: String,
}

// ユーザーを読む（DB の代わり）。0.3 秒かかる
async fn load_user(user_id: i64) -> Result<User, String> {
    tokio::time::sleep(Duration::from_millis(300)).await;
    if user_id == 1 {
        Ok(User {
            name: String::from("hanako"),
        })
    } else {
        Err(format!("ユーザー {user_id} はいません"))
    }
}

// ノートの件数を数える（DB の代わり）。0.5 秒かかる
async fn count_notes(user_id: i64) -> Result<u32, String> {
    tokio::time::sleep(Duration::from_millis(500)).await;
    Ok(user_id as u32 * 3)
}

// 1つのリクエストの処理: 2つの問い合わせを同時に進め、どちらかが失敗したらすぐ失敗を返す
async fn dashboard(user_id: i64) -> Result<String, String> {
    let (user, count) = tokio::try_join!(load_user(user_id), count_notes(user_id))?;
    Ok(format!("{} さんのノートは {count} 件", user.name))
}

#[tokio::main]
async fn main() {
    for user_id in [1, 2] {
        let start = Instant::now();
        let result = dashboard(user_id).await;
        println!("{result:?} / {:.1} 秒", start.elapsed().as_secs_f64());
    }
}
```

`src/bin/u01_try_join.rs` として保存して実行します。

```
cargo run -q --bin u01_try_join
```

▼ 実行結果

```
Ok("hanako さんのノートは 3 件") / 0.5 秒
Err("ユーザー 2 はいません") / 0.3 秒
```

ユーザー 2 の失敗は 0.3 秒で返ってきました。ノートを数える 0.5 秒を待っていません。

▼ コードを1つずつ分解して解説

解説1: try_join! は、Result を返す Future のための join!

- `tokio::try_join!(A, B)` の A と B は、どちらも `Result` を返す Future です。
- 両方が `Ok` なら、`Ok((Aの中身, Bの中身))` を返します。どれか1つでも `Err` になったら、**その時点で**その `Err` を返します。
- 後ろの `?` で、`Err` なら `dashboard` からそのエラーを返し、`Ok` なら中身のタプルを取り出して `(user, count)` に分けています（第2巻 UNIT 09 の `?` と同じです）。

解説2: 失敗したら、残りの Future は途中で捨てられる

- ユーザー 2 では、0.3 秒の時点で `load_user` が `Err` を返しました。`try_join!` はすぐに `Err` を返し、まだ途中だった `count_notes` の Future は **drop**（ドロップ：片付け）されます。
- drop された Future は、二度と poll されません。つまり `count_notes` は、0.3 秒の時点で**途中で止まった**ことになります。
- これは便利な反面、「途中まで書き込んだところで止まる」と困る処理もあります。Future が途中で捨てられることを**キャンセル**と言い、UNIT 06 でくわしく扱います。

解説3: エラーの型は、そろえる必要がある

- `try_join!` に渡す Future は、エラーの型が同じでなければなりません。ここでは、どちらも `Result<_, String>` にそろえています。
- 実際のアプリでは、DB のエラーと検証のエラーのように型が違うことがよくあります。そのときは、第4巻 UNIT 05 で作ったような、アプリ共通のエラー型（RustNote の `AppError` など）に変換してから渡します。

例1-4: 小さな実行係を自分で作って、poll の様子をのぞく

ここは、Future の中身をのぞく発展の内容です。ふだんのアプリで、自分で Future を実装したり poll を呼んだりすることは、ほとんどありません。難しく感じたら、実行結果の流れだけ眺めて、例1-5へ進んでもかまいません。

> **▼ このコードがやること（先に日本語で）**：決まった回数だけ「まだです」と答える Future `Countdown` を自分で作ります。それを2回 `.await` する `make_tea` を、Tokio を使わずに、自分で作った最小の実行係 `block_on` で動かします。poll が何回呼ばれ、そのたびにどこまで進むかを表示します。

```rust
use std::future::Future;
use std::pin::{Pin, pin};
use std::task::{Context, Poll, Waker};

// 決まった回数だけ「まだです（Pending）」と答え、その後「できました（Ready）」と答える Future
struct Countdown {
    remaining: u32,
}

impl Future for Countdown {
    type Output = &'static str;

    fn poll(mut self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output> {
        if self.remaining == 0 {
            println!("    Countdown: Ready を返す");
            Poll::Ready("完了")
        } else {
            println!(
                "    Countdown: Pending を返す（あと {} 回）",
                self.remaining
            );
            self.remaining -= 1;
            // 「準備ができたので、もう一度 poll してください」と実行係に知らせる
            cx.waker().wake_by_ref();
            Poll::Pending
        }
    }
}

// async fn の中に .await が2つある。コンパイラはこれを状態機械に変換する
async fn make_tea() -> String {
    println!("  [make_tea] 開始");
    let first = Countdown { remaining: 2 }.await;
    println!("  [make_tea] 1つ目の待ちが終わった: {first}");
    let second = Countdown { remaining: 1 }.await;
    println!("  [make_tea] 2つ目の待ちが終わった: {second}");
    String::from("お茶")
}

// 最小の実行係（executor）: Ready が返るまで poll を呼び続ける
fn block_on<F: Future>(future: F) -> F::Output {
    // poll するには、Future が動かない場所に固定（pin）されている必要がある
    let mut future = pin!(future);
    // 何もしない Waker（起こし係）。この実行係はどうせすぐ次の poll をする
    let mut cx = Context::from_waker(Waker::noop());
    let mut count = 0;
    loop {
        count += 1;
        println!("実行係: {count} 回目の poll");
```

```
        if let Poll::Ready(value) = future.as_mut().poll(&mut cx) {
            return value;
        }
    }
}

fn main() {
    let tea = make_tea();
    println!("make_tea() を呼んだ。まだ何も表示されていないはず");
    let result = block_on(tea);
    println!("結果: {result}");
}
```

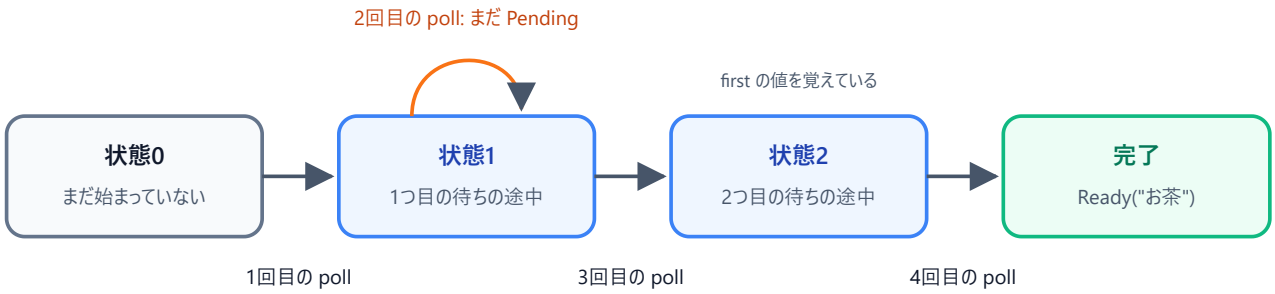
`src/bin/u01_mini_executor.rs` として保存して実行します。このプログラムは Tokio を使っていません（`#[tokio::main]` もありません）。

```
cargo run -q --bin u01_mini_executor
```

▼ 実行結果

```
make_tea() を呼んだ。まだ何も表示されていないはず
実行係: 1 回目の poll
  [make_tea] 開始
    Countdown: Pending を返す（あと 2 回）
実行係: 2 回目の poll
    Countdown: Pending を返す（あと 1 回）
実行係: 3 回目の poll
    Countdown: Ready を返す
  [make_tea] 1つ目の待ちが終わった: 完了
    Countdown: Pending を返す（あと 1 回）
実行係: 4 回目の poll
    Countdown: Ready を返す
  [make_tea] 2つ目の待ちが終わった: 完了
結果: お茶
```

`make_tea` は、コンパイラによって次のような「状態機械」に作り変えられています。poll されるたびに、今の状態から次の `Pending` まで一気に進みます。



3回目の poll は、1つ目の待ちを終えて、2つ目の待ちの Pending まで一気に進んだ

図5-5: make_tea が作り変えられた状態機械。poll されるたびに、今の状態から次の Pending まで進み、.await をまたいで使う変数（first）は状態の中に保存される

▼ コードを1つずつ分解して解説

解説1: Future は、poll という関数を1つ持つ trait

- `Future` は標準ライブラリにある trait（第4巻 UNIT 02）で、`poll` という関数を1つだけ持っています。`type Output` は、できあがったときの値の型です。
- `poll` の答えは `Poll::Pending`（まだです）か `Poll::Ready(値)`（できました）のどちらかです。
- `Countdown` は、`remaining` が 0 になるまで `Pending` を返し、0 になったら `Ready("完了")` を返します。

解説2: Pending を返すときは、必ず「起こす約束」をする

- `cx.waker().wake_by_ref();` は、「もう一度 poll してください」とランタイムに知らせる呼び出しです。
- `Countdown` は「すぐに準備ができる」ので、その場で起こしています。本物のタイマーなら、「時間になったら起こす」ように Waker をタイマーに預けます（図の③）。
- `Pending` を返すのに、誰も Waker で起こさないと、その Future は**二度と poll されず、止まったまま**になります。自分で Future を実装するときに、いちばん気をつける約束です。

解説3: block_on は「Ready が返るまで poll し続ける」だけの実行係

- `pin!(future)` は、Future をメモリの中で**動かないように留めて**います（pin：ピン留め）。async fn が作る状態機械は、自分の中の変数を指す参照を持つことがあるので、poll の途中で場所が変わると困るからです。`poll` の引数が `Pin<&mut Self>` になっているのは、この約束のためです。
- `Waker::noop()` は、何もしない Waker です。この実行係は、`Pending` が返っても**すぐに次の poll をする**ので、起こしてもらう必要がありません。
- 本物のランタイム（Tokio）は、こんなふうにぐるぐる poll し続けたりはしません。`Pending` を返した Future は脇に置いて、ほかの Future を進め、**Waker で起こされた Future だけ**をもう一度 poll します。だから、待っている Future がいくつあっても CPU を無駄に使いません。

解説4: async fn は、.await のところで区切られた状態機械になる

- 実行結果の「3 回目の poll」を見てください。1つ目の `Countdown` が `Ready` を返すと、`make_tea` は止まらずにそのまま次の行へ進み、2つ目の `Countdown` が `Pending` を返したところで、ようやく poll から戻っています。

- つまり、1回の poll は「次の `Pending` まで」進みます。`.await` は、そこで止まるかもしれない**区切り**です。
- `.await` の後でも使う変数（ここでは `first`）は、止まっている間も覚えておかなければならないので、状態機械の中に保存されます。次の例1-5で、それを Future の大きさで確かめます。

例1-5: .await をまたぐ変数は、Future の中にしまわれる

> **▼ このコードがやること（先に日本語で）**：1KB（1024 バイト）の配列を、`.await` **の後**でも使う関数と、`.await` **の前**に使い終わる関数を作ります。それぞれが返す Future の大きさを `size_of_val` で測って比べます。

```rust
use std::mem::size_of_val;
use std::time::Duration;

async fn wait() {
    tokio::time::sleep(Duration::from_millis(10)).await;
}

// 1KB の配列を作り、.await の「後」でも使う
async fn keeps_buffer() -> usize {
    let buffer = [1u8; 1024];
    wait().await;
    buffer.len()
}

// 1KB の配列は .await の「前」に使い終わる
async fn uses_buffer_before_await() -> usize {
    let len = {
        let buffer = [1u8; 1024];
        buffer.len()
    };
    wait().await;
    len
}

#[tokio::main]
async fn main() {
    let a = wait();
    let b = keeps_buffer();
    let c = uses_buffer_before_await();
    println!(
        "wait() の Future:                    {} バイト",
        size_of_val(&a)
    );
    println!(
        "keeps_buffer() の Future:            {} バイト",
        size_of_val(&b)
    );
    println!(
        "uses_buffer_before_await() の Future: {} バイト",
        size_of_val(&c)
    );
    println!("結果: {} {} {:?}", b.await, c.await, a.await);
}
```

`src/bin/u01_future_size.rs` として保存して実行します。

```
cargo run -q --bin u01_future_size
```

▼ 実行結果

```
wait() の Future:                    120 バイト
keeps_buffer() の Future:            1152 バイト
uses_buffer_before_await() の Future: 136 バイト
結果: 1024 1024 ()
```

▼ コードを1つずつ分解して解説

解説1: .await の後で使う配列は、Future の中に入る

- `keeps_buffer` の Future は 1152 バイトで、`wait()` の Future（120 バイト）より 1KB ほど大きくなっています。`.await` で止まっている間も、後で `buffer.len()` を使うために、配列を Future の中に取っておく必要があるからです。
- `uses_buffer_before_await` では、配列は `{ }` の中で使い終わって消えます。`.await` のときには配列はもう無いので、Future の中には `len`（数字1つ）しか入らず、136 バイトで済んでいます。
- 数字の細かい内訳（何バイトが何に使われるか）は、Rust の版によって変わることがあります。覚えるのは「**`.await` をまたいで使う変数は、Future の中にしまわれる**」という一点です。

解説2: この性質が、UNIT 02 のエラーの正体になる

- `.await` をまたいで持っている値は、Future の一部です。UNIT 02 では、Future を別のスレッドに渡して動かします。そのとき、Future の中に「スレッドをまたげない値」（第3巻 UNIT 10 の `Rc` など）がしまわれていると、コンパイルエラーになります。
- 大きな配列を持ったまま `.await` すると、Future も大きくなります。大きなデータは `Vec` や `Box` にすると、本体はヒープに置かれ、Future の中には小さな「持ち手」（`Vec` なら 24 バイト）だけが入ります。

> **ヒント:** `size_of_val` は、値がメモリの中で何バイトを使うかを返す関数です（`std::mem::size_of_val`）。ふだんのアプリで Future の大きさを気にすることはあまりありませんが、「`.await` をまたぐと何が起きるか」を目で見るにはちょうどよい道具です。

よくあるエラーと読み方

エラー1: main に async を付けただけ（E0752）

`#[tokio::main]` を書き忘れて、`main` に `async` だけを付けた場合です。

```
async fn main() {
    println!("こんにちは");
}
```

```
   Compiling async_lab v0.1.0 (...\async_lab)
error[E0752]: `main` function is not allowed to be `async`
 --> src\bin\e01_async_main.rs:1:1
  |
1 | async fn main() {
  | ^^^^^^^^^^^^^^^ `main` function is not allowed to be `async`

For more information about this error, try `rustc --explain E0752`.
error: could not compile `async_lab` (bin "e01_async_main") due to 1 previous error
```

- **何が起きたか:**「`main` 関数は `async` にできません」というエラーです。`async fn main` は Future を返しますが、`main` を呼ぶ OS は、その Future を poll する方法を知りません。
- **直し方:** `main` の上の行に `#[tokio::main]` を足します。これで、Tokio のランタイムが Future を動かしてくれる、ふつうの `main` に書き換わります。

エラー2: async ではない関数の中で .await（E0728）

```rust
use std::time::Duration;

// async が付いていない、ふつうの関数
fn load_note() -> String {
    tokio::time::sleep(Duration::from_millis(100)).await;
    String::from("ノート")
}

#[tokio::main]
async fn main() {
    println!("{}", load_note());
}
```

```
   Compiling async_lab v0.1.0 (...\async_lab)
error[E0728]: `await` is only allowed inside `async` functions and blocks
 --> src\bin\e01_await_in_sync.rs:5:52
  |
4 | fn load_note() -> String {
  | ------------------------ this is not `async`
5 |     tokio::time::sleep(Duration::from_millis(100)).await;
  |                                                    ^^^^^ only allowed inside
`async` functions and blocks

For more information about this error, try `rustc --explain E0728`.
error: could not compile `async_lab` (bin "e01_await_in_sync") due to 1 previous error
```

- **何が起きたか:**「`await` は `async` の関数かブロックの中でしか使えません」というエラーです。`this is not async` の線が、`async` が付いていない関数の行を指しています。
- **直し方:** `fn load_note()` を `async fn load_note()` にします。すると `load_note()` は Future を返すようになるので、呼ぶ側も `load_note().await` に直します。

> **ヒント:** 関数の中で `.await` を使うと、その関数も `async` にする必要があり、その関数を呼ぶ側でも `.await` が必要になり…と、`async` は呼び出し元へ向かって広がっていきます。いちばん外側は `#[tokio::main]` の `main` で受け止めます。同期の関数から async の処理をどうしても呼びたいときの方法と、その落とし穴は UNIT 02 で扱います。

エラー3: .await の付け忘れ（E0308）

```
async fn count_notes() -> u32 {
    3
}

#[tokio::main]
async fn main() {
    // .await を付け忘れた
    let count: u32 = count_notes();
    println!("ノートは {count} 件");
}
```

```
   Compiling async_lab v0.1.0 (...\async_lab)
error[E0308]: mismatched types
 --> src\bin\e01_forgot_await.rs:8:22
  |
8 |     let count: u32 = count_notes();
  |                ---   ^^^^^^^^^^^^^ expected `u32`, found future
  |                |
  |                expected due to this
  |
note: calling an async function returns a future
 --> src\bin\e01_forgot_await.rs:8:22
  |
8 |     let count: u32 = count_notes();
  |                      ^^^^^^^^^^^^^
help: consider `await`ing on the `Future`
  |
8 |     let count: u32 = count_notes().await;
  |                                   ++++++

For more information about this error, try `rustc --explain E0308`.
error: could not compile `async_lab` (bin "e01_forgot_await") due to 1 previous error
```

- **何が起きたか:** 「型が合いません。`u32` が来るはずなのに、future が来ました」というエラーです。`note:` の行が「async 関数を呼ぶと future が返ります」と、原因をそのまま教えてくれています。
- **直し方:** `help:` のとおり、`count_notes().await` にします。`++++++` の印は「ここに `.await` を足して」という意味です。

警告: 結果を使わない Future の .await 忘れ（unused implementer of Future）

いちばん危ないのは、エラーにならずに**ビルドが通ってしまう**場合です。

```
async fn save_note() {
    println!("保存しました");
}

#[tokio::main]
async fn main() {
    // .await を付け忘れた（結果も使っていない）
    save_note();
    println!("終わり");
}
```

```
   Compiling async_lab v0.1.0 (...\async_lab)
warning: unused implementer of `Future` that must be used
 --> src\bin\e01_unused_future.rs:8:5
  |
8 |     save_note();
  |     ^^^^^^^^^^^
  |
  = note: futures do nothing unless you `.await` or poll them
  = note: `#[warn(unused_must_use)]` (part of `#[warn(unused)]`) on by default

warning: `async_lab` (bin "e01_unused_future") generated 1 warning
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.70s
     Running `target\debug\e01_unused_future.exe`
終わり
```

- **何が起きたか:** プログラムは動きましたが、表示は「終わり」だけで、「**保存しました」が出ていません。**`save_note()` は Future を作っただけで、誰も `.await` しなかったので、中身が1行も動かずに捨てられました。`note: futures do nothing unless you .await or poll them`（Future は `.await` か poll しない限り何もしません）が、そのことを教えています。
- **直し方:** `save_note().await;` にします。
- **見落とさない工夫:** 警告は、ほかの出力に紛れて見落としがちです。`cargo clippy -- -D warnings` を使うと、警告を**エラーとして扱い**、ビルドを失敗させてくれます。

```
cargo clippy --bin e01_unused_future -- -D warnings
```

```
    Checking async_lab v0.1.0 (...\async_lab)
error: unused implementer of `std::future::Future` that must be used
 --> src\bin\e01_unused_future.rs:8:5
  |
8 |     save_note();
  |     ^^^^^^^^^^^
  |
  = note: futures do nothing unless you `.await` or poll them
  = note: `-D unused-must-use` implied by `-D warnings`
  = help: to override `-D warnings` add `#[allow(unused_must_use)]`

error: could not compile `async_lab` (bin "e01_unused_future") due to 1 previous error
```

> **注意:**「保存したつもりで保存されていない」は、本番では気づくのがとても遅れる種類のバグです。RustNote の自動チェック（ `Rust/rustnote/ci.yml` ）でも、`cargo clippy --all-targets --locked -- -D warnings` を実行して、警告が1つでもあれば失敗にしています。

RustNote ではここで使う

RustNote の入口 `Rust/rustnote/src/main.rs` は、この単元で学んだ形そのものです（説明に関係する部分を抜き出しています）。

```rust
#[tokio::main]
async fn main() -> anyhow::Result<()> {
    // .env が無くてもエラーにしない（本番では環境変数を直接設定する）
    dotenvy::dotenv().ok();

    let config = Config::from_env().context("設定を読み込めませんでした")?;

    //  （アセットの読み込みは省略）

    let db = Db::open(&config)
        .await
        .with_context(|| format!("DB（{}）を開けませんでした", config.database_url))?;
    let applied = db::migrate(&db)
        .await
        .context("マイグレーションに失敗しました")?;

    //  （起動のログとルーターの組み立ては省略）

    topcoat::start(router)
        .await
        .context("サーバーを起動できませんでした（HOST / PORT を確認してください）")?;
    Ok(())
}
```

- `#[tokio::main]` の `async fn main` で、Tokio のランタイムの上でアプリ全体を動かしています。
- `Db::open(&config).await` は「DB を開き終わるまで待つ」、`db::migrate(&db).await` は「表の準備が終わるまで待つ」です。どちらも前の結果を次で使うので、**順番に** `.await` しています（例1-2の解説4）。
- `.await` の後に続く `.context(...)?` は、「待ち終わって `Result` が出てきたら、エラーなら説明を付けて返す」という意味です。`.await` が先、`?` が後、の順番で読みます。
- 最後の `topcoat::start(router).await` は、**サーバーが止まるまで**終わりません。サーバーは、ここで待ちながら、届いたリクエストを1つずつ別の仕事（タスク）として動かし続けます。そのしくみが、次の UNIT 02 のテーマです。

ノートを読む処理（`src/repository/notes.rs` の `find`）も、`.await` の連続です。

```
async fn find(&self, owner: UserId, id: NoteId) -> Result<Option<Note>, RepoError> {
    let conn = self.db.connect().await?;
    let sql = format!("SELECT {NOTE_COLUMNS} FROM notes WHERE id = ?1 AND user_id = ?
2");
    let mut rows = conn
        .query(&sql, libsql::params![id.get(), owner.get()])
        .await?;
    match rows.next().await? {
        Some(row) => Ok(Some(row_to_note(&row)?)),
        None => Ok(None),
    }
}
```

DB につなぐ・SQL を送る・1行目を受け取る、の3か所で `.await` しています。DB の返事を待っている間、同じサーバーの中では、ほかの人のリクエストが進んでいます。これが、RustNote が1つのプログラムで大勢の利用者をさばける理由です。

練習問題

問1: 次のプログラムを実行すると、何がどの順番で表示されるでしょうか。実行する前に予想してください（`src/bin/u01_q1.rs`）。

```
// 番号を受け取り、始めたことを表示して、10 倍した値を返す
async fn job(number: u32) -> u32 {
    println!("  仕事{number} を始めます");
    number * 10
}

#[tokio::main]
async fn main() {
    let first = job(1);
    let second = job(2);
    println!("札を2枚作りました");
    // 2枚目の札から先に .await する
    let b = second.await;
    let a = first.await;
    println!("結果: {a} {b}");
}
```

問2: 次のプログラムは、0.9 秒かかります。3つの問い合わせはお互いに関係がありません。`tokio::join!` を使って速くしてください。書き換えた後、全体で何秒になるかも予想してください（`src/bin/u01_q2.rs`）。

```rust
use std::time::{Duration, Instant};

// DB への問い合わせの代わり: 指定した時間だけ待ってから、名前を返す
async fn load(name: &str, millis: u64) -> String {
    tokio::time::sleep(Duration::from_millis(millis)).await;
    name.to_string()
}

#[tokio::main]
async fn main() {
    let start = Instant::now();
    let user = load("ユーザー", 200).await;
    let notes = load("ノート", 300).await;
    let tags = load("タグ", 400).await;
    println!(
        "{user}・{notes}・{tags} / {:.1} 秒",
        start.elapsed().as_secs_f64()
    );
}
```

```
ユーザー・ノート・タグ / 0.9 秒
```

問3: `tokio::try_join!` に渡した2つの Future が、**両方とも**失敗します。`load_user` は 0.3 秒後に、`count_notes` は 0.1 秒後に失敗します。表示される結果と時間を予想してください（`src/bin/u01_q3.rs`）。

```rust
use std::time::{Duration, Instant};

// 0.3 秒後に失敗する
async fn load_user() -> Result<String, String> {
    tokio::time::sleep(Duration::from_millis(300)).await;
    Err(String::from("ユーザーを読めません"))
}

// 0.1 秒後に失敗する
async fn count_notes() -> Result<u32, String> {
    tokio::time::sleep(Duration::from_millis(100)).await;
    Err(String::from("ノートを数えられません"))
}

#[tokio::main]
async fn main() {
    let start = Instant::now();
    let result = tokio::try_join!(load_user(), count_notes());
    println!("{result:?} / {:.1} 秒", start.elapsed().as_secs_f64());
}
```

解答と解説

問1の解答:

```
札を2枚作りました
  仕事2 を始めます
  仕事1 を始めます
結果: 10 20
```

- `job(1)` と `job(2)` を呼んだ時点では、どちらの中身も動いていません。だから「札を2枚作りました」が最初です。
- 中身が動くのは `.await` したときなので、先に `.await` した `second`（仕事2）が先に始まります。**呼んだ順番ではなく、`.await` した順番**で動く、というのがポイントです。
- `a` は `first`（仕事1）の結果の 10、`b` は `second` の結果の 20 です。

問2の解答:

```rust
use std::time::{Duration, Instant};

// DB への問い合わせの代わり: 指定した時間だけ待ってから、名前を返す
async fn load(name: &str, millis: u64) -> String {
    tokio::time::sleep(Duration::from_millis(millis)).await;
    name.to_string()
}

#[tokio::main]
async fn main() {
    let start = Instant::now();
    // 3つの待ち時間を重ねる。全体の時間は、いちばん長い 400 ミリ秒で決まる
    let (user, notes, tags) = tokio::join!(
        load("ユーザー", 200),
        load("ノート", 300),
        load("タグ", 400)
    );
    println!(
        "{user}・{notes}・{tags} / {:.1} 秒",
        start.elapsed().as_secs_f64()
    );
}
```

`src/bin/u01_q2_answer.rs` として保存して実行した結果です。

```
ユーザー・ノート・タグ / 0.4 秒
```

- 順番に待つと 0.2 + 0.3 + 0.4 = 0.9 秒、`join!` で重ねると、いちばん長い 0.4 秒です。

- `tokio::join!` の結果は、渡した順番どおりのタプル `(user, notes, tags)` で返ります。終わった順番（ユーザー → ノート → タグ）ではなく、**書いた順番**です。

問3の解答:

```
Err("ノートを数えられません") / 0.1 秒
```

- `try_join!` は、**最初に起きた**失敗を返します。0.1 秒の時点で `count_notes` が失敗したので、その `Err` がすぐに返りました。`load_user` は途中で捨てられ、0.3 秒の失敗は起きる前に終わっています。
- 書いた順番（`load_user` が先）ではなく、**時間の順番**で決まる点に注意しましょう。どちらのエラーが返るかは、それぞれの待ち時間しだいで変わります。

まとめ

- `async fn` を呼ぶと Future（仕事の札）が返るだけで、中身は `.await` したときに初めて動く。Future は poll されて `Pending` と `Ready` を返しながら進み、Waker で起こされる。
- お互いに関係のない待ちは `tokio::join!` / `tokio::try_join!` で重ねると、全体の時間が「足し算」から「いちばん長いもの」になる。これは並行で、CPU を2つ使う並列ではない。
- `.await` の付け忘れはエラーにならず警告だけのことがあるので、`cargo clippy -- -D warnings` で止める。

UNIT 02: Tokio のランタイム・spawn・JoinHandle

この単元のゴール

- Tokio の**ランタイム**が持つ係（仕事を割り振るスケジューラー・タイマー・I/O）と、`#[tokio::main]` が中でしていることを説明できる
- `tokio::spawn` で**タスク**を作り、`JoinHandle`（ジョインハンドル）で結果を受け取れる。タスクが何本ものスレッドで同時に動くことを、実行結果で確かめられる
- タスクの中で起きた panic と、`abort` による取り消しを、`JoinError` で見分けられる
- `tokio::spawn` に渡す仕事が `'static` と `Send` でなければならない理由を説明し、E0373 と「future cannot be sent between threads safely」のエラーを直せる
- ランタイムの中で `block_on` を呼んではいけない理由を説明できる

たとえ話でつかむ

UNIT 01 のラーメン屋は、店員が1人でした。今度は、料理人が何人もいる大きな厨房を考えます。

- 厨房全体が**ランタイム**です。料理人の1人1人が、**ワーカースレッド**（仕事を進めるスレッド）です。
- 注文の伝票を厨房の「注文レール」に掛けることが、`tokio::spawn` です。レールに掛かった伝票1枚1枚が**タスク**です。

- 伝票を掛けた人には、**引換券**（番号札）が渡されます。これが `JoinHandle` です。引換券を出せば、できあがった料理を受け取れます（`.await`）。
- 手の空いた料理人は、レールから次の伝票を取ります。自分の分が無くなったら、ほかの料理人の伝票も手伝います。麺をゆでる間（`Pending`）は、伝票を脇に置いて別の伝票を進めます。
- 店じまい（`main` の終わり）になると、レールに残った伝票は、途中でも全部捨てられます。

伝票は、どの料理人が手に取るか分かりません。途中で別の料理人に渡ることもあります。だから、伝票に添える物には2つの約束があります。

- **手渡ししてよい物（`Send`）であること。**第3巻 UNIT 10 で学んだ `Rc` のように「1人の手の中でしか使えない物」は添えられません。
- **店が開いている間ずっと使える物（`'static`）であること。**「隣の人から借りた物」を添えると、料理ができる前に貸し主が帰ってしまうかもしれません。借り物ではなく、自分の物（所有している値）を添えます。

> **用語:** タスク（task：ランタイムの上で動く、1つの独立した仕事。`tokio::spawn` で作る。中身は1つの Future）

> **用語:** ワーカースレッド（worker thread：ランタイムが用意した、タスクを poll して進めるスレッド。ふつうは CPU のコア数と同じ本数）

> **用語:** JoinHandle（ジョインハンドル： `tokio::spawn` が返す引換券。 `.await` すると、タスクの戻り値を `Result` に包んで受け取れる）

図で理解する

Tokio のランタイムの中は、次のようになっています。

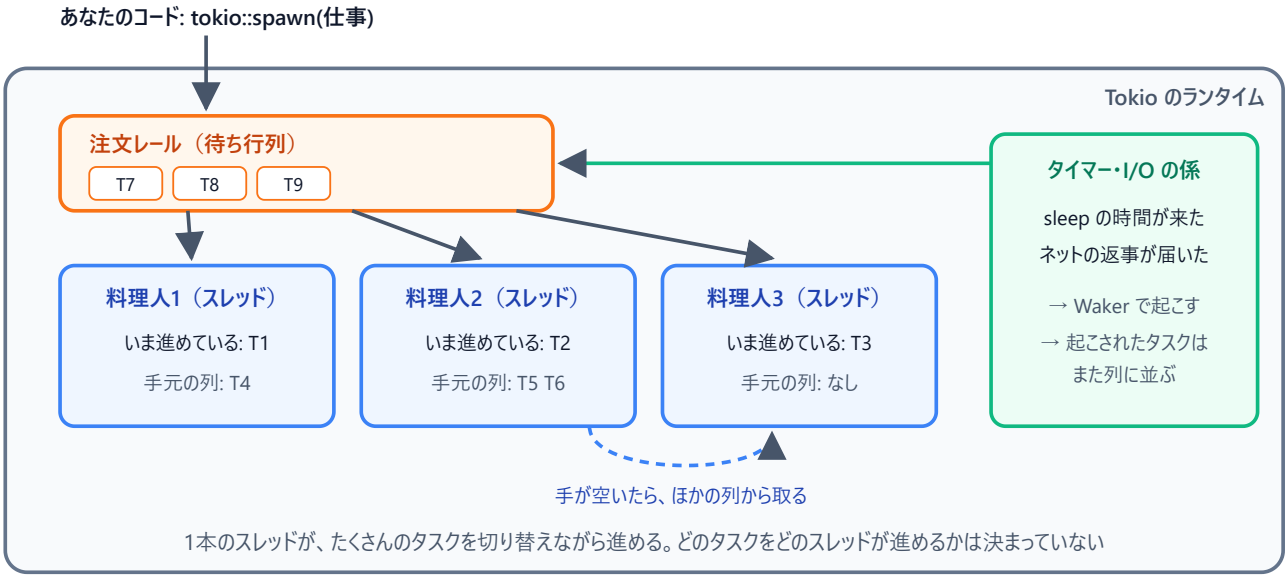


図5-6: Tokio のランタイムの中。spawn したタスクは待ち行列に並び、ワーカースレッドが取り出して poll する。待ちが終わったタスクは、タイマーや I/O の係に起こされて、また列に並ぶ

- **スケジューラー**（仕事の割り振り係）が、待ち行列に並んだタスクを、ワーカースレッドに配ります。手が空いたスレッドは、ほかのスレッドの列からもタスクを取ります。
- **タイマーと I/O の係**は、`sleep` の時間が来たときや、ネットワークの返事が届いたときに、待っていたタスクを Waker で起こします（UNIT 01 の図の④）。
- タスクは、`.await` で止まるたびに、**別のスレッドで続きが動くことがあります**。これが、タスクに `Send` が要る理由です。

次は、1つのタスクが作られてから、結果が届くまでの道すじです。

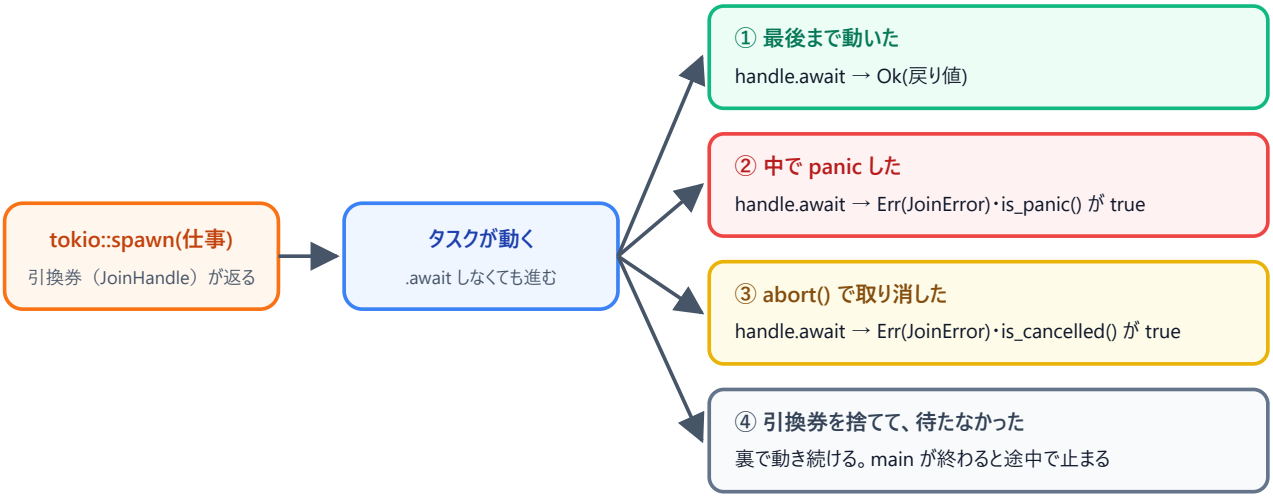


図5-7: タスクの一生。引換券を .await すると、最後まで動いたなら Ok、panic や取り消しなら JoinError が返る。引換券を捨てたタスクは裏で動き続けるが、main が終わると途中で止まる

UNIT 01 の Future と、この単元のタスクには、大きな違いが1つあります。

- **Future は、`.await` するまで動きません**（UNIT 01 の例1-1）。
- **タスクは、`tokio::spawn` した瞬間から動き始めます。**引換券を `.await` するのは「結果を受け取るため」で、「動かすため」ではありません。

第3巻 UNIT 10 で学んだ `std::thread::spawn`（OS のスレッド）と、この単元の `tokio::spawn`（Tokio のタスク）を比べると、次のようになります。

	`std::thread::spawn`（第3巻）	`tokio::spawn`（この単元）
作るもの	OS のスレッド	Tokio のタスク（中身は1つの Future）
1つあたりの重さ	重い。Rust の標準では、1本ごとに 2MB のスタックを用意する	軽い。Future の大きさ分（UNIT 01 の例1-5）だけ
待つとき	`thread::sleep` でスレッドごと眠る	`.await` で席を譲り、スレッドはほかのタスクを進める
結果の受け取り	`handle.join()`	`handle.await`
たくさん作ると	本数が増えるほど、メモリと切り替えの手間が重くなる	10万個でも動く（例2-5）
向いている仕事	長い計算・同期の処理	待ち時間の多い仕事（DB・ネットワーク）

まず動かす最小コード

例2-1: tokio::spawn でタスクを3つ作り、引換券で結果を受け取る

> **▼ このコードがやること（先に日本語で）**：0.3 秒・0.1 秒・0.2 秒待つタスクを3つ作ります。それぞれ、終わったときに「何秒の時点で、どのスレッドで終わったか」を表示します。その後、作った順番に引換券を `.await` して、戻り値を受け取ります。

```rust
use std::time::{Duration, Instant};

// 1つのタスクの仕事: 指定の時間だけ待ち、終わったら「どのスレッドで動いたか」を表示する
async fn work(number: u32, millis: u64, start: Instant) -> u32 {
    tokio::time::sleep(Duration::from_millis(millis)).await;
    let thread = std::thread::current();
    println!(
        "  タスク{number} が終わりました ({:.1} 秒 / {:?}) ",
        start.elapsed().as_secs_f64(),
        thread.id()
    );
    number * 100
}

#[tokio::main]
async fn main() {
    let start = Instant::now();
    // tokio::spawn でタスクを作ると、すぐに裏で動き始める。手元には JoinHandle（引換券）が残る
    let first = tokio::spawn(work(1, 300, start));
    let second = tokio::spawn(work(2, 100, start));
    let third = tokio::spawn(work(3, 200, start));
    println!("3つのタスクを作りました");

    // 引換券を .await すると、そのタスクの戻り値を受け取れる
    for handle in [first, second, third] {
        let value = handle.await.expect("タスクが panic しました");
        println!("受け取った値: {value}");
    }
    println!("合計 {:.1} 秒", start.elapsed().as_secs_f64());
}
```

`src/bin/u02_spawn.rs` として保存して実行します。

```
cargo run -q --bin u02_spawn
```

▼ 実行結果

```
3つのタスクを作りました
  タスク2 が終わりました (0.1 秒 / ThreadId(13))
  タスク3 が終わりました (0.2 秒 / ThreadId(13))
  タスク1 が終わりました (0.3 秒 / ThreadId(13))
受け取った値: 100
受け取った値: 200
受け取った値: 300
合計 0.3 秒
```

▼ コードを1つずつ分解して解説

解説1: spawn したタスクは、すぐに動き始める

- `tokio::spawn(work(1, 300, start))` は、`work(...)` が作った Future をタスクとして待ち行列に入れ、引換券（`JoinHandle<u32>`）をすぐに返します。`spawn` そのものは待ちません。
- 3つのタスクは、引換券を `.await` する前から動いています。「タスク2が終わりました」が 0.1 秒の時点で表示されているのは、そのためです。終わった順番は、待ち時間の短い順（2 → 3 → 1）です。
- 全体は、いちばん長い 0.3 秒で終わっています。UNIT 01 の `join!` と同じく、3つの待ちが重なりました。

解説2: 引換券の .await は、結果を受け取るため

- `for handle in [first, second, third]` で、作った順番に `.await` しています。タスク1（0.3 秒）を待っている間に、タスク2と3はとっくに終わっていますが、その結果はタスクの中に取っておかれます。
- だから「受け取った値」は、終わった順番ではなく、`.await` した順番（100 → 200 → 300）で表示されます。
- `handle.await` の結果は `Result<u32, JoinError>` です。タスクが panic していなければ `Ok`、panic していたら `Err` です。ここでは `expect` で中身を取り出しています（`Err` のときの例は例2-3）。

解説3: ThreadId は「どのスレッドで動いたか」の番号

- `std::thread::current().id()` は、今このコードを動かしているスレッドの番号です。
- この実行では、3つとも `ThreadId(13)` で終わりました。3つのタスクはほとんど「待っているだけ」なので、1本のスレッドで十分にさばけたのです。実行するたびに番号は変わり、別々のスレッドに分かれることもあります。

> **ヒント:** 同じ時刻に起こされたタスクが複数あると、それらは別々のスレッドで**本当に同時に**（並列に）動くことがあります。そのとき、どちらの `println!` が先に表示されるかは決まっていません。この巻の実行結果で、同じ待ち時間のタスクの表示の順番が教材と入れ替わるのは、このためです。

例2-2: #[tokio::main] を使わずに、ランタイムを自分で組み立てる

> **▼ このコードがやること（先に日本語で）：** `#[tokio::main]` を使わずに、ワーカースレッドが2本のランタイムを自分で作ります。その上で 0.1 秒待つタスクを4つ動かし、それぞれがどのスレッドで終わったかを表示します。最後に、4つの戻り値の合計を表示します。

```rust
use std::time::Duration;

// #[tokio::main] を使わずに、ランタイムを自分で組み立てる
fn main() -> std::io::Result<()> {
    let runtime = tokio::runtime::Builder::new_multi_thread()
        // ワーカースレッドを2本にする（既定は CPU のコア数）
        .worker_threads(2)
        // 時間（sleep・timeout）と I/O（ネットワーク）の機能を有効にする
        .enable_all()
        .build()?;

    // block_on: ここで渡した async ブロックが終わるまで、main のスレッドは待つ
    let total = runtime.block_on(async {
        let mut handles = Vec::new();
        for number in 1..=4 {
            handles.push(tokio::spawn(async move {
                tokio::time::sleep(Duration::from_millis(100)).await;
                let thread = std::thread::current();
                println!(
                    "タスク{number}: {} {:?}",
                    thread.name().unwrap_or("?"),
                    thread.id()
                );
                number * 10
            }));
        }
        let mut total = 0;
        for handle in handles {
            total += handle.await.unwrap();
        }
        total
    });
    println!("合計: {total}");
    Ok(())
}
```

`src/bin/u02_runtime.rs` として保存して実行します。

```
cargo run -q --bin u02_runtime
```

▼ 実行結果

```
タスク3: tokio-rt-worker ThreadId(3)
タスク2: tokio-rt-worker ThreadId(3)
タスク1: tokio-rt-worker ThreadId(3)
タスク4: tokio-rt-worker ThreadId(2)
合計: 100
```

▼ コードを1つずつ分解して解説

解説1: Builder で、ランタイムの形を決める

- `tokio::runtime::Builder::new_multi_thread()` は、「ワーカースレッドを何本も使うランタイム」を作る準備です。
- `.worker_threads(2)` で、ワーカースレッドの本数を2本にしています。実行結果に `ThreadId(3)` と `ThreadId(2)` の2種類しか出てこないのは、そのためです。指定しなければ、CPU のコア数と同じ本数になります。
- `.enable_all()` は、タイマー（`sleep` など）と I/O（ネットワーク）の係を有効にします。これを忘れると、`sleep` を使ったところで `A Tokio 1.x context was found, but timers are disabled.`（タイマーが有効になっていません）という panic になります。
- `.build()?` でランタイムができあがります。作るのに失敗することもあるので `Result` が返り、`?` で `main` のエラーとして返しています。

解説2: block_on は、同期の世界と async の世界の境目

- `runtime.block_on(Future)` は、「この Future が終わるまで、今のスレッドで待ち続ける」関数です。`main` はふつうの関数（同期の世界）なので、ここで async の世界に入ります。
- `block_on` が返した値（ここでは合計の `100`）は、ふつうの値として `main` で使えます。
- `#[tokio::main]` は、ちょうどこの形を自動で書いてくれる印です。`#[tokio::main] async fn main() { 中身 }` は、おおよそ次のように書き換えられます。

```
fn main() {
    tokio::runtime::Builder::new_multi_thread()
        .enable_all()
        .build()
        .unwrap()
        .block_on(async { /* 中身 */ })
}
```

解説3: スレッドの名前は tokio-rt-worker

- `thread.name()` は、スレッドの名前です。Tokio のワーカースレッドには `tokio-rt-worker` という名前が付いています。panic したときのメッセージにも、この名前が出てきます（例2-3）。
- 4つのタスクは、どれも 0.1 秒後に同時に起こされたので、表示の順番（3 → 2 → 1 → 4）は実行するたびに変わります。

> **ヒント:** `#[tokio::main(flavor = "current_thread")]` と書くと、ワーカースレッドを作らず、`main` のスレッド1本だけでタスクを動かすランタイムになります。テスト用の `#[tokio::test]` は、何も指定しなければこの1本の形です。並行には動きますが、並列にはなりません。

もう一歩: 実務に近い形

例2-3: タスクの中の panic と、取り消し（abort）を見分ける

> **▼ このコードがやること（先に日本語で）:** ①では、タスクの中で panic を起こします。②では、10 秒かかるタスクを作ってすぐに `abort`（取り消し）します。どちらも引換券を `.await` して、返ってきた `JoinError` が panic なのか取り消しなのかを調べます。最後に、`main` が最後まで動いたことを表示します。

```
use std::time::Duration;

#[tokio::main]
async fn main() {
    // ① タスクの中で panic する
    let panicking = tokio::spawn(async {
        let notes: Vec<&str> = Vec::new();
        // 空の Vec の 0 番目を読もうとして panic する
        notes[0].to_string()
    });
    match panicking.await {
        Ok(value) => println!("① 成功: {value}"),
        Err(error) => println!("① 失敗: {error} / is_panic = {}", error.is_panic()),
    }

    // ② 終わる前に abort（取り消し）する
    let slow = tokio::spawn(async {
        tokio::time::sleep(Duration::from_secs(10)).await;
        "10 秒の仕事が終わった"
    });
    slow.abort();
    match slow.await {
        Ok(value) => println!("② 成功: {value}"),
        Err(error) => println!("② 失敗: {error} / is_cancelled = {}",
error.is_cancelled()),
    }

    println!("main は最後まで動きました");
}
```

`src/bin/u02_join_error.rs` として保存して実行します。

```
cargo run -q --bin u02_join_error
```

▼ 実行結果

```
thread 'tokio-rt-worker' (18992) panicked at src\bin\u02_join_error.rs:9:14:
index out of bounds: the len is 0 but the index is 0
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
① 失敗: task 13 panicked with message "index out of bounds: the len is 0 but the index
is 0" / is_panic = true
② 失敗: task 14 was cancelled / is_cancelled = true
main は最後まで動きました
```

▼ コードを1つずつ分解して解説

解説1: タスクの panic は、そのタスクの中で止まる

- 最初の3行は、panic したときに Rust が表示するメッセージです。`thread 'tokio-rt-worker' (18992) panicked at ...:9:14:` は「ワーカースレッドの1本が、9行目14文字目で panic しました」という意味です。かっこの中の数字は OS のスレッドの番号で、実行するたびに変わります。
- 大事なのは、その後も `main` が動き続けていることです。Tokio は、タスクの中の panic を受け止めて、そのタスクだけを終わらせ、引換券に「panic しました」という `JoinError` を渡します。
- サーバーで言えば、1つのリクエストの処理で panic が起きても、サーバー全体は落ちずに、ほかの人のリクエストは処理され続けます。

解説2: JoinError の is_panic と is_cancelled

- `handle.await` が `Err(error)` を返したら、`error.is_panic()` で「panic したのか」、`error.is_cancelled()` で「取り消されたのか」を調べられます。
- `{error}` で表示すると、`task 13 panicked with message "..."` や `task 14 was cancelled` のように、タスクの番号と理由が出ます。この番号も、実行するたびに変わることがあります。

解説3: abort は、次の .await の区切りで止める

- `slow.abort()` は、タスクに「取り消し」を伝えます。タスクは、次に poll されるとき（`.await` の区切り）で止まり、中の Future は drop されます。10 秒待たずに、すぐ `is_cancelled = true` が返りました。
- 取り消しは、`.await` の区切りでしか起きません。`.await` の無い長い計算の途中では止まらないことに注意しましょう（UNIT 06 と UNIT 07 でくわしく扱います）。

> **注意:** panic が「サーバー全体は落ちない」からといって、panic に頼ってエラーを扱うのはやめましょう。panic したタスクで途中まで書き換えていたデータは、そのまま残ります。予想できる失敗（入力の間違い・ノートが無いなど）は、`Result` で返すのが基本です（第4巻 UNIT 05）。

例2-4: 引換券を捨てたタスクは、main が終わると途中で止まる

> **▼ このコードがやること（先に日本語で）**：0.1 秒ごとに5回表示する「裏のタスク」を作り、引換券は受け取らずに捨てます。`main` は 0.25 秒だけ待ってから終わります。裏のタスクが最後まで動くかどうかを確かめます。

```
use std::time::Duration;

#[tokio::main]
async fn main() {
    // JoinHandle を捨てて、結果を待たないタスク（切り離されたタスク）を作る
    tokio::spawn(async {
        for step in 1..=5 {
            println!("  裏のタスク: {step} 回目");
            tokio::time::sleep(Duration::from_millis(100)).await;
        }
        println!("  裏のタスク: 最後まで終わりました");
    });

    tokio::time::sleep(Duration::from_millis(250)).await;
    println!("main が終わります");
}
```

`src/bin/u02_detached.rs` として保存して実行します。

```
cargo run -q --bin u02_detached
```

▼ 実行結果

```
  裏のタスク: 1 回目
  裏のタスク: 2 回目
  裏のタスク: 3 回目
main が終わります
```

▼ コードを1つずつ分解して解説

解説1: 引換券を捨てても、タスクは止まらない

- `tokio::spawn(...)` の戻り値（引換券）を変数に入れずに捨てています。それでもタスクは、裏で動き続けます。引換券を捨てても、タスクは取り消されません（`abort` とは違います）。
- このような、誰も結果を待っていないタスクを「切り離されたタスク」（detached task）と呼びます。

解説2: main が終わると、ランタイムごと片付けられる

- 0.25 秒の時点で `main` が終わると、ランタイムが片付けられ、残っていたタスクは**途中でも全部捨てられます**。だから「4 回目」以降も、「最後まで終わりました」も表示されていません。
- 裏で「メールを送る」「ログを保存する」のような仕事をさせていたら、その仕事は黙って失われます。エラーも出ません。

> **注意:** 「最後までやり遂げてほしい仕事」を、引換券を捨てた `tokio::spawn` に任せてはいけません。終わるまで待つ方法は練習問題2で、サーバーを止めるときに処理中の仕事を待つ方法は UNIT 06 で、サーバーが落ちても仕事を失わない方法は UNIT 10 で学びます。

例2-5: タスクは軽い。10万個作っても約1秒

> **▼ このコードがやること（先に日本語で）:** 1 秒待つだけのタスクを10万個作り、作るのにかかった時間を表示します。その後、全部の引換券を `.await` して戻り値を足し合わせ、全体でかかった時間を表示します。

```
use std::time::{Duration, Instant};

#[tokio::main]
async fn main() {
    let start = Instant::now();
    let mut handles = Vec::new();
    // 1 秒待つだけのタスクを 10 万個作る
    for number in 0..100_000u64 {
        handles.push(tokio::spawn(async move {
            tokio::time::sleep(Duration::from_secs(1)).await;
            number
        }));
    }
    println!(
        "{} 個のタスクを作りました（{:.2} 秒）",
        handles.len(),
        start.elapsed().as_secs_f64()
    );

    // 全部のタスクの結果を受け取って、足し合わせる
    let mut total = 0;
    for handle in handles {
        total += handle.await.expect("タスクが panic しました");
    }
    println!(
        "全部終わりました: 合計 {total}（{:.2} 秒）",
        start.elapsed().as_secs_f64()
    );
}
```

`src/bin/u02_many_tasks.rs` として保存して実行します。

```
cargo run -q --bin u02_many_tasks
```

▼ 実行結果

```
100000 個のタスクを作りました（0.19 秒）
全部終わりました: 合計 4999950000（1.20 秒）
```

▼ コードを1つずつ分解して解説

解説1: 10万個の「待ち」が重なって、全体は約1秒

- 10万個のタスクを作るのに 0.19 秒、全部が終わるまでで 1.20 秒でした。1 秒の待ちが10万個重なったので、全体もほぼ 1 秒です。
- 同じことを OS のスレッド（`std::thread::spawn`）で10万本作ろうとすると、スタックの予約だけで 2MB × 10万 = 約200GB になります。OS のスレッドは、こんなに大量に作るための仕組みではありません。待ち時間の多い仕事を大量にさばくなら、タスクが向いています。
- 合計の 4999950000 は、0 から 99999 までを足した数です。全部のタスクの戻り値を、1つも取りこぼさずに受け取れています。

解説2: async move で、number をタスクへ移す

- `async move { ... }` の `move` は、「ブロックの中で使う外の変数を、借りるのではなく、タスクへ移す（所有権ごと渡す）」という意味です。`number` はループのたびに変わる変数なので、借りたままではタスクより先に消えてしまいます。
- `move` を付け忘れたときのエラーは、この後の「よくあるエラー1」で読みます。

> **注意:** 「タスクは軽い」とはいえ、外から来たリクエストの数だけ無制限にタスクを作ると、メモリや DB の接続が足りなくなります。同時に動かす数に上限を付ける方法は、UNIT 03 と UNIT 05 で学びます。

よくあるエラーと読み方

エラー1: move を付けずに外の変数を使う（E0373）

```
#[tokio::main]
async fn main() {
    let title = String::from("買い物メモ");
    // move を付けずに、外の変数 title を借りて使う
    let handle = tokio::spawn(async {
        println!("保存: {title}");
    });
    handle.await.unwrap();
}
```

```
   Compiling async_lab v0.1.0 (...\async_lab)
error[E0373]: async block may outlive the current function, but it borrows `title`,
which is owned by the current function
 --> src\bin\e02_spawn_borrow.rs:5:31
  |
5 |     let handle = tokio::spawn(async {
  |                               ^^^^^ may outlive borrowed value `title`
6 |         println!("保存: {title}");
  |                          ----- `title` is borrowed here
  |
  = note: async blocks are not executed immediately and must either take a reference or
ownership of outside variables they use
help: to force the async block to take ownership of `title` (and any other referenced
variables), use the `move` keyword
  |
5 |     let handle = tokio::spawn(async move {
  |                                     ++++

For more information about this error, try `rustc --explain E0373`.
error: could not compile `async_lab` (bin "e02_spawn_borrow") due to 1 previous error
```

- **何が起きたか:**「async ブロックは、今の関数より長生きするかもしれないのに、今の関数が持っている `title` を借りています」というエラーです。タスクは `main` とは別に動くので、`main` の `title` が先に消えても動き続けるかもしれません。借り物を持ったまま貸し主より長生きするのは、第3巻で学んだ借用のルールに反します。
- `tokio::spawn` は、渡す仕事に `'static`（「借り物を持っていない」、つまり、いつまでも使える）を求めます。エラー2の出力にある `F: Future + Send + 'static` が、その条件です。
- **直し方:** `help:` のとおり `async move` にして、`title` の持ち主をタスクへ移します。

```rust
#[tokio::main]
async fn main() {
    let title = String::from("買い物メモ");
    // move を付けて、title の持ち主をタスクへ移す
    let handle = tokio::spawn(async move {
        println!("保存: {title}");
    });
    handle.await.unwrap();
}
```

```
保存: 買い物メモ
```

（`src/bin/u02_fix_move.rs` として実行した結果です。）

エラー2: Rc を持ったまま .await する（future cannot be sent between threads safely）

```rust
use std::rc::Rc;
use std::time::Duration;

#[tokio::main]
async fn main() {
    let handle = tokio::spawn(async {
        // Rc はスレッドをまたいで渡せない（Send ではない）型
        let title = Rc::new(String::from("買い物メモ"));
        // Rc を持ったまま .await する
        tokio::time::sleep(Duration::from_millis(10)).await;
        println!("保存: {title}");
    });
    handle.await.unwrap();
}
```

```
   Compiling async_lab v0.1.0 (...\async_lab)
error: future cannot be sent between threads safely
   --> src\bin\e02_spawn_rc.rs:6:18
    |
  6 |       let handle = tokio::spawn(async {
    |  __________________^
  7 | |         // Rc はスレッドをまたいで渡せない（Send ではない）型
  8 | |         let title = Rc::new(String::from("買い物メモ"));
...   |
 11 | |         println!("保存: {title}");
 12 | |     });
    | |______^ future created by async block is not `Send`
    |
    = help: within `{async block@src\bin\e02_spawn_rc.rs:6:31: 6:36}`, the trait `Send`
is not implemented for `Rc<String>`
note: future is not `Send` as this value is used across an await
   --> src\bin\e02_spawn_rc.rs:10:55
    |
  8 |         let title = Rc::new(String::from("買い物メモ"));
    |             ----- has type `Rc<String>` which is not `Send`
  9 |         // Rc を持ったまま .await する
 10 |         tokio::time::sleep(Duration::from_millis(10)).await;
    |                                                       ^^^^^ await occurs here,
with `title` maybe used later
note: required by a bound in `tokio::spawn`
   --> ...\tokio-1.53.3\src\task\spawn.rs:176:21
    |
174 |     pub fn spawn<F>(future: F) -> JoinHandle<F::Output>
    |            ----- required by a bound in this function
175 |     where
176 |         F: Future + Send + 'static,
    |                     ^^^^ required by this bound in `spawn`

error: could not compile `async_lab` (bin "e02_spawn_rc") due to 1 previous error
```

エラーが長いので、上から順に区切って読みます。

- `future cannot be sent between threads safely` / `future created by async block is not Send`：「この Future は、スレッドの間で安全に受け渡せません。async ブロックが作った Future が `Send` ではありません」。
- `the trait Send is not implemented for Rc<String>`：原因の型は `Rc<String>` です。第3巻 UNIT 10 で学んだとおり、`Rc` は `Send` ではありません。
- `future is not Send as this value is used across an await`：「この値が `.await` をまたいで使われているので、Future が `Send` になりません」。`title` が8行目で作られ、10行目の `.await` の後（11行目）でも使われています。UNIT 01 の例1-5で見たとおり、`.await` をまたぐ変数は Future の中にしまわれるので、Future 全体が `Send` でなくなりました。

- `required by a bound in tokio::spawn` / `F: Future + Send + 'static`：`tokio::spawn` の定義で、渡す Future に `Send` が求められている、という出どころの説明です。タスクは `.await` のたびに別のスレッドで続きが動くかもしれないので、この条件があります。

直し方: スレッドをまたいで共有したいなら、`Rc` の代わりに `Arc` を使います（第3巻 UNIT 10）。

```
use std::sync::Arc;
use std::time::Duration;

#[tokio::main]
async fn main() {
    let handle = tokio::spawn(async {
        // Arc はスレッドをまたいで渡せる（Send な）型
        let title = Arc::new(String::from("買い物メモ"));
        tokio::time::sleep(Duration::from_millis(10)).await;
        println!("保存: {title}");
    });
    handle.await.unwrap();
}
```

```
保存: 買い物メモ
```

（`src/bin/u02_fix_rc.rs` として実行した結果です。）

> **ヒント**: そもそも共有する必要がないなら、ふつうの `String` で十分です。また、`Rc` を `.await` の**前**に使い終わって `{ }` の中で消しておけば、Future の中にしまわれないので、`Send` のエラーは出ません。「`.await` をまたいで持っている値は何か？」を確かめるのが、このエラーを読むコツです。UNIT 05 では、同じ形のエラーが `MutexGuard`（鍵）で起きます。

エラー3: ランタイムの中で block_on を呼ぶ（Cannot start a runtime from within a runtime）

コンパイルは通るのに、動かすと panic する例です。

```rust
// 同期の関数の中で、非同期の処理を「待ちきろう」とする
fn load_title() -> String {
    let runtime = tokio::runtime::Runtime::new().unwrap();
    runtime.block_on(async { String::from("買い物メモ") })
}

#[tokio::main]
async fn main() {
    // すでにランタイムの中（async fn main の中）から呼んでしまう
    let title = load_title();
    println!("{title}");
}
```

```
   Compiling async_lab v0.1.0 (...\async_lab)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.72s
     Running `target\debug\e02_block_on.exe`

thread 'main' (22388) panicked at ...\tokio-
1.53.3\src\runtime\scheduler\multi_thread\mod.rs:91:9:
Cannot start a runtime from within a runtime. This happens because a function (like
`block_on`) attempted to block the current thread while the thread is being used to
drive asynchronous tasks.
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
error: process didn't exit successfully: `target\debug\e02_block_on.exe` (exit code:
101)
```

- **何が起きたか:**「ランタイムの中で、ランタイムを始めることはできません。`block_on` のような関数が、async のタスクを動かしているスレッドを止めて待とうとしました」という panic です。
- `block_on` は「終わるまでこのスレッドで待ち続ける」関数でした（例2-2）。ワーカースレッドがそこで待ち続けると、そのスレッドで動くはずのほかのタスクが全部止まってしまいます。Tokio はそれを防ぐために、わざと panic して知らせています。
- 今動いているランタイムを借りる `tokio::runtime::Handle::current().block_on(...)` に書き換えても、同じ panic になります（`e02_block_on_handle.rs` で確かめた結果も、同じメッセージでした）。
- **直し方:** 待つ処理を含む関数は `async fn` にして、呼ぶ側で `.await` します。async の世界の中では、`block_on` を使いません。

```rust
// 待つ処理を含む関数は、async fn にして .await で待つ
async fn load_title() -> String {
    String::from("買い物メモ")
}

#[tokio::main]
async fn main() {
    let title = load_title().await;
    println!("{title}");
}
```

```
買い物メモ
```

（`src/bin/u02_fix_block_on.rs` として実行した結果です。）

> **ヒント:** `block_on` を使ってよいのは、`main` の入口（例2-2）のように「まだ async の世界の外にいる」場所だけです。逆に、async の世界から、時間のかかる同期の処理を呼びたいときは、`tokio::task::spawn_blocking` を使います（UNIT 07）。

RustNote ではここで使う

RustNote が使う Web フレームワーク Topcoat は、`topcoat::start(router).await` の中で、ブラウザからの**接続を1つ受け付けるたびに、タスクを1つ作って**います。Topcoat 0.10.0 のソース（`topcoat-router-0.10.0/src/service.rs` の `internal_serve`）から、その部分を抜き出すと次のとおりです（コメントは日本語に置き換え、関係の無い部分は省略しています）。

```rust
loop {
    // 新しい接続を待つ。止める合図（Ctrl+C）が来たら、ループを抜ける
    let accepted = tokio::select! {
        accepted = listener.accept() => accepted,
        () = &mut shutdown => break,
    };
    let (stream, remote_addr) = accepted?;
    // （省略）

    // 1つの接続を、1つのタスクで処理する
    tokio::spawn(async move {
        // （この接続のリクエストを読み、ルーターに渡して、答えを書き戻す。省略）
    });
}
```

- 接続ごとに `tokio::spawn` するので、Aさんのリクエストが DB の返事を待っている間も、Bさん・Cさんのリクエストは別のタスクとして進みます。どれかのタスクで panic が起きても、ほかの接続は影響を受けません（例2-3）。
- `tokio::select!` は「どちらか先に起きたほうを選ぶ」書き方で、UNIT 03 で学びます。止める合図を受けたときに、処理中の接続を待ってから終わる部分は、UNIT 06 で読みます。

タスクは、どのワーカースレッドで動くか分かりません。そのため、リクエストを処理するコードが使う共有の部品は、スレッドをまたいで使えなければなりません。RustNote の `AppState`（`src/state.rs`）が、保存先を次のように持っているのはそのためです。

```
#[derive(Clone)]
pub struct AppState {
    /// DB そのもの（GET /health の確認に使う）
    pub db: Db,
    pub users: Arc<dyn UserRepository + Send + Sync>,
    pub notes: Arc<dyn NoteRepository + Send + Sync>,
    pub sessions: Arc<dyn SessionRepository + Send + Sync>,
}
```

- `Arc` は、たくさんのタスクで同じものを共有するための入れ物です（第3巻 UNIT 10）。`Rc` では、エラー2と同じエラーになります。
- `dyn NoteRepository + Send + Sync` は、「中身は `NoteRepository` を実装した何かで、別のスレッドへ渡してよく（`Send`）、何本ものスレッドから同時に使ってよい（`Sync`）もの」という意味です（第4巻 UNIT 04 の `dyn`）。

もう1つ、パスワードを確かめる `src/auth.rs` の `verify_password` も、引換券の結果を受け取る形をしています。

```
    let joined = tokio::task::spawn_blocking(move || {
        // （パスワードとハッシュを照らし合わせる。省略）
    })
    .await;
    match joined {
        Ok(Ok(matches)) => Ok(matches),
        Ok(Err(error)) => Err(AppError::internal(error)),
        Err(error) => Err(AppError::internal(error)),
    }
```

`spawn_blocking` は UNIT 07 で学ぶ「重い計算用の spawn」です。引換券を `.await` した結果が `Result` で、その中身もまた `Result` なので、`Ok(Ok(..))`・`Ok(Err(..))`・`Err(..)` の3つに分けています。この形は、練習問題3で自分でも書きます。

練習問題

問1: 次のプログラムは、コンパイルエラー（E0373）になります。「hanako さんのノート1」〜「3」を表示し、最後に「持ち主: hanako」と表示するように直してください（`src/bin/u02_q1.rs`）。

```
#[tokio::main]
async fn main() {
    let owner = String::from("hanako");
    let mut handles = Vec::new();
    for number in 1..=3 {
        handles.push(tokio::spawn(async {
            format!("{owner} さんのノート{number}")
        }));
    }
    for handle in handles {
        println!("{}", handle.await.unwrap());
    }
}
```

> **ヒント:** `async move` にするだけでは、別のエラー（E0382）になります。ループの1回目で `owner` がタスクへ移ってしまい、2回目に使えなくなるからです。

問2: 例2-4の `u02_detached.rs` を書き換えて、裏のタスクが「最後まで終わりました」と表示してから、`main` が終わるようにしてください（`src/bin/u02_q2.rs`）。

問3: 次の `load_user` を、ユーザー 1・2・3 のそれぞれについて `tokio::spawn` で動かします。引換券を `.await` した結果を、「成功」「アプリのエラー」「タスクが失敗（panic）」の3つに分けて表示してください。`handle.await` の型が何になるかも考えましょう（`src/bin/u02_q3.rs`）。

```
use std::time::Duration;

// 1 なら成功、2 ならアプリのエラー、それ以外は panic する
async fn load_user(user_id: i64) -> Result<String, String> {
    tokio::time::sleep(Duration::from_millis(100)).await;
    match user_id {
        1 => Ok(String::from("hanako")),
        2 => Err(format!("ユーザー {user_id} はいません")),
        _ => panic!("ユーザー {user_id} で想定外のことが起きました"),
    }
}
```

解答と解説

問1の解答:

```rust
#[tokio::main]
async fn main() {
    let owner = String::from("hanako");
    let mut handles = Vec::new();
    for number in 1..=3 {
        // タスクごとに自分用の写しを作り、それを move で持っていく
        let owner = owner.clone();
        handles.push(tokio::spawn(async move {
            format!("{owner} さんのノート{number}")
        }));
    }
    for handle in handles {
        println!("{}", handle.await.unwrap());
    }
    // 元の owner は、まだ main で使える
    println!("持ち主: {owner}");
}
```

```
hanako さんのノート1
hanako さんのノート2
hanako さんのノート3
持ち主: hanako
```

- 元のコードのエラーは、`owner` と `number` の2つについて E0373 が出ます（どちらも借りているため）。
- ヒントのとおり、`async move` にするだけだと、次のエラーになります。

```
   Compiling async_lab v0.1.0 (...\async_lab)
error[E0382]: use of moved value: `owner`
 --> src\bin\e02_q1_move_only.rs:6:35
  |
3 |     let owner = String::from("hanako");
  |         ----- move occurs because `owner` has type `String`, which does not
implement the `Copy` trait
4 |     let mut handles = Vec::new();
5 |     for number in 1..=3 {
  |     ------------------ inside of this loop
6 |         handles.push(tokio::spawn(async move {
  |                                   ^^^^^^^^^^ value moved here, in previous
iteration of loop
7 |             format!("{owner} さんのノート{number}")
  |                      ----- use occurs due to use in coroutine
  |
help: consider cloning the value before moving it into the closure
  |
6 ~         let value = owner.clone();
7 ~         handles.push(tokio::spawn(async move {
8 ~             format!("{value} さんのノート{number}")
  |

For more information about this error, try `rustc --explain E0382`.
error: could not compile `async_lab` (bin "e02_q1_move_only") due to 1 previous error
```

- `value moved here, in previous iteration of loop` は「ループの前の回で、もう移してしまいました」という意味です。`coroutine`（コルーチン）は、async ブロックのコンパイラの中での呼び名です。
- `help:` のとおり、`move` の前に `clone()` で写しを作り、写しのほうをタスクへ移します。解答では、写しにも同じ `owner` という名前を付けて（シャドーイング：第2巻 UNIT 01）、`format!` の中を書き換えずに済ませています。`number` は `Copy` な整数なので、`move` だけで写されます。
- 長い文字列や大きなデータを何十個ものタスクで共有するなら、`clone` で毎回写す代わりに、`Arc<String>` にして `Arc::clone` で「持ち手」だけを増やす方法もあります（第3巻 UNIT 10）。

問2の解答:

```rust
use std::time::Duration;

#[tokio::main]
async fn main() {
    // JoinHandle を変数に取っておく
    let background = tokio::spawn(async {
        for step in 1..=5 {
            println!("  裏のタスク: {step} 回目");
            tokio::time::sleep(Duration::from_millis(100)).await;
        }
        println!("  裏のタスク: 最後まで終わりました");
    });

    tokio::time::sleep(Duration::from_millis(250)).await;
    // main を終える前に、裏のタスクが終わるのを待つ
    background.await.expect("裏のタスクが panic しました");
    println!("main が終わります");
}
```

```
  裏のタスク: 1 回目
  裏のタスク: 2 回目
  裏のタスク: 3 回目
  裏のタスク: 4 回目
  裏のタスク: 5 回目
  裏のタスク: 最後まで終わりました
main が終わります
```

- 引換券を捨てずに `background` に取っておき、`main` の最後で `.await` しました。`main` は、裏のタスクが終わるまでそこで待つので、5回目まで表示されます。
- `.await` の位置が 0.25 秒の `sleep` の後でも、裏のタスクはその間ずっと動いています。引換券の `.await` は「結果を受け取るため」（解説2）だからです。

問3の解答:

```rust
use std::time::Duration;

// 1 なら成功、2 ならアプリのエラー、それ以外は panic する
async fn load_user(user_id: i64) -> Result<String, String> {
    tokio::time::sleep(Duration::from_millis(100)).await;
    match user_id {
        1 => Ok(String::from("hanako")),
        2 => Err(format!("ユーザー {user_id} はいません")),
        _ => panic!("ユーザー {user_id} で想定外のことが起きました"),
    }
}

#[tokio::main]
async fn main() {
    for user_id in 1..=3 {
        let handle = tokio::spawn(load_user(user_id));
        // handle.await の型は Result<Result<String, String>, JoinError>
        match handle.await {
            Ok(Ok(name)) => println!("{user_id}: 成功 {name}"),
            Ok(Err(message)) => println!("{user_id}: アプリのエラー {message}"),
            Err(join_error) => println!("{user_id}: タスクが失敗 {join_error}"),
        }
    }
}
```

```
1: 成功 hanako
2: アプリのエラー ユーザー 2 はいません

thread 'tokio-rt-worker' (29704) panicked at src\bin\u02_q3.rs:9:14:
ユーザー 3 で想定外のことが起きました
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
3: タスクが失敗 task 15 panicked with message "ユーザー 3 で想定外のことが起きました"
```

- `handle.await` の型は `Result<Result<String, String>, JoinError>` です。外側の `Result` は「タスクが最後まで動いたか」、内側の `Result` は「`load_user` の処理が成功したか」を表します。
- `match` で `Ok(Ok(name))`・`Ok(Err(message))`・`Err(join_error)` の3つに分けると、「アプリのエラー」と「タスクそのものの失敗」を区別できます。RustNote の `verify_password` と同じ形です。
- `load_user(user_id)` はそれ自体が Future なので、`async move { ... }` で包まずに、そのまま `tokio::spawn` に渡せます。`user_id` は関数の引数として Future の中に移されています。

まとめ

- Tokio のランタイムは、ワーカースレッド・スケジューラー・タイマーと I/O の係でできている。`#[tokio::main]` は、ランタイムを作って `block_on` で `main` の中身を動かす形の書き換え。

- `tokio::spawn` したタスクは、`.await` しなくてもすぐ動き、引換券（`JoinHandle`）の `.await` で `Result<戻り値, JoinError>` を受け取る。引換券を捨てたタスクは、`main` が終わると途中で捨てられる。
- タスクに渡す仕事は `Send + 'static` でなければならない。外の変数は `move` で移し、共有は `Arc`、`.await` をまたいで `Rc` を持たない。ランタイムの中で `block_on` は使わない。

UNIT 03: JoinSet・select!・同時に動かす数の管理

この単元のゴール

- `JoinSet`（ジョインセット）で子のタスクをまとめて作り、**終わった順に**結果を受け取れる
- 親が `?` などで先に抜けると、`JoinSet` に残った子のタスクが取り消されることを、実行結果で確かめられる
- 「1つ失敗したら全体を止める」集め方と、「失敗しても全部集める」集め方を書き分けられる
- 同時に動かすタスクの数に、上限を付けられる
- `tokio::select!` で、先に終わったほうの結果だけを使い、負けたほうが捨てられることを説明できる
- ループの中の `select!` で Future を `pin!` で留めて使い回せる。E0271・E0277（`Unpin`）のエラーと、「仕事がいつまでも終わらない」間違いを直せる

たとえ話でつかむ

この単元では、3つの場面を使います。

- **JoinSet は、遠足の班です。** 先生（親のタスク）は、子どもたち（子のタスク）を班にして連れて行きます。戻ってきた子から順に点呼を取ります（`join_next`）。先生が帰ると決めたら、まだ遊んでいる子も**全員連れて帰ります**（取り消し）。だれかを置き去りにすることはありません。
- **select! は、2つの店に出す出前の注文です。** 同じ料理を2つの店に頼み、先に届いたほうを食べます。遅れたほうには「もういりません」と電話して、作るのをやめてもらいます（負けた Future は捨てられる）。
- **同時に動かす数の上限は、試着室です。** 試着室が2つしかないお店では、3人目は、どちらかが空くまで外で待ちます。待たせることで、お店の中があふれません。

> **用語:** JoinSet（ジョインセット：子のタスクをまとめて持つ袋。`spawn` で入れ、`join_next` で終わった順に1つずつ取り出す。袋を捨てると、残った子は取り消される）

> **用語:** 構造化された並行処理（structured concurrency：子のタスクが親より長生きしないように、親が子の始まりと終わりを責任を持って管理する書き方）

> **用語:** select!（セレクト：複数の Future を同時に進め、最初に終わったものの枝だけを実行するマクロ。残りの Future は捨てる）

図で理解する

まず、JoinSet です。親のタスクが袋を持ち、子のタスクを入れて、終わった順に受け取ります。

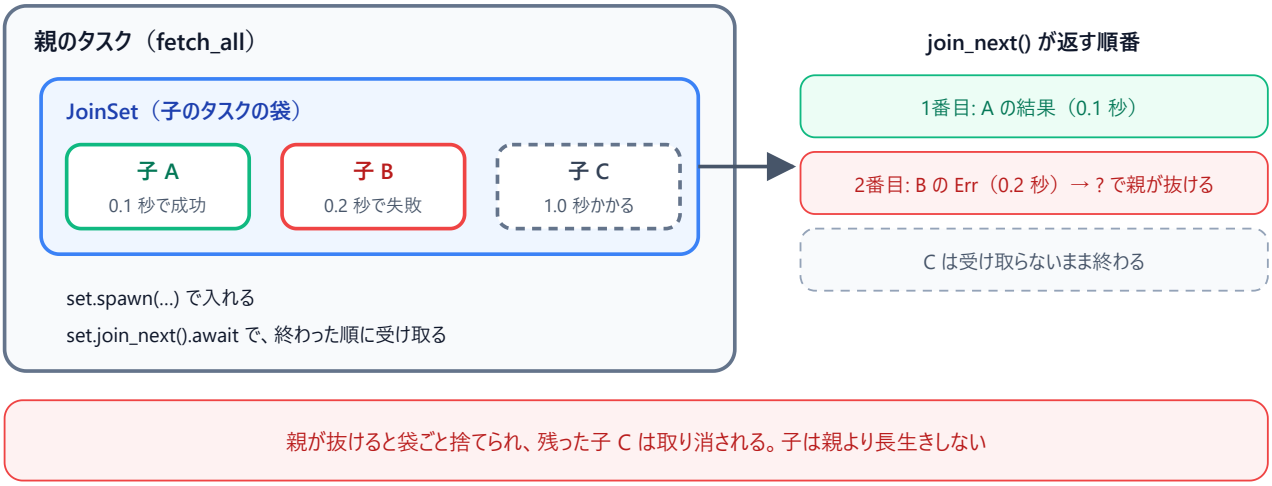


図5-8: JoinSet のしくみ。子のタスクを袋に入れ、終わった順に受け取る。親が途中で抜けると、袋が捨てられて残りの子も取り消される

- UNIT 02 の `tokio::spawn` だけで作ったタスクは、引換券を捨てると、親とは関係なく動き続けました（例2-4）。親が先に終わっても、子は残ります。
- JoinSet に入れたタスクは、**袋が捨てられると一緒に取り消されます**。親の関数が `return` や `?` で抜けると、関数の中の変数として持っていた袋も片付けられるので、「親が終わったのに、子だけが裏で動き続ける」ことが起きません。

次に、`select!` です。2つの Future を同時に進め、先に終わったほうを選びます。

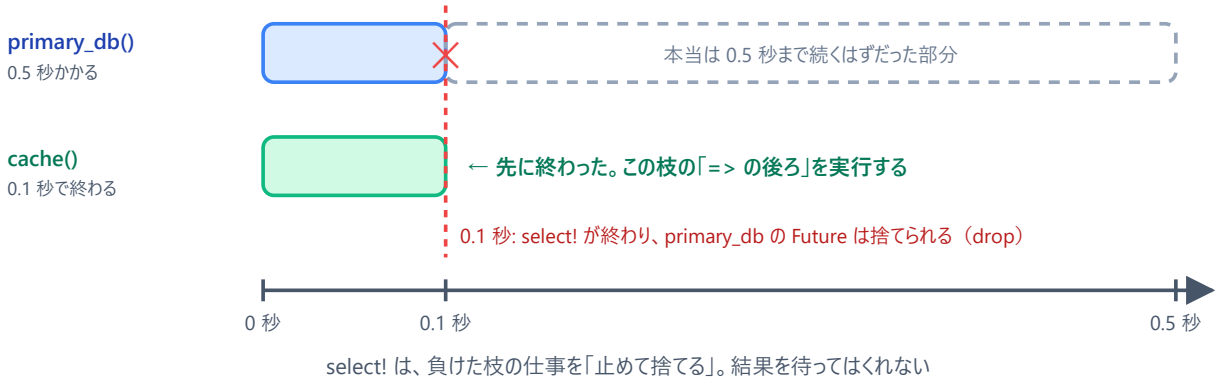


図5-9: select! で2つの Future を競争させたところ。先に終わった cache の枝だけが実行され、primary_db の Future は 0.1 秒の時点で捨てられる

- `select!` は、渡した Future を**同じタスクの中で**同時に進めます（`join!` と同じく並行で、並列ではありません）。
- どれか1つが終わったら、その枝の `=>` の後ろだけを実行し、ほかの Future は drop します。drop された Future は二度と poll されないので、**途中で止まります**（UNIT 01 の例1-3と同じキャンセルです）。

まず動かす最小コード

例3-1: JoinSet で子のタスクを作り、終わった順に受け取る

> **▼ このコードがやること（先に日本語で）**：画像を縮小する仕事の代わりに、0.3 秒・0.1 秒・0.2 秒かかる3つの子のタスクを JoinSet に入れます。`join_next` で、終わった順に結果を1つずつ受け取り、そのときの時刻を表示します。

```rust
use std::time::{Duration, Instant};
use tokio::task::JoinSet;

// 画像の縮小などの仕事の代わり: 指定の時間だけ待ってから結果を返す
async fn make_thumbnail(name: &'static str, millis: u64) -> String {
    tokio::time::sleep(Duration::from_millis(millis)).await;
    format!("{name} の縮小画像")
}

#[tokio::main]
async fn main() {
    let start = Instant::now();
    // JoinSet: 子タスクをまとめて持つ「親の袋」
    let mut set = JoinSet::new();
    set.spawn(make_thumbnail("a.png", 300));
    set.spawn(make_thumbnail("b.png", 100));
    set.spawn(make_thumbnail("c.png", 200));
    println!("子タスクの数: {}", set.len());

    // join_next: 「終わった順に」1つずつ結果を受け取る。全部受け取ると None
    while let Some(joined) = set.join_next().await {
        let thumbnail = joined.expect("子タスクが panic しました");
        println!("{:.1} 秒: {thumbnail}", start.elapsed().as_secs_f64());
    }
    println!("全部終わりました（残り {} 個）", set.len());
}
```

`src/bin/u03_joinset.rs` として保存して実行します。

```
cargo run -q --bin u03_joinset
```

▼ 実行結果

```
子タスクの数: 3
0.1 秒: b.png の縮小画像
0.2 秒: c.png の縮小画像
0.3 秒: a.png の縮小画像
全部終わりました（残り 0 個）
```

▼ コードを1つずつ分解して解説

解説1: set.spawn は、袋の中にタスクを作る

- `JoinSet::new()` で空の袋を作り、`set.spawn(Future)` で子のタスクを作って袋に入れます。`tokio::spawn` と同じく、入れた瞬間から動き始めます。
- `set.len()` は、袋の中に残っている（まだ受け取っていない）タスクの数です。

解説2: join_next は「終わった順」

- UNIT 02 の例2-1では、引換券を作った順番に `.await` したので、結果も作った順に受け取りました。JoinSet の `join_next()` は、**いちばん早く終わったもの**から返します。実行結果が b → c → a の順になっているのはそのためです。
- 先に終わったものから順に処理を進められるので、「全部そろうまで何もできない」ことがありません。
- 戻り値は `Option<Result<T, JoinError>>` です。袋が空になると `None` が返るので、`while let Some(joined) = ...` で「袋が空になるまで繰り返す」と書けます。中の `Result` は、UNIT 02 と同じく「panic したかどうか」です。

解説3: 袋に入れられるのは、同じ型を返すタスクだけ

- `JoinSet<String>` のように、袋には「何の型を返すタスクの袋か」が決まっています。ここでは `make_thumbnail` が `String` を返すので、`JoinSet<String>` になりました（型は Rust が推論しています）。
- 違う型を返すタスクを入れるとコンパイルエラーになります（「よくあるエラー1」）。

> **ヒント:** 全部の結果を一度にまとめて受け取ればよいときは、`set.join_all().await` も使えます。全部が終わるのを待ち、結果を `Vec` で返します（例3-4で使います）。ただし、`join_all` は子のどれかが panic すると、自分も panic します（tokio 1.53.3 のソースで、panic をそのまま投げ直す作りになっていることを確かめました）。1つずつ確かめたいときは `join_next` を使います。

例3-2: select! で、先に終わったほうを使う

> **▼ このコードがやること（先に日本語で）:** 遅い「本命の DB」と速い「予備のキャッシュ」に、同時に問い合わせます。`select!` で先に終わったほうの結果を使い、負けたほうが途中で片付けられることを、`Drop` で表示して確かめます。①は キャッシュが速い場合、②は DB が速い場合です。

```rust
use std::time::{Duration, Instant};

struct Announce(&'static str);

impl Drop for Announce {
    fn drop(&mut self) {
        println!("  片付け: {}", self.0);
    }
}

// 本命の DB（遅い）
async fn primary_db(millis: u64) -> &'static str {
    let _announce = Announce("primary_db");
    tokio::time::sleep(Duration::from_millis(millis)).await;
    println!("  primary_db: 最後まで終わりました");
    "本命の DB の結果"
}

// 予備のキャッシュ（速い）
async fn cache(millis: u64) -> &'static str {
    let _announce = Announce("cache");
    tokio::time::sleep(Duration::from_millis(millis)).await;
    println!("  cache: 最後まで終わりました");
    "キャッシュの結果"
}

async fn race(db_millis: u64, cache_millis: u64) {
    let start = Instant::now();
    // 2つの Future を同時に進め、先に終わったほうの枝だけを実行する
    let winner = tokio::select! {
        value = primary_db(db_millis) => format!("DB が勝ち: {value}"),
        value = cache(cache_millis) => format!("キャッシュが勝ち: {value}"),
    };
    println!("{winner} ({:.1} 秒) ", start.elapsed().as_secs_f64());
}

#[tokio::main]
async fn main() {
    println!("① DB 500ms / キャッシュ 100ms");
    race(500, 100).await;
    println!("② DB 100ms / キャッシュ 500ms");
    race(100, 500).await;
}
```

`src/bin/u03_select.rs` として保存して実行します。

```
cargo run -q --bin u03_select
```

▼ 実行結果

```
① DB 500ms / キャッシュ 100ms
  cache: 最後まで終わりました
  片付け: cache
  片付け: primary_db
キャッシュが勝ち: キャッシュの結果（0.1 秒）
② DB 100ms / キャッシュ 500ms
  primary_db: 最後まで終わりました
  片付け: primary_db
  片付け: cache
DB が勝ち: 本命の DB の結果（0.1 秒）
```

▼ コードを1つずつ分解して解説

解説1: select! の枝は「パターン = Future => 式」

- `select!` の中の1行1行を**枝**と呼びます。`value = primary_db(db_millis) => format!(...)` は、「`primary_db(...)` が終わったら、その結果を `value` に入れて、`=>` の後ろの式を実行する」という意味です。
- 先に終わった枝の式の値が、`select!` 全体の値になります。ここでは、どちらの枝も `String` を返すので、`winner` は `String` です。すべての枝の式は、同じ型でなければなりません。

解説2: 負けた Future は、その場で捨てられる

- ①では、0.1 秒でキャッシュが終わりました。`片付け: cache` はキャッシュの関数が最後まで動いて自然に片付いた印、`片付け: primary_db` は、**負けた DB の Future が途中で捨てられた**印です。「primary_db: 最後まで終わりました」は表示されていません。
- ②では、勝ち負けが逆になりました。どちらの場合も、全体は勝ったほうの 0.1 秒で終わり、負けたほうの残りの時間は待っていません。

解説3: Drop で「片付け」を目に見えるようにする

- `Announce` は、片付けられるときに表示するだけの小さな struct です。第3巻 UNIT 01 で学んだ `Drop` trait を実装しています。
- `let _announce = Announce("cache");` のように関数の最初で作っておくと、関数が最後まで動いて終わったときも、途中で Future ごと捨てられたときも、必ず `drop` が呼ばれます。キャンセルされたかどうかを調べるときに便利な道具です。
- 変数名を `_announce` にしているのは、「使わないけれど、関数の終わりまで持っておく」ためです。`_` 1文字だけにすると、その場ですぐに片付けられてしまうので注意しましょう。

> **注意:** `select!` で負けた Future は、途中のどこで止まるか分かりません。「ファイルに半分だけ書いた」「お金を引き落としたのに、記録を書く前に止まった」のように、途中で止まると困る処理を、そのまま `select!` の枝に入れないようにしましょう。途中で捨てられても困らない性質を**キャンセル安全**（cancel safety）と呼びます。tokio のドキュメントには、関数ごとに「この関数はキャンセル安全です（This method is cancel safe）」のような説明が書いてあります。

もう一歩: 実務に近い形

例3-3: 1つ失敗したら、残りの子をまとめて取り消す

> **▼ このコードがやること（先に日本語で）:** 3つの外部サービス（A は 0.1 秒、B は 0.2 秒、C は 1.0 秒）に同時に問い合わせます。B だけが失敗します。1つでも失敗したら全体を失敗にし、まだ終わっていない C は待たずに取り消します。C が本当に途中で取り消されたかを、片付けの表示と時刻で確かめます。

```rust
use std::time::{Duration, Instant};
use tokio::task::JoinSet;

// 片付けのときに表示する印。タスクが途中で取り消されると、ここで分かる
struct Announce(&'static str);

impl Drop for Announce {
    fn drop(&mut self) {
        println!("  片付け: {}", self.0);
    }
}

// 外部サービスへの問い合わせの代わり。name が "B" のときだけ失敗する
async fn fetch(name: &'static str, millis: u64) -> Result<String, String> {
    let _announce = Announce(name);
    tokio::time::sleep(Duration::from_millis(millis)).await;
    if name == "B" {
        return Err(format!("{name} は応答しませんでした"));
    }
    Ok(format!("{name} の結果"))
}

async fn fetch_all() -> Result<Vec<String>, String> {
    let mut set = JoinSet::new();
    set.spawn(fetch("A", 100));
    set.spawn(fetch("B", 200));
    set.spawn(fetch("C", 1000));

    let mut results = Vec::new();
    while let Some(joined) = set.join_next().await {
        // 1つ目の ? は「panic したか」、2つ目の ? は「Err を返したか」
        let value = joined.map_err(|e| e.to_string())??;
        results.push(value);
    }
    Ok(results)
    // 途中で return Err した場合も、ここで set が片付けられ、残りの子タスクは取り消される
}

#[tokio::main]
async fn main() {
    let start = Instant::now();
    let result = fetch_all().await;
    println!("{result:?} ({:.1} 秒) ", start.elapsed().as_secs_f64());
    // 取り消された子タスクの片付けが表示される時間を少しだけ待つ
    tokio::time::sleep(Duration::from_millis(50)).await;
    println!(
        "main が終わります ({:.1} 秒) ",
        start.elapsed().as_secs_f64()
```

```
    );
}
```

`src/bin/u03_joinset_fail.rs` として保存して実行します。

```
cargo run -q --bin u03_joinset_fail
```

▼ 実行結果

```
  片付け: A
  片付け: B
Err("B は応答しませんでした") (0.2 秒)
  片付け: C
main が終わります (0.3 秒)
```

▼ コードを1つずつ分解して解説

解説1: ?? で「panic」と「アプリのエラー」の両方を返す

- `joined` の型は `Result<Result<String, String>, JoinError>` です（UNIT 02 の練習問題3と同じ形）。
- `joined.map_err(|e| e.to_string())` で、外側のエラー（`JoinError`）を `String` に変えています。これで外側も内側もエラーの型が `String` にそろうので、`?` を2回続けて書けます。1つ目の `?` で panic を、2つ目の `?` で `fetch` の `Err` を、`fetch_all` のエラーとして返します。

解説2: 親が抜けると、袋の中の子は取り消される

- 0.2 秒で B の `Err` を受け取ると、2つ目の `?` で `fetch_all` から抜けます。そのとき、関数の中の変数 `set` が片付けられ、JoinSet は**中に残っていた子（C）を全部取り消します**。
- 実行結果の `片付け: C` は、`main が終わります (0.3 秒)` より前に表示されています。C は 1.0 秒かかるはずだったのに、0.3 秒より前に片付けられました。プログラムが終わったからではなく、JoinSet が捨てられたことで取り消されたのです。tokio 1.53.3 のソースでも、JoinSet が片付けられるときに、中のタスクを1つずつ `abort` する作りになっていることを確かめました。
- `片付け: C` が結果の行より後に表示されるのは、取り消しの合図を受けた C が、ランタイムの上で少し遅れて片付けられるからです。`main` の最後で 0.05 秒待っているのは、その表示を見るためだけです。

解説3: 「全部そろわないと意味がない」ときの形

- 画面に A・B・C の3つの情報を並べるページで、1つでも欠けたらエラーページを出す、というような場面では、この「1つ失敗したら全部やめる」形が向いています。失敗が分かった時点で残りを取り消すので、むだな問い合わせが続きません。
- 失敗しても、そろった分だけで画面を出したいときは、エラーも結果として集める形にします（練習問題3）。

例3-4: 同時に動かす数を、2つまでに抑える

> **▼ このコードがやること（先に日本語で）**：0.2 秒かかるダウンロードを6つ行います。ただし、同時に動かすのは2つまでにします。袋に2つ入っているときは、どれか1つが終わるまで待ってから、次を入れます。それぞれを始めた時刻と、全体の時間を表示します。

```rust
use std::time::{Duration, Instant};
use tokio::task::JoinSet;

// 外部へのリクエストの代わり。どれも 200ms かかる
async fn download(id: u32) -> u32 {
    tokio::time::sleep(Duration::from_millis(200)).await;
    id
}

#[tokio::main]
async fn main() {
    // 同時に動かすのは最大 2 個まで
    const LIMIT: usize = 2;
    let start = Instant::now();
    let mut set = JoinSet::new();
    let mut done = Vec::new();

    for id in 1..=6 {
        // 袋がいっぱいなら、どれか1つ終わるまで待ってから次を入れる
        if set.len() >= LIMIT {
            // 袋には LIMIT 個入っているので、join_next は必ず Some を返す
            let joined = set.join_next().await.expect("袋は空ではない");
            done.push(joined.expect("子タスクが panic しました"));
        }
        println!("{:.2} 秒: {id} 番を開始", start.elapsed().as_secs_f64());
        set.spawn(download(id));
    }
    // 残りを全部受け取る。join_all は全部終わるのを待って Vec で返す
    done.extend(set.join_all().await);

    done.sort();
    println!("{done:?} ({:.2} 秒) ", start.elapsed().as_secs_f64());
}
```

`src/bin/u03_limited.rs` として保存して実行します。

```
cargo run -q --bin u03_limited
```

▼ 実行結果

```
0.00 秒: 1 番を開始
0.00 秒: 2 番を開始
0.21 秒: 3 番を開始
0.21 秒: 4 番を開始
0.41 秒: 5 番を開始
0.41 秒: 6 番を開始
[1, 2, 3, 4, 5, 6]（0.63 秒）
```

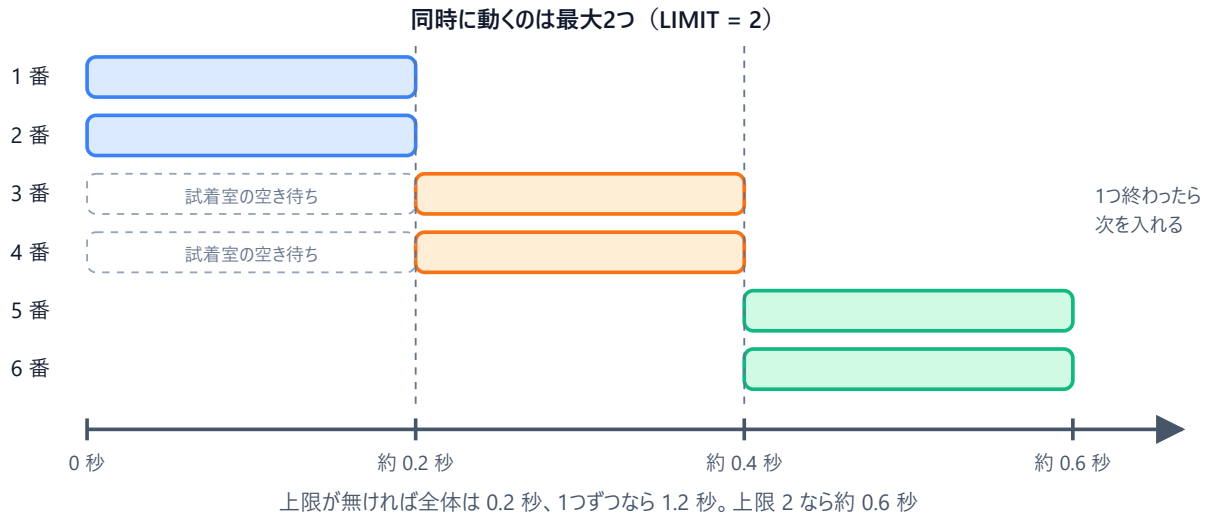


図5-10: 同時に動かす数を2つまでにしたときの動き。3番以降は、前のどれかが終わるまで始まらない。待たせることで、相手のサーバーに一度に押し寄せない

▼ コードを1つずつ分解して解説

解説1: 袋の中の数で、次を入れるかどうかを決める

- ループのたびに `set.len()`（袋の中の、動いているタスクの数）を見ます。`LIMIT`（2）以上なら、`join_next().await` で1つ終わるのを待ってから、次を入れます。袋の中は、いつも2つ以下に保たれます。
- 実行結果では、1・2番が 0 秒、3・4番が約 0.2 秒、5・6番が約 0.4 秒に始まっています。全体は約 0.6 秒です。
- 0.2 秒ではなく 0.21 秒のように少し遅れているのは、タイマーの都合で、少し遅れて起こされることがあるためです。パソコンや OS によって数字は少し変わります。

解説2: なぜわざわざ待たせるのか

- 相手のサーバーや DB には、一度に受けられる数に限りがあります。1万件のダウンロードを一度に始めると、相手に断られたり、自分のメモリや接続が足りなくなったりします。
- 上限を付けると、全体の時間は少し長くなりますが、**一度に使う資源の量が上限で決まる**ので、件数が増えても安全に動きます。「全部同時」と「1つずつ」の間の、ちょうどよいところを選べるのが、この形のよいところです。

解説3: 残りは join_all でまとめて受け取る

- ループが終わった時点で、袋にはまだ最後の2つが残っています。`set.join_all().await` は、残りの全部が終わるのを待ち、結果を `Vec` で返します。

- `join_next` も `join_all` も、結果は**終わった順**に並びます。番号の順に並べたいので、最後に `done.sort()` で並べ替えています。

> **ヒント:** 同時に動かす数の上限は、`Semaphore`（セマフォ）という道具を使っても作れます。たくさんのタスクが、それぞれ別の場所から「入ってよいか」を確かめたいときは、そちらが便利です。UNIT 05 で学びます。

例3-5: 締め切りまで、仕事を繰り返す（select! をループの中で使う）

> **▼ このコードがやること（先に日本語で）:** 1 秒後の締め切りを1つ作り、それまでの間、0.3 秒かかる仕事を繰り返します。ループのたびに `select!` で「締め切り」と「次の仕事」を競争させ、締め切りが来たらループを抜けます。

```
use std::pin::pin;
use std::time::{Duration, Instant};

#[tokio::main]
async fn main() {
    let start = Instant::now();
    // 全体の締め切り（1 秒後）。pin! で留めておくと、&mut で何度でも見張れる
    let mut deadline = pin!(tokio::time::sleep(Duration::from_secs(1)));
    let mut ticks = 0;
    loop {
        tokio::select! {
            _ = &mut deadline => {
                println!("{:.1} 秒: 締め切りです", start.elapsed().as_secs_f64());
                break;
            }
            _ = tokio::time::sleep(Duration::from_millis(300)) => {
                ticks += 1;
                println!("{:.1} 秒: {ticks} 回目の仕事", start.elapsed().as_secs_f64());
            }
        }
    }
}
```

`src/bin/u03_deadline.rs` として保存して実行します。

```
cargo run -q --bin u03_deadline
```

▼ 実行結果

```
0.3 秒: 1 回目の仕事
0.6 秒: 2 回目の仕事
0.9 秒: 3 回目の仕事
1.0 秒: 締め切りです
```

▼ コードを1つずつ分解して解説

解説1: 締め切りは、ループの外で1回だけ作る

- 締め切りの `sleep` は、ループの**外**で1回だけ作っています。もしループの中で `sleep(Duration::from_secs(1))` を毎回作ると、毎回「今から1秒」になってしまい、締め切りがいつまでも来ません。
- ループの中では、`&mut deadline` で締め切りを**借りて**見張ります。`select!` の別の枝が勝っても、借りていただけなので、締め切りの Future そのものは捨てられず、次のループで続きから見張れます。

解説2: &mut で借りて poll するには、pin! で留める必要がある

- `pin!(...)` は、UNIT 01 の例1-4で使ったのと同じ「動かないように留める」マクロです（`std::pin::pin`）。
- `tokio::time::sleep` が作る Future（`Sleep`）は、poll している途中で場所が動くと困る種類の Future です。`&mut` で借りて何度も poll するには、先に留めておく必要があります。留めずに書くとどうなるかは、「よくあるエラー2」で見ます。

解説3: 0.9 秒の次の仕事は、締め切りで捨てられた

- 4回目の仕事は 0.9 秒に始まり、1.2 秒に終わるはずでした。しかし 1.0 秒に締め切りの枝が勝ったので、4回目の `sleep(300ms)` は捨てられ、表示されていません。
- 締め切りを守るために、途中の仕事を捨てる、という形です。捨てられて困る仕事なら、締め切りを過ぎても最後まで終わらせるようにします（UNIT 06 の graceful shutdown）。

よくあるエラーと読み方

エラー1: JoinSet に違う型を返すタスクを入れる（E0271）

```rust
use tokio::task::JoinSet;

#[tokio::main]
async fn main() {
    let mut set = JoinSet::new();
    set.spawn(async { 42 });
    // 1つ目と違う型の値を返すタスクを入れる
    set.spawn(async { String::from("文字列") });
    while let Some(joined) = set.join_next().await {
        println!("{:?}", joined);
    }
}
```

```
   Compiling async_lab v0.1.0 (...\async_lab)
error[E0271]: expected `{async block@e03_joinset_types.rs:8:15}` to be a future that
resolves to `{integer}`, but it resolves to `String`
   --> src\bin\e03_joinset_types.rs:8:15
    |
  8 |     set.spawn(async { String::from("文字列") });
    |         ----- ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ expected integer, found `String`
    |         |
    |         required by a bound introduced by this call
    |
note: required by a bound in `JoinSet::<T>::spawn`
   --> ...\tokio-1.53.3\src\task\join_set.rs:144:19
    |
142 |     pub fn spawn<F>(&mut self, task: F) -> AbortHandle
    |            ----- required by a bound in this associated function
143 |     where
144 |         F: Future<Output = T>,
    |                   ^^^^^^^^^^ required by this bound in `JoinSet::<T>::spawn`

For more information about this error, try `rustc --explain E0271`.
error: could not compile `async_lab` (bin "e03_joinset_types") due to 1 previous error
```

- **何が起きたか:**「この async ブロックは整数（`{integer}`）を返す Future のはずなのに、`String` を返しています」というエラーです。1つ目の `set.spawn(async { 42 })` で、Rust は「これは整数を返すタスクの袋だ」と決めました。そこに `String` を返すタスクを入れようとしたので、型が合いません。
- `F: Future<Output = T>` は、「入れるタスクの戻り値の型（`Output`）は、袋の型 `T` と同じでなければならない」という条件です。

- **直し方**: 戻り値の型をそろえます。違う種類の結果を1つの袋で扱いたいときは、`enum` を作って包みます（第2巻 UNIT 08）。たとえば `enum Answer { Count(i32), Title(String) }` を作り、`async { Answer::Count(42) }` と `async { Answer::Title(String::from("文字列")) }` のように入れます。

エラー2: 締め切りを留めずに &mut で見張る（E0277: PhantomPinned cannot be unpinned）

例3-5の `pin!` を付け忘れた場合です。

```
use std::time::Duration;

#[tokio::main]
async fn main() {
    // 全体の締め切り（1 秒後）
    let deadline = tokio::time::sleep(Duration::from_secs(1));
    let mut ticks = 0;
    loop {
        tokio::select! {
            // 同じ締め切りを、ループのたびに見張りたい
            _ = &mut deadline => {
                println!("締め切りです");
                break;
            }
            _ = tokio::time::sleep(Duration::from_millis(300)) => {
                ticks += 1;
                println!("{ticks} 回目の仕事");
            }
        }
    }
}
```

同じ形のエラーが3つ出ます。1つ目だけを載せます。

```
   Compiling async_lab v0.1.0 (...\async_lab)
error[E0277]: `PhantomPinned` cannot be unpinned
  --> src\bin\e03_select_unpin.rs:9:9
   |
 9 | /         tokio::select! {
10 | |             // 同じ締め切りを、ループのたびに見張りたい
11 | |             _ = &mut deadline => {
12 | |                 println!("締め切りです");
...  |
19 | |         }
   | |         ^
   | |         |
   | |_________within `(PhantomData<&()>, PhantomPinned)`, the trait `Unpin` is not
implemented for `PhantomPinned`
   |           required by a bound introduced by this call
   |
   = note: consider using the `pin!` macro
           consider using `Box::pin` if you need to access the pinned value outside of
the current scope
   = note: required because it appears within the type `(PhantomData<&()>,
PhantomPinned)`
   = note: required for `Sleep` to implement `Unpin`
   = note: required for `&mut Sleep` to implement `Future`
   = note: required for `&mut Sleep` to implement `IntoFuture`
   = note: this error originates in the macro `$crate::select` which comes from the
expansion of the macro `tokio::select` (in Nightly builds, run with -Z macro-backtrace
for more info)
```

難しそうな言葉が並んでいますが、下から順に読むと筋が通ります。

- `required for &mut Sleep to implement Future`:「`&mut Sleep`（借りた締め切り）を Future として使うには、条件があります」。
- `required for Sleep to implement Unpin`: その条件は「`Sleep` が `Unpin`（動かしても平気）であること」です。
- `the trait Unpin is not implemented for PhantomPinned`: でも `Sleep` の中には「動かしてはいけない」という印（`PhantomPinned`）が入っているので、`Unpin` ではありません。
- `note: consider using the pin! macro`: だから「`pin!` マクロを使ってください」。これが直し方です。
- **直し方**: 例3-5のとおり、`let mut deadline = pin!(tokio::time::sleep(...));` のように、作るときに `pin!` で留めます。留めた Future は、`&mut deadline` で借りて何度でも poll できます。
- `Unpin` は「poll の途中で場所を動かしても平気な型」に付く印です。多くの型は自動で `Unpin` ですが、`Sleep` や async fn が作る Future の多くは `Unpin` ではありません。「`&mut` で借りて select! に渡すときは、`pin!` が要る」と覚えておけば十分です。

エラー3（エラーが出ない間違い）：ループの中の select! で、仕事が毎回最初からやり直しになる

コンパイルは通り、panic もしないのに、仕事がいつまでも終わらない例です。0.5 秒かかる仕事を進めながら、0.2 秒ごとに見回りの表示を出すつもりで書きました。

```
use std::time::{Duration, Instant};

// 0.5 秒かかる仕事
async fn long_job() -> &'static str {
    println!("  long_job: 始めます");
    tokio::time::sleep(Duration::from_millis(500)).await;
    "仕事の結果"
}

#[tokio::main]
async fn main() {
    let start = Instant::now();
    for round in 1..=4 {
        tokio::select! {
            // ループのたびに、新しい long_job() を作ってしまう
            result = long_job() => {
                println!("{:.1} 秒: {result}", start.elapsed().as_secs_f64());
                return;
            }
            _ = tokio::time::sleep(Duration::from_millis(200)) => {
                println!("{:.1} 秒: 見回り {round} 回目", start.elapsed().as_secs_f64());
            }
        }
    }
    println!(
        "{:.1} 秒: 仕事は一度も終わりませんでした",
        start.elapsed().as_secs_f64()
    );
}
```

`src/bin/u03_select_restart.rs` として実行した結果です（終わらないと困るので、ループは4回までにしています）。

```
  long_job: 始めます
0.2 秒: 見回り 1 回目
  long_job: 始めます
0.4 秒: 見回り 2 回目
  long_job: 始めます
0.6 秒: 見回り 3 回目
  long_job: 始めます
0.8 秒: 見回り 4 回目
0.8 秒: 仕事は一度も終わりませんでした
```

- **何が起きたか:** 「始めます」が4回も表示されています。`select!` の枝に `long_job()` と書いたので、**ループのたびに新しい `long_job` の Future が作られ**、0.2 秒で見回りの枝が勝つたびに、途中まで進んだ仕事が捨てられていました。0.5 秒かかる仕事は、0.2 秒より長く続けてもらえないので、永遠に終わりません。
- **直し方:** 例3-5の締め切りと同じく、仕事の Future をループの外で1回だけ作り、`pin!` で留めて `&mut` で借ります。

```
use std::pin::pin;
use std::time::{Duration, Instant};

// 0.5 秒かかる仕事
async fn long_job() -> &'static str {
    println!("  long_job: 始めます");
    tokio::time::sleep(Duration::from_millis(500)).await;
    "仕事の結果"
}

#[tokio::main]
async fn main() {
    let start = Instant::now();
    // 仕事の Future はループの外で1回だけ作り、pin! で留める
    let mut job = pin!(long_job());
    for round in 1..=4 {
        tokio::select! {
            // &mut で借りて進める。負けても捨てられず、次のループで続きから進む
            result = &mut job => {
                println!("{:.1} 秒: {result}", start.elapsed().as_secs_f64());
                return;
            }
            _ = tokio::time::sleep(Duration::from_millis(200)) => {
                println!("{:.1} 秒: 見回り {round} 回目", start.elapsed().as_secs_f64());
            }
        }
    }
    println!(
        "{:.1} 秒: 仕事は一度も終わりませんでした",
        start.elapsed().as_secs_f64()
    );
}
```

`src/bin/u03_select_resume.rs` として実行した結果です。

```
  long_job: 始めます
0.2 秒: 見回り 1 回目
0.4 秒: 見回り 2 回目
0.5 秒: 仕事の結果
```



図5-11: ループの中の select! の2つの書き方。枝の中で毎回 Future を作ると、負けるたびに捨てられて最初からやり直しになる。外で作って &mut で借りれば、続きから進む

- 「始めます」は1回だけで、見回りを2回はさんで、0.5 秒で仕事が終わりました。`&mut job` で借りているので、見回りの枝が勝っても `job` は捨てられず、次のループでは**続きから**進みます。
- `select!` をループの中で使うときは、「この枝の Future は、ループのたびに作り直されていないか？」を必ず確かめましょう。作り直してよいのは、例3-5の `sleep(300ms)` のように、毎回最初からでかまわない Future だけです。

RustNote ではここで使う

RustNote のコードそのものには、`JoinSet` も `select!` も出てきません。1つのリクエストの処理は、DB に順番に問い合わせるだけで済んでいるからです。一方で、RustNote を動かしている Topcoat の中では、この単元の形がそのまま使われています。UNIT 02 で読んだ Topcoat 0.10.0 の `internal_serve`（`topcoat-router-0.10.0/src/service.rs`）を、もう少し詳しく見てみましょう（コメントは日本語に置き換え、関係の無い部分は省略しています）。

```
// 止める合図の Future を、ループの外で1回だけ留める
let mut shutdown = pin!(shutdown);

loop {
    let accepted = tokio::select! {
        // 新しい接続を受け付ける
        accepted = listener.accept() => accepted,
        // 止める合図（Ctrl+C）が来たら、ループを抜ける
        () = &mut shutdown => break,
    };
    let (stream, remote_addr) = accepted?;
    //  （1つの接続を、1つのタスクで処理する。UNIT 02 で読んだ部分。省略）
}
```

- 止める合図 `shutdown` は、ループの外で `pin!` で留め、ループの中では `&mut shutdown` で借りています。例3-5の締め切りと、まったく同じ形です。もし枝の中で毎回作り直していたら、エラー3のように、合図を見逃し続けるかもしれません。

- 接続を受け付ける `listener.accept()` は、ループのたびに新しく作っています。これが許されるのは、tokio の `TcpListener::accept` がキャンセル安全だからです。tokio 1.53.3 のソースには「このメソッドはキャンセル安全です。select! の枝で使い、ほかの枝が先に終わったときは、新しい接続を受け付けていないことが保証されます」と書かれています。途中で捨てても、接続を取りこぼしません。
- 止める合図を受けたら、処理中の接続をしばらく待ってから終わります。その部分は、UNIT 04 の channel と UNIT 06 の graceful shutdown で読みます。

> **ヒント:** 自分のアプリで「1つの画面のために、関係の無い3つの問い合わせをする」ようになったら、UNIT 01 の `try_join!` か、この単元の JoinSet を思い出しましょう。問い合わせの数が決まっているなら `try_join!`、数が実行するまで分からないなら JoinSet が向いています。

練習問題

問1: 例3-4の `u03_limited.rs` で、`LIMIT` を `2` から `3` に変えると、それぞれの番号が始まる時刻と、全体の時間はどうなるでしょうか。予想してから実行してください（`src/bin/u03_q1.rs`）。

問2: 0.8 秒かかる問い合わせ `slow_query` に、「指定した時間を過ぎたら時間切れ」のエラーを返す仕組みを、`select!` で作ってください。300 ミリ秒と 1000 ミリ秒の2通りで試します（`src/bin/u03_q2.rs`）。

```
// 遅い問い合わせ（0.8 秒かかる）
async fn slow_query() -> &'static str {
    tokio::time::sleep(Duration::from_millis(800)).await;
    "問い合わせの結果"
}
```

問3: 例3-3の `fetch_all` を書き換えて、B が失敗しても止めずに、A・B・C の全部の結果（成功も失敗も）を `Vec<Result<String, String>>` で返すようにしてください。全体で何秒かかるかも予想しましょう（`src/bin/u03_q3.rs`）。

解答と解説

問1の解答: `const LIMIT: usize = 3;` に変えて実行した結果です（コメントの「最大 2 個まで」も「最大 3 個まで」に直しました。ほかは例3-4と同じです）。

```
0.00 秒: 1 番を開始
0.00 秒: 2 番を開始
0.00 秒: 3 番を開始
0.21 秒: 4 番を開始
0.21 秒: 5 番を開始
0.21 秒: 6 番を開始
[1, 2, 3, 4, 5, 6]（0.43 秒）
```

- 最初の3つが同時に始まり、約 0.2 秒後に残りの3つが始まります。6つを3つずつ2回に分けたので、全体は約 0.4 秒です。
- 上限を上げるほど全体は速くなりますが、一度に使う資源（相手のサーバーへの接続など）も増えます。上限は「速さ」と「相手への負担」の兼ね合いで決めます。

問2の解答:

```
use std::time::{Duration, Instant};

// 遅い問い合わせ（0.8 秒かかる）
async fn slow_query() -> &'static str {
    tokio::time::sleep(Duration::from_millis(800)).await;
    "問い合わせの結果"
}

// 問い合わせと limit のタイマーを競争させ、タイマーが勝ったら時間切れにする
async fn query_with_limit(limit: Duration) -> Result<&'static str, String> {
    tokio::select! {
        value = slow_query() => Ok(value),
        _ = tokio::time::sleep(limit) => Err(format!("{} ミリ秒で時間切れ",
limit.as_millis())),
    }
}

#[tokio::main]
async fn main() {
    for millis in [300, 1000] {
        let start = Instant::now();
        let result = query_with_limit(Duration::from_millis(millis)).await;
        println!("{result:?} ({:.1} 秒) ", start.elapsed().as_secs_f64());
    }
}
```

```
Err("300 ミリ秒で時間切れ") (0.3 秒)
Ok("問い合わせの結果") (0.8 秒)
```

- 問い合わせとタイマーを競争させ、タイマーが勝ったら `Err` を返します。300 ミリ秒ではタイマーが勝ち、0.3 秒で時間切れになりました。1000 ミリ秒では問い合わせが先に終わり、0.8 秒で結果が返りました。
- 時間切れのとき、`slow_query` の Future は途中で捨てられます（例3-2と同じです）。
- 「時間を区切って待つ」はとてもよく使うので、tokio には専用の関数 `tokio::time::timeout` が用意されています。UNIT 06 で学びます。中でやっていることは、この解答とほぼ同じです。

問3の解答:

```rust
use std::time::{Duration, Instant};
use tokio::task::JoinSet;

// 外部サービスへの問い合わせの代わり。name が "B" のときだけ失敗する
async fn fetch(name: &'static str, millis: u64) -> Result<String, String> {
    tokio::time::sleep(Duration::from_millis(millis)).await;
    if name == "B" {
        return Err(format!("{name} は応答しませんでした"));
    }
    Ok(format!("{name} の結果"))
}

// 失敗があっても止めずに、全部の結果を集める
async fn fetch_all() -> Vec<Result<String, String>> {
    let mut set = JoinSet::new();
    set.spawn(fetch("A", 100));
    set.spawn(fetch("B", 200));
    set.spawn(fetch("C", 1000));

    let mut results = Vec::new();
    while let Some(joined) = set.join_next().await {
        // panic は Err に変えて、アプリのエラーと同じように集める
        let result = joined.unwrap_or_else(|e| Err(e.to_string()));
        results.push(result);
    }
    results
}

#[tokio::main]
async fn main() {
    let start = Instant::now();
    for result in fetch_all().await {
        println!("{result:?}");
    }
    println!("{:.1} 秒", start.elapsed().as_secs_f64());
}
```

```
Ok("A の結果")
Err("B は応答しませんでした")
Ok("C の結果")
1.0 秒
```

- `?` を使わずに、`Result` をそのまま `results` に入れていきます。B の `Err` も1つの結果として集めるので、ループは止まらず、C（1.0 秒）まで待ちます。全体は、いちばん長い C の 1.0 秒です。

- `joined.unwrap_or_else(|e| Err(e.to_string()))` は、「panic していなければ中身の `Result` をそのまま使い、panic していたら（`JoinError` なら）それを `Err` の文字列に変える」という意味です。panic した子も、1つの失敗として数えられます。
- 例3-3（1つ失敗したら止める）と、この解答（全部集める）のどちらを選ぶかは、画面の作りで決まります。「一部が欠けても表示できる」なら、この形のほうが利用者に親切です。

まとめ

- JoinSet は子のタスクの袋で、`join_next` で終わった順に受け取れる。袋が捨てられると残った子は取り消されるので、子が親より長生きしない。同時に動かす数の上限も、袋の中の数で作れる。
- `select!` は、先に終わった枝だけを実行し、負けた Future をその場で捨てる。途中で捨てられて困る処理（キャンセル安全でない処理）は、枝に入れない。
- ループの中の `select!` で同じ Future を使い続けるときは、ループの外で作って `pin!` で留め、`&mut` で借りる。留め忘れは E0277（`Unpin`）、作り直しは「いつまでも終わらない」間違いになる。

UNIT 04: channel・backpressure・所有権の移動

この単元のゴール

- `mpsc`（エムピーエスシー）の channel で、何人もの送り手から1人の受け手へ値を渡せる。値が**持ち主ごと**受け手へ移ることを説明できる
- 送る口（`Sender`）が全部片付くと、受け取りのループが終わるしくみを説明し、`drop(tx)` の書き忘れで処理が終わらなくなる間違いを直せる
- 容量の決まった channel で、受け手より速い送り手を待たせられる（**backpressure**：バックプレッシャー）。混んでいるときに `try_send` ですぐ断れる
- `oneshot` で返事を1回だけ受け取り、「依頼を受ける係」のタスク（アクター）を作れる
- `broadcast`（全員に配る）と `watch`（最新の値だけを共有する）を、目的に合わせて選べる
- E0382（送った値をまた使う）・`SendError`（受け手がいない）・`Lagged`（読み飛ばし）を読んで直せる

たとえ話でつかむ

channel は、**郵便受け付きの仕事の受付箱**です。

- 送り手は、依頼の紙を箱に入れます（`send`）。紙は箱に入れた瞬間に、送り手の手元からなくなります。コピーが残るわけではありません。これが「**持ち主ごと移る**」です。
- 受け手は、箱から紙を1枚ずつ取り出して処理します（`recv`）。箱が空なら、次の紙が来るまで待ちます。
- 送り手が全員帰って、もう誰も紙を入れに来ないと分かったら、受け手は残りを片付けて帰ります（`recv` が `None` を返す）。だれか1人でも「まだ入れるかもしれない人」が残っていると、受け手はいつまでも待ち続けます。

- 箱には**入る枚数の上限**があります。いっぱいのときに来た送り手は、空きができるまで箱の前で待たされます。これが**backpressure**（背圧：後ろから押し返す力）です。受け手の処理が遅いと、その遅さが送り手に伝わって、仕事が入ってくる速さが自然に抑えられます。

> **用語:** channel（チャネル：タスクの間で値を渡すための通り道。送る口 `Sender` と受け取る口 `Receiver` の2つで1組）

> **用語:** mpsc（multi-producer, single-consumer：送り手は何人でもよく、受け手は1人の channel）

> **用語:** backpressure（バックプレッシャー：受け手が追いつかないとき、送り手を待たせたり断ったりして、入ってくる仕事の速さを抑えるしくみ）

図で理解する

まず、mpsc の channel です。送る口は `clone` で何人にも配れますが、受け取る口は1つだけです。

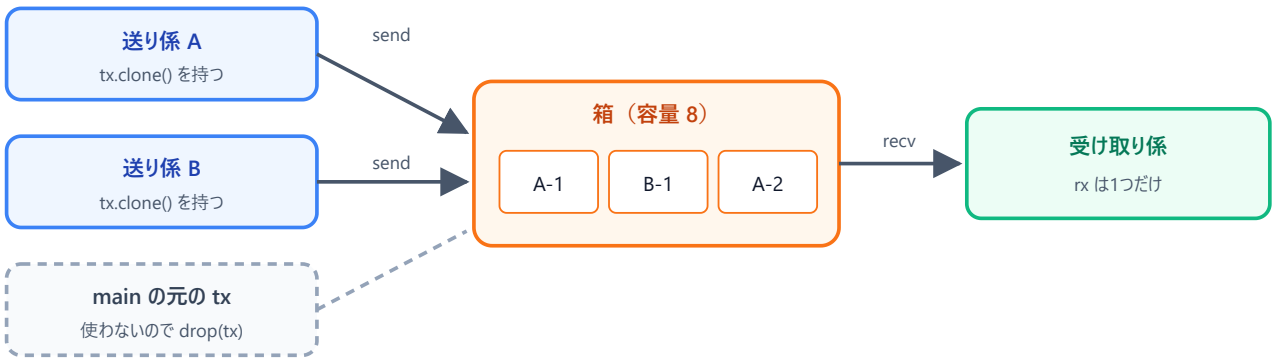


図5-12: mpsc の channel。送る口は clone して何人にも配れ、受け取る口は1つ。送る口が全部なくなったら、受け取り係は「もう来ない」と分かって終われる

- 送る口 `tx`（transmitter）は `clone` で増やせます。どの送り手が送った値も、同じ箱に入ります。
- 受け取る口 `rx`（receiver）は1つだけです。箱に入った順に、1件ずつ受け取ります。
- `main` が最初に作った `tx` も、送る口の1つに数えられます。使わないなら `drop(tx)` で片付けないと、受け取り係は「まだ送ってくる人がいる」と思って待ち続けます（「よくあるエラー4」）。

次は、受け取り係が遅いときに、送り係がどう待たされるかです。容量 2 の箱に、送り係が5件を一気に入れようとし、受け取り係は1件に 0.3 秒かかります（例4-2）。

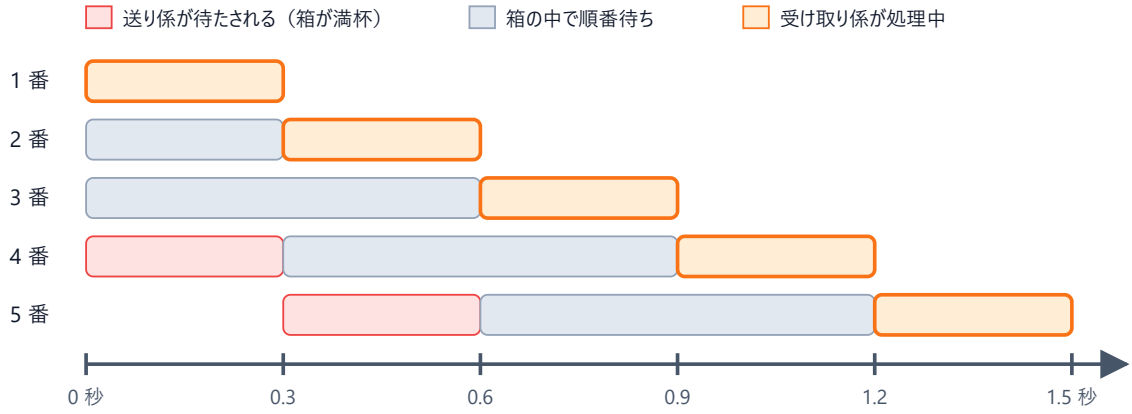


図5-13: 容量 2 の箱で起きる backpressure。4番と5番は、箱に空きができるまで送り係が待たされる

最後に、tokio にある4種類の channel を並べておきます。この単元で、全部を1回ずつ使います。

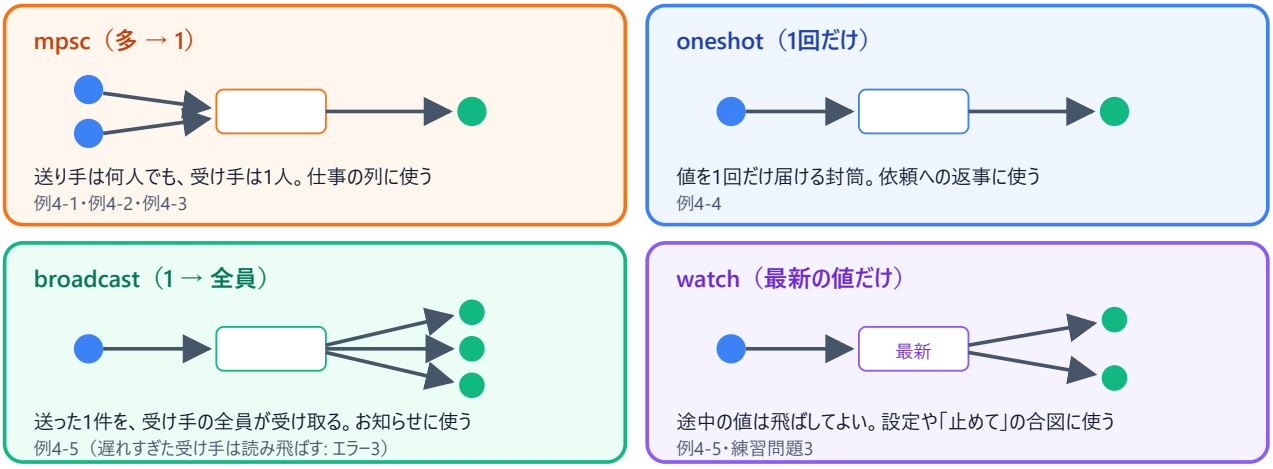


図5-14: tokio の4種類の channel。仕事の列は mpsc、1回の返事は oneshot、全員へのお知らせは broadcast、最新の状態の共有は watch を使う

まず動かす最小コード

例4-1: 2人の送り係から、1人の受け取り係へ依頼を渡す

> **▼ このコードがやること（先に日本語で）**: 「このノートを保存して」という依頼 `SaveRequest` を運ぶ channel を作ります。送り係のタスクを2つ（A と B）作り、それぞれ2件ずつ依頼を送ります。受け取り係のタスクは、箱から1件ずつ受け取って表示し、送る口が全部閉じたら、受け取った件数を返して終わります。

```rust
use tokio::sync::mpsc;

// 送るもの: 「このノートを保存して」という依頼
struct SaveRequest {
    title: String,
}

#[tokio::main]
async fn main() {
    // 箱に入るのは最大 8 件（bounded = 上限つき）。tx が送る口、rx が受け取る口
    let (tx, mut rx) = mpsc::channel::<SaveRequest>(8);

    // 受け取る側のタスク（1つだけ）
    let worker = tokio::spawn(async move {
        let mut saved = 0;
        // recv は「次の1件」を待つ。送る口が全部片付けられると None が返ってループが終わる
        while let Some(request) = rx.recv().await {
            println!("  受け取り: {}", request.title);
            saved += 1;
        }
        println!("  送る口がすべて閉じたので終わります");
        saved
    });

    // 送る側のタスクを2つ作る。送る口は clone して1つずつ渡す
    let mut senders = Vec::new();
    for who in ["A", "B"] {
        let tx = tx.clone();
        senders.push(tokio::spawn(async move {
            for n in 1..=2 {
                let title = format!("{who}-{n}");
                // send に渡した値は、受け取る側へ「持ち主ごと」移る
                tx.send(SaveRequest { title }).await.unwrap();
            }
        }));
    }
    // main が持っている元の送る口も片付ける（忘れると recv が永遠に待つ）
    drop(tx);

    for sender in senders {
        sender.await.unwrap();
    }
    let saved = worker.await.unwrap();
    println!("保存した件数: {saved}");
}
```

`src/bin/u04_mpsc.rs` として保存して実行します。

```
cargo run -q --bin u04_mpsc
```

▼ 実行結果

```
  受け取り: B-1
  受け取り: A-1
  受け取り: A-2
  受け取り: B-2
  送る口がすべて閉じたので終わります
保存した件数: 4
```

▼ コードを1つずつ分解して解説

解説1: mpsc::channel(容量) で、送る口と受け取る口を1組作る

- `mpsc::channel::<SaveRequest>(8)` は、「`SaveRequest` を最大8件まで入れておける箱」を作り、送る口 `tx` と受け取る口 `rx` を組で返します。`::<SaveRequest>` は運ぶ値の型の指定です。
- 容量の決まった channel を **bounded**（上限つき）と呼びます。上限の無い `mpsc::unbounded_channel` もありますが、送り係が待たされないので、受け取り係が遅いと箱が際限なく大きくなり、メモリを使い果たすおそれがあります。この巻では、上限つきを使います。

解説2: 送る口は clone して配り、受け取る口は1つだけ

- `let tx = tx.clone();` で送る口を増やし、`async move` でそれぞれの送り係のタスクへ移しています（UNIT 02 の練習問題1と同じ形）。
- 受け取る口 `rx` は `clone` できません。受け取り係のタスクに `async move` で移して、そのタスクだけが受け取ります。
- 実行結果の順番（B-1 → A-1 → A-2 → B-2）は、2つの送り係のどちらが先に動いたかで決まり、実行するたびに変わることがあります。ただし、同じ送り係が送った A-1 と A-2 の順番は、必ず守られます。

解説3: send した値は、持ち主ごと受け取り係へ移る

- `tx.send(SaveRequest { title })` に渡した値は、箱を通って受け取り係へ**移動**します。送った後の送り係の手元には、もう残っていません（送った後に使おうとすると E0382:「よくあるエラー1」）。
- 受け取り係は、受け取った `request` の持ち主になります。だから、自由に読んだり書き換えたり、別の場所へ渡したりできます。コピーではないので、大きな値でも送るのは速く、2つのタスクが同じ値を同時にさわることもありません。
- `send` は、箱に空きがあればすぐに終わり、いっぱいなら空くまで待つ async 関数です（例4-2）。受け取り係がもういなければ `Err` を返します（「よくあるエラー2」）。

解説4: 送る口が全部片付くと、recv が None を返す

- `rx.recv().await` は、次の1件が来るまで待ち、来たら `Some(値)` を返します。
- **送る口が1つ残らず片付けられ**、箱も空になると、`None` を返します。`while let Some(request) = ...` のループは、これで自然に終わります。

- 送る口は3つありました。送り係 A と B の `tx` は、それぞれのタスクが終わるときに片付けられます。`main` の元の `tx` は、`drop(tx)` で明示的に片付けています。

もう一歩: 実務に近い形

例4-2: 受け取り係が遅いと、送り係が待たされる（backpressure）

> **▼ このコードがやること（先に日本語で）**: 容量 2 の箱を作ります。受け取り係は1件の処理に 0.3 秒かかります。送り係は5件を一気に送ろうとします。送り係が箱に入れられた時刻と、受け取り係が処理を終えた時刻を表示して、送り係がどこで待たされたかを確かめます。

```
use std::time::{Duration, Instant};
use tokio::sync::mpsc;

#[tokio::main]
async fn main() {
    let start = Instant::now();
    // 箱の容量は 2 件だけ
    let (tx, mut rx) = mpsc::channel::<u32>(2);

    // 遅い受け取り係: 1件の処理に 300ms かかる
    let consumer = tokio::spawn(async move {
        while let Some(job) = rx.recv().await {
            tokio::time::sleep(Duration::from_millis(300)).await;
            println!(
                "{:.1} 秒:   受け取り係 {job} 番を処理した",
                start.elapsed().as_secs_f64()
            );
        }
    });

    // 速い送り係: 5件を一気に送ろうとする
    for job in 1..=5 {
        // 箱がいっぱいなら、空きができるまでここで待たされる（= バックプレッシャー）
        tx.send(job).await.unwrap();
        println!(
            "{:.1} 秒: 送り係 {job} 番を箱に入れた",
            start.elapsed().as_secs_f64()
        );
    }
    drop(tx);
    consumer.await.unwrap();
}
```

`src/bin/u04_backpressure.rs` として保存して実行します。

```
cargo run -q --bin u04_backpressure
```

▼ 実行結果

```
0.0 秒: 送り係 1 番を箱に入れた
0.0 秒: 送り係 2 番を箱に入れた
0.0 秒: 送り係 3 番を箱に入れた
0.3 秒:    受け取り係 1 番を処理した
0.3 秒: 送り係 4 番を箱に入れた
0.6 秒:    受け取り係 2 番を処理した
0.6 秒: 送り係 5 番を箱に入れた
0.9 秒:    受け取り係 3 番を処理した
1.2 秒:    受け取り係 4 番を処理した
1.5 秒:    受け取り係 5 番を処理した
```

▼ コードを1つずつ分解して解説

解説1: 容量 2 なのに、0 秒で3件入ったのはなぜか

- 1番は、箱に入った直後に受け取り係が `recv` で取り出し、処理を始めました。箱の中には残っていません。
- その後の2番と3番で、箱の2件分がいっぱいになります。だから 0 秒の時点で入れられたのは3件（処理中の1件と、箱の中の2件）です。図の「1番」から「3番」の行のとおりです。

解説2: 4番と5番は、箱に空きができるまで待たされた

- 4番の `tx.send(4).await` は、箱がいっぱいなので、そこで止まって待ちます。0.3 秒に受け取り係が1番を終えて2番を取り出すと、箱に空きが1つでき、4番が入りました。5番も同じように 0.6 秒まで待たされています。
- 送り係は、**受け取り係の速さ（0.3 秒に1件）でしか先に進めなくなりました。**これが backpressure です。箱の中に貯まる仕事は、いつも2件までです。

解説3: もし上限が無かったら

- 上限の無い channel では、送り係は待たされずに5件とも 0 秒で入れ終わり、箱の中に4件が貯まります。5件なら問題ありませんが、1秒に1000件の依頼が来て、受け取り係が1秒に3件しか処理できなければ、箱の中身は1秒に997件ずつ増え続けます。いつかメモリが足りなくなり、サーバーごと落ちます。
- 上限があれば、送り係の側（たとえば HTTP のリクエストを受けているタスク）が待たされるので、それ以上の仕事を受け付けられなくなります。「待たせる」代わりに「すぐ断る」こともできます（次の例4-3）。

例4-3: 混んでいるときは、待たずにすぐ断る（try_send）

> **▼ このコードがやること（先に日本語で）**：容量 2 の箱に、受け取り係が動き出す前に5件の依頼が一気に来たとします。`try_send` で入れようとし、箱がいっぱいなら待たずに「混んでいるので断りました」と表示します。最後に、受け付けた分だけを処理します。

```rust
use tokio::sync::mpsc::{self, error::TrySendError};

#[tokio::main]
async fn main() {
    // 箱の容量は 2 件
    let (tx, mut rx) = mpsc::channel::<u32>(2);

    // 受け取り係が動き出す前に、5 件の依頼が一気に来たとする
    for job in 1..=5 {
        // try_send は待たない。箱がいっぱいなら、すぐに Full のエラーを返す
        match tx.try_send(job) {
            Ok(()) => println!("{job} 番: 受け付けました"),
            Err(TrySendError::Full(job)) => {
                println!("{job} 番: 混んでいるので断りました（503 を返す）");
            }
            Err(TrySendError::Closed(job)) => println!("{job} 番: 受け取り係がいません"),
        }
    }
    drop(tx);

    // 受け付けた分だけを処理する
    while let Some(job) = rx.recv().await {
        println!("受け取り係: {job} 番を処理しました");
    }
}
```

`src/bin/u04_try_send.rs` として保存して実行します。

```
cargo run -q --bin u04_try_send
```

▼ 実行結果

```
1 番: 受け付けました
2 番: 受け付けました
3 番: 混んでいるので断りました（503 を返す）
4 番: 混んでいるので断りました（503 を返す）
5 番: 混んでいるので断りました（503 を返す）
受け取り係: 1 番を処理しました
受け取り係: 2 番を処理しました
```

▼ コードを1つずつ分解して解説

解説1: try_send は async ではなく、すぐに結果を返す

- `tx.try_send(値)` は `.await` を付けません。箱に空きがあれば入れて `Ok(())`、いっぱいなら待たずに `Err(TrySendError::Full(値))` を返します。受け取り係がもういなければ `Err(TrySendError::Closed(値))` です。
- どちらのエラーも、入れられなかった値を**返してくれます**（`Full(job)` の `job`）。値は持ち主ごと移る約束なので、失敗したときに値が消えてしまわないよう、送り手に戻ってくるのです。

解説2: 「待たせる」か「断る」かは、場面で選ぶ

- Web サーバーでは、混雑したときに利用者を何十秒も待たせるより、すぐに「503 Service Unavailable（今は混んでいます）」を返して、少し後でやり直してもらうほうが親切なことがあります。`try_send` の `Full` は、その判断のきっかけになります。
- 一方、裏で動く集計のように、少しくらい遅れてもかまわない仕事なら、`send(...).await` で待たせて、確実に受け付けるほうが向いています。

> **注意:** `select!` の枝で `tx.send(値)` を使い、ほかの枝が先に終わると、送るはずだった値は**捨てられて失われます**（tokio 1.53.3 のソースに「the message is dropped and will be lost」と書かれています）。UNIT 03 のキャンセル安全の話です。一方、`rx.recv()` はキャンセル安全で、負けても箱の中の値は失われません。

例4-4: 返事は oneshot で受け取る（アクターの形）

> **▼ このコードがやること（先に日本語で）**：合計の数字を1つだけ持つ「カウンター係」のタスクを作ります。ほかのタスクは、mpsc の channel で「足して」「今の合計を教えて」と依頼を送ります。依頼には、返事を入れてもらう封筒（`oneshot` の送る口）を同封します。10・20・30 を順に足し、最後に合計を聞きます。

```rust
use tokio::sync::{mpsc, oneshot};

// 係（アクター）への依頼。返事を入れてもらう封筒（oneshot の送る口）を同封する
enum Command {
    Add {
        amount: u64,
        reply: oneshot::Sender<u64>,
    },
    Get {
        reply: oneshot::Sender<u64>,
    },
}

// カウンター係: 数字はこのタスクだけが持つので、Mutex がいらない
async fn counter_actor(mut rx: mpsc::Receiver<Command>) {
    let mut total = 0;
    while let Some(command) = rx.recv().await {
        match command {
            Command::Add { amount, reply } => {
                total += amount;
                // 返事は1回だけ送れる。依頼した側がもういなければ Err になるが、無視してよい
                let _ = reply.send(total);
            }
            Command::Get { reply } => {
                let _ = reply.send(total);
            }
        }
    }
}

#[tokio::main]
async fn main() {
    let (tx, rx) = mpsc::channel(16);
    tokio::spawn(counter_actor(rx));

    for amount in [10, 20, 30] {
        let (reply_tx, reply_rx) = oneshot::channel();
        tx.send(Command::Add {
            amount,
            reply: reply_tx,
        })
        .await
        .unwrap();
        // 返事が届くまで待つ
        let total = reply_rx.await.unwrap();
        println!("{amount} を足した → 合計 {total}");
    }

    let (reply_tx, reply_rx) = oneshot::channel();
```

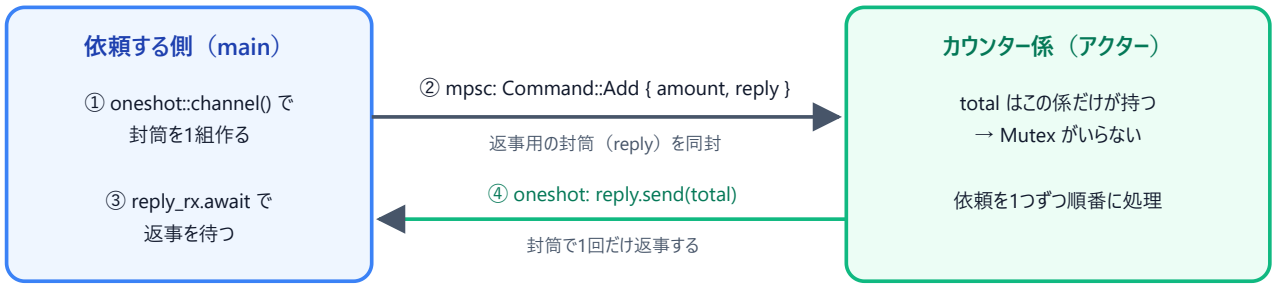
```
    tx.send(Command::Get { reply: reply_tx }).await.unwrap();
    println!("最終的な合計: {}", reply_rx.await.unwrap());
}
```

`src/bin/u04_oneshot.rs` として保存して実行します。

```
cargo run -q --bin u04_oneshot
```

▼ 実行結果

```
10 を足した → 合計 10
20 を足した → 合計 30
30 を足した → 合計 60
最終的な合計: 60
```



依頼は mpsc の列に並ぶので、何人から同時に来ても、係は1つずつ順番に処理できる

図5-15: アクターの形。依頼は mpsc で係に届け、返事は依頼に同封した oneshot の封筒で受け取る。データは係のタスクだけが持つ

▼ コードを1つずつ分解して解説

解説1: oneshot は、値を1回だけ届ける封筒

- `oneshot::channel()` は、値を**1回だけ**運べる channel の組（送る口と受け取る口）を作ります。送る口の `send` は1回しか呼べません（`send` が送る口を持ち主ごと受け取るので、2回目は書けない作りになっています）。
- 受け取る口 `reply_rx` は、それ自体を `.await` すると、返事が届くまで待って `Result` を返します。係が返事をせずに封筒（送る口）を捨てたら `Err` です。

解説2: 依頼に封筒を同封すると、返事の行き先を迷わない

- `Command::Add { amount, reply: reply_tx }` のように、依頼そのものに返事用の送る口を入れて送ります。係は、受け取った依頼の `reply` に返事を入れるだけで、正しい依頼主に届きます。何人が同時に依頼しても、返事が入れ違うことはありません。
- `let _ = reply.send(total);` の `let _ =` は、「依頼主がもう待っていなくて送れなかった（`Err`）としても、気にしない」という意味です。

解説3: アクターなら、共有のデータに鍵がいらない

- 合計 `total` を持っているのは、カウンター係のタスクだけです。ほかのタスクは、直接さわらずに、依頼を送って係にやってもらいます。依頼は channel の列に並ぶので、係は1つずつ順番に処理します。
- だから、第3巻 UNIT 10 で使った `Arc<Mutex<…>>` の鍵が要りません。「データを持つ係を1人決めて、メッセージで頼む」このやり方を**アクター**（actor）と呼びます。共有のデータを守るもう1つの方法（鍵を使う方法）は、UNIT 05 で学びます。

> **用語**: アクター（actor：自分だけのデータを持ち、メッセージで届いた依頼を1つずつ処理するタスク。データを共有せずに済むので、鍵がいらない）

例4-5: 全員に配る broadcast と、最新の値だけを共有する watch

> **▼ このコードがやること（先に日本語で）**：前半では、broadcast の channel で「ノートが作られました」「削除されました」の2つのお知らせを送り、メール係とログ係の**両方**が受け取ることを確かめます。後半では、watch の channel で「ログの細かさ」の設定を `info` → `debug` → `warn` → `error` と変え、見張り役のタスクが受け取った値を表示します。

```rust
use std::time::Duration;
use tokio::sync::{broadcast, watch};

#[tokio::main]
async fn main() {
    // ---- broadcast: 送った1件を、受け取り手「全員」が受け取る ----
    let (news_tx, _) = broadcast::channel::<String>(16);
    let mut handles = Vec::new();
    for name in ["メール係", "ログ係"] {
        // 受け取り口は subscribe で増やす（送る前に増やしておく）
        let mut news_rx = news_tx.subscribe();
        handles.push(tokio::spawn(async move {
            while let Ok(news) = news_rx.recv().await {
                println!("  {name} が受け取り: {news}");
            }
            println!("  {name}: 送り手がいなくなったので終わり");
        }));
    }
    news_tx
        .send(String::from("ノート 1 が作られました"))
        .unwrap();
    news_tx
        .send(String::from("ノート 1 が削除されました"))
        .unwrap();
    drop(news_tx);
    for handle in handles {
        handle.await.unwrap();
    }

    // ---- watch: 「最新の値」だけを共有する。途中の値は飛ばされてもよい ----
    let (level_tx, mut level_rx) = watch::channel("info");
    let watcher = tokio::spawn(async move {
        // changed: 値が変わるまで待つ。送り手がいなくなると Err
        while level_rx.changed().await.is_ok() {
            println!("  ログの細かさが変わった: {}", *level_rx.borrow());
        }
    });
    level_tx.send("debug").unwrap();
    tokio::time::sleep(Duration::from_millis(50)).await;
    // 立て続けに2回変えると、見張り役が見るのは最新の "error" だけのことが多い
    level_tx.send("warn").unwrap();
    level_tx.send("error").unwrap();
    tokio::time::sleep(Duration::from_millis(50)).await;
    drop(level_tx);
    watcher.await.unwrap();
}
```

`src/bin/u04_broadcast_watch.rs` として保存して実行します。

```
cargo run -q --bin u04_broadcast_watch
```

▼ 実行結果

```
メール係 が受け取り: ノート 1 が作られました
メール係 が受け取り: ノート 1 が削除されました
ログ係 が受け取り: ノート 1 が作られました
メール係: 送り手がいなくなったので終わり
ログ係 が受け取り: ノート 1 が削除されました
ログ係: 送り手がいなくなったので終わり
ログの細かさが変わった: debug
ログの細かさが変わった: error
```

▼ コードを1つずつ分解して解説

解説1: broadcast は、受け取り口ごとに全部のお知らせが届く

- `broadcast::channel::<String>(16)` は、最大16件まで覚えておけるお知らせの channel です。受け取り口は `news_tx.subscribe()` で、受け手の数だけ作ります。最初に返ってくる受け取り口（`_`）は使わないので、すぐ捨てています。
- メール係もログ係も、2件のお知らせを**両方とも**受け取りました。mpsc では1件は1人にしか届きませんが、broadcast では全員に届きます。そのため、運ぶ値は `Clone` できる型でなければなりません（受け手ごとに写しが渡されます）。
- `subscribe` で作った受け取り口には、**作った後に**送られたお知らせだけが届きます。お知らせを送る前に、受け取り口を作っておく必要があります。
- 2つの係の表示が混ざっているのは、2つのタスクが別々に動いているからです。それぞれの係の中では、送った順番（作られた → 削除された）が守られています。

解説2: watch は「今の値」だけを覚えている

- `watch::channel("info")` は、最初の値 `"info"` を持った channel を作ります。`level_tx.send("debug")` で値を書き換えると、見張り役の `level_rx.changed().await` が「変わったよ」と知らせてくれます。`*level_rx.borrow()` で今の値を読みます。
- `"warn"` と `"error"` を立て続けに送ったので、見張り役が起きたときには、値はもう `"error"` になっていました。`"warn"` は表示されていません。watch は「途中の値は飛ばしてもよいから、最新の値を知りたい」ときのための channel です。ログの細かさのような設定や、「止めて」という合図（練習問題3）に向いています。

> **注意:** `level_rx.borrow()` が返すものを持ったまま `.await` しないようにしましょう。`borrow` は、値を読んでいる間、中で読み込み用の鍵をかけています。持ったまま待つと、送り手が値を書き換えられずに待たされます。UNIT 05 の「鍵を持ったまま `.await` しない」と同じ注意です。

よくあるエラーと読み方

エラー1: 送った値を、もう一度使う（E0382）

```rust
use tokio::sync::mpsc;

#[tokio::main]
async fn main() {
    let (tx, mut rx) = mpsc::channel::<String>(8);
    let title = String::from("買い物メモ");
    tx.send(title).await.unwrap();
    // 送ったあとで、もう一度 title を使おうとする
    println!("送ったもの: {title}");
    println!("受け取ったもの: {:?}", rx.recv().await);
}
```

```
   Compiling async_lab v0.1.0 (...\async_lab)
error[E0382]: borrow of moved value: `title`
 --> src\bin\e04_send_moved.rs:9:23
  |
6 |     let title = String::from("買い物メモ");
  |         ----- move occurs because `title` has type `String`, which does not
implement the `Copy` trait
7 |     tx.send(title).await.unwrap();
  |             ----- value moved here
8 |     // 送ったあとで、もう一度 title を使おうとする
9 |     println!("送ったもの: {title}");
  |                            ^^^^^ value borrowed here after move
  |
help: consider cloning the value if the performance cost is acceptable
  |
7 |     tx.send(title.clone()).await.unwrap();
  |                  ++++++++

For more information about this error, try `rustc --explain E0382`.
error: could not compile `async_lab` (bin "e04_send_moved") due to 1 previous error
```

- **何が起きたか:** 「移動した後の値 `title` を借りようとしています」というエラーです。`value moved here` の線が、`tx.send(title)` で値が箱へ移ったことを示しています。第3巻 UNIT 02 で学んだ「移動」が、channel でもそのまま起きています。
- **直し方:** 送った後にも使いたいなら、`help:` のとおり `tx.send(title.clone())` で写しを送ります。ただ、多くの場合は、**送る前に**表示などを済ませて、送った後は使わないように順番を入れ替えるのが自然です。「送ったら、もう自分の物ではない」と考えましょう。

エラー2: 受け取り係がいない channel に送る（SendError: channel closed）

コンパイルは通り、実行すると panic する例です。

```
use tokio::sync::mpsc;

#[tokio::main]
async fn main() {
    let (tx, rx) = mpsc::channel::<String>(8);
    // 受け取る側が先にいなくなった（たとえば受け取り係のタスクが panic して終わった）
    drop(rx);

    // ① エラーを確かめる書き方
    match tx.send(String::from("ノート1")).await {
        Ok(()) => println!("送れました"),
        Err(error) => println!("送れません: {error} / 戻ってきた値: {}", error.0),
    }
    // ② unwrap してしまう書き方
    tx.send(String::from("ノート2")).await.unwrap();
}
```

```
   Compiling async_lab v0.1.0 (...\async_lab)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.72s
     Running `target\debug\e04_send_closed.exe`
送れません: channel closed / 戻ってきた値: ノート1

thread 'main' (25592) panicked at src\bin\e04_send_closed.rs:15:44:
called `Result::unwrap()` on an `Err` value: SendError { .. }
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
error: process didn't exit successfully: `target\debug\e04_send_closed.exe` (exit code:
101)
```

- **何が起きたか:** 受け取る口 `rx` が片付けられた後なので、`send` は `Err(SendError)` を返しました。①では `channel closed`（channel は閉じています）と表示し、送れなかった値（`error.0` の「ノート1」）も戻ってきています。②では、その `Err` を `unwrap` したので panic しました。
- **直し方:** 受け取り係のタスクは、panic などで先に終わることがあります。`send` の結果は `unwrap` せず、`Err` なら「受け取り係がいない」として扱います。サーバーなら、ログに残して 500（サーバーの中の問題）を返すのがふつうです。
- 例4-1の `unwrap` は、受け取り係が送り手より先に終わらない作りなので、練習のために省略していました。実際のアプリでは、`expect("…")` で理由を書くか、エラーとして返しましょう。

エラー3: broadcast で、遅れた受け手が読み飛ばす（RecvError::Lagged）

```
use tokio::sync::broadcast;

#[tokio::main]
async fn main() {
    // 容量 2 の broadcast
    let (tx, mut rx) = broadcast::channel::<u32>(2);
    // 受け取り手が読む前に 5 件送ってしまう
    for n in 1..=5 {
        tx.send(n).unwrap();
    }
    // 読み始める
    loop {
        match rx.recv().await {
            Ok(n) => println!("受け取り: {n}"),
            Err(broadcast::error::RecvError::Lagged(skipped)) => {
                println!("追いつけず {skipped} 件を読み飛ばしました");
            }
            Err(broadcast::error::RecvError::Closed) => break,
        }
        // 箱の中に読むものが残っていなければ終わる
        if rx.is_empty() {
            break;
        }
    }
}
```

```
追いつけず 3 件を読み飛ばしました
受け取り: 4
受け取り: 5
```

（`cargo run` の `Compiling ...` などの行は省いています。）

- **何が起きたか**: broadcast は、mpsc と違って送り手を待たせません。容量（2件）を超えて送ると、**古いお知らせから上書きして**消していきます。遅れていた受け手が次に `recv` すると、まず `Err(RecvError::Lagged(3))`（3件を読み飛ばしました）が返り、その後で残っている新しい2件（4 と 5）を受け取れます。
- 1件の送り手のせいで全員が止まらないように、遅れた受け手のほうが読み飛ばす作りになっているのです。
- **直し方**: 読み飛ばしてもよいお知らせなら、`Lagged` を受け取ったらログに残して続けます。1件も落としてはいけない仕事なら、broadcast ではなく、受け手ごとに mpsc の channel を用意するなど、別の作りにします。容量を大きくすると、読み飛ばしは起きにくくなります。

エラー4（エラーが出ない間違い）: drop(tx) を忘れて、受け取り係が終わらない

```rust
use std::time::Duration;
use tokio::sync::mpsc;

#[tokio::main]
async fn main() {
    let (tx, mut rx) = mpsc::channel::<String>(8);

    let worker = tokio::spawn(async move {
        let mut saved = 0;
        while let Some(title) = rx.recv().await {
            println!("  受け取り: {title}");
            saved += 1;
        }
        saved
    });

    let sender = {
        let tx = tx.clone();
        tokio::spawn(async move {
            for n in 1..=2 {
                tx.send(format!("A-{n}")).await.unwrap();
            }
        })
    };
    sender.await.unwrap();
    // drop(tx) を書き忘れた。main の tx が残っているので、箱は閉じない

    // 受け取り係の終わりを、1 秒だけ待ってみる
    tokio::select! {
        saved = worker => println!("保存した件数: {}", saved.unwrap()),
        _ = tokio::time::sleep(Duration::from_secs(1)) => {
            println!("1 秒待っても、受け取り係が終わりません");
        }
    }
}
```

`src/bin/u04_forgot_drop.rs` として実行した結果です（そのままだと永遠に終わらないので、UNIT 03 の `select!` で1秒だけ待つようにしています）。

```
  受け取り: A-1
  受け取り: A-2
1 秒待っても、受け取り係が終わりません
```

- **何が起きたか:** 2件は受け取れましたが、受け取り係は終わりません。送り係のタスクの `tx` は片付けられましたが、`main` が最初に作った `tx` が残っているので、受け取り係の `recv` は「まだ送ってくるかもしれない」と待ち続けています。エラーも panic も出ず、プログラムが止まったように見えます。
- **直し方:** 送り係に `clone` を配り終えたら、`main` の元の `tx` を `drop(tx)` で片付けます（例4-1）。あるいは、`clone` せずに元の `tx` をそのまま最後の送り係へ移す書き方もあります。
- `select!` の枝に `worker`（`JoinHandle`）をそのまま書けたのは、`JoinHandle` が `Unpin` な Future だからです（UNIT 03 のエラー2のような `pin!` は要りません）。

> **ヒント:** 「受け取りのループが終わらない」ときは、まず「送る口（`tx`）をまだ持っている人はだれか？」を数えましょう。`clone` した数と、片付けた数が合っていないことがほとんどです。

RustNote ではここで使う

RustNote のコードそのものには、channel は出てきません。1つのリクエストの中で、ほかのタスクへ仕事を渡す場面がまだ無いからです（UNIT 10 のバックグラウンドジョブで、仕事を渡すときに使います）。

一方、RustNote を動かしている Topcoat の `internal_serve`（`topcoat-router-0.10.0/src/service.rs`）では、watch の channel が「止める合図」として使われています。Topcoat 0.10.0 のソースから、channel を作る部分と、合図を出す部分を抜き出します（コメントは日本語に置き換え、関係の無い部分は省略しています）。

```
// どの channel も「値を送る」のではなく「閉じる」ことで合図する。
// 接続のタスクは受け取り口の写しを持ち、送り口が片付くと changed() が終わる。
// 止め始めの合図
let (drain_tx, drain_rx) = watch::channel(());
// 猶予の終わりの合図
let (cutoff_tx, cutoff_rx) = watch::channel(());
// 生きているタスクを数える。各タスクが受け取り口の写しを持ち、
// 最後の1つが片付くと done_tx.closed() が終わる
let (done_tx, done_rx) = watch::channel(());

// （接続を受け付けるループ。接続ごとに drain_rx などの clone をタスクへ渡す。省略）

// 止め始めの合図を出す: 送り口を片付ける
drop(drain_rx);
drop(drain_tx);
drop(done_rx);
// 全部のタスクが終わるか、猶予の時間が過ぎるまで待つ
tokio::select! {
    () = done_tx.closed() => {}
    () = tokio::time::sleep(service.shutdown_timeout) => {}
}
```

- **送り口を片付けることが合図になっています。**例4-1で、送る口が全部片付くと `recv` が `None` を返したのと同じく、watch でも送り口が片付くと、受け取り口の `changed()` が `Err` を返して終わります。値（ここでは中身の無い `()`）を送る必要さえありません。
- **受け取り口の数で、生きているタスクを数えています。**接続のタスクは `done_rx` の写しを1つずつ持ち、タスクが終わると写しが片付きます。最後の1つが片付くと、`done_tx.closed()` が終わります。「全部の接続が終わるまで待つ」が、channel の受け取り口の数だけで書けています。
- 猶予の時間（`shutdown_timeout`）と `select!` で競争させて、待ちすぎないようにしています。この止め方の全体は、UNIT 06 の graceful shutdown で読みます。

練習問題

問1: 例4-2の `u04_backpressure.rs` で、箱の容量を `2` から `5` に変えると、送り係が箱に入れる時刻と、受け取り係が最後の5番を処理し終える時刻はどう変わるでしょうか。予想してから実行してください（`src/bin/u04_q1.rs`）。

問2: 例4-4のカウンター係に、「合計を 0 に戻し、戻す前の合計を返事する」依頼 `Reset` を足してください。10 を足す → 20 を足す → リセット → 5 を足す、の順に依頼し、それぞれの返事を表示します（`src/bin/u04_q2.rs`）。返事の封筒を作って送って待つ、の3行は、毎回同じなので関数にまとめてみましょう。

問3: `watch::channel(false)` を「止める合図」に使います。仕事係のタスクは、0.1 秒ごとに「仕事 N 回目」と表示し、合図の値が `true` になったら止まって、仕事をした回数を返します。`main` は 0.35 秒たったら合図を送り、仕事係が止まるのを待ってください。UNIT 03 の `select!` を使います（`src/bin/u04_q3.rs`）。

解答と解説

問1の解答: `mpsc::channel::<u32>(5)` に変えて実行した結果です（コメントも「箱の容量を 5 件に増やす」に直しました。ほかは例4-2と同じです）。

```
0.0 秒: 送り係 1 番を箱に入れた
0.0 秒: 送り係 2 番を箱に入れた
0.0 秒: 送り係 3 番を箱に入れた
0.0 秒: 送り係 4 番を箱に入れた
0.0 秒: 送り係 5 番を箱に入れた
0.3 秒:    受け取り係 1 番を処理した
0.6 秒:    受け取り係 2 番を処理した
0.9 秒:    受け取り係 3 番を処理した
1.2 秒:    受け取り係 4 番を処理した
1.6 秒:    受け取り係 5 番を処理した
```

- 箱に5件とも入るので、送り係は待たされずに 0 秒で全部を入れ終わりました。

- それでも、受け取り係が最後の5番を終えるのは、容量 2 のときと同じ約 1.5 秒です（1.6 秒になっているのは、タイマーの遅れが積み重なったためです）。**容量を大きくしても、全体の処理は速くなりません。**変わるのは「送り係が待つか」と「箱の中に何件まで貯まるか（メモリ）」だけです。全体を速くしたいなら、受け取り係を速くするか、増やす必要があります。

問2の解答:

```rust
use tokio::sync::{mpsc, oneshot};

// 係（アクター）への依頼
enum Command {
    Add {
        amount: u64,
        reply: oneshot::Sender<u64>,
    },
    // 合計を 0 に戻し、戻す前の合計を返事する
    Reset {
        reply: oneshot::Sender<u64>,
    },
}

async fn counter_actor(mut rx: mpsc::Receiver<Command>) {
    let mut total = 0;
    while let Some(command) = rx.recv().await {
        match command {
            Command::Add { amount, reply } => {
                total += amount;
                let _ = reply.send(total);
            }
            Command::Reset { reply } => {
                let old = total;
                total = 0;
                let _ = reply.send(old);
            }
        }
    }
}

// 依頼を送り、返事を待つ。返事の封筒は、依頼を作る関数 make で中に入れてもらう
async fn ask(
    tx: &mpsc::Sender<Command>,
    make: impl FnOnce(oneshot::Sender<u64>) -> Command,
) -> u64 {
    let (reply_tx, reply_rx) = oneshot::channel();
    tx.send(make(reply_tx)).await.expect("係がいません");
    reply_rx.await.expect("係が返事をしませんでした")
}

#[tokio::main]
async fn main() {
    let (tx, rx) = mpsc::channel(16);
    tokio::spawn(counter_actor(rx));

    let total = ask(&tx, |reply| Command::Add { amount: 10, reply }).await;
    println!("10 を足した → 合計 {total}");
    let total = ask(&tx, |reply| Command::Add { amount: 20, reply }).await;
```

```
    println!("20 を足した → 合計 {total}");
    let old = ask(&tx, |reply| Command::Reset { reply }).await;
    println!("リセットした（それまでの合計 {old}）");
    let total = ask(&tx, |reply| Command::Add { amount: 5, reply }).await;
    println!("5 を足した → 合計 {total}");
}
```

```
10 を足した → 合計 10
20 を足した → 合計 30
リセットした（それまでの合計 30）
5 を足した → 合計 5
```

- `Command` に `Reset` を足し、係の `match` に「`old` に今の合計を取っておき、0 に戻して、`old` を返事する」枝を足しました。`match` は全部の場合を書かないとコンパイルエラーになるので、依頼の種類を足したのに係の処理を書き忘れる、ということが起きません（第2巻 UNIT 08）。
- `ask` 関数は、「封筒を作る → 依頼に入れて送る → 返事を待つ」をまとめたものです。依頼の種類ごとに中身が違うので、「封筒を受け取って `Command` を作る関数」（クロージャ）を `make` として渡しています。`|reply| Command::Reset { reply }` は、「封筒 `reply` を受け取って、`Command::Reset { reply }` を作る」クロージャです（第2巻 UNIT 10）。
- 送る口を `&mpsc::Sender<Command>` で借りているのは、`send` が `&self`（借りるだけ）で呼べるからです。

問3の解答:

```rust
use std::time::{Duration, Instant};
use tokio::sync::watch;

// 止める合図（true）が来るまで、0.1 秒ごとに仕事をする
async fn worker(mut stop: watch::Receiver<bool>, start: Instant) -> u32 {
    let mut count = 0;
    loop {
        tokio::select! {
            changed = stop.changed() => {
                // 送り手がいなくなった（Err）か、値が true になったら止まる
                if changed.is_err() || *stop.borrow() {
                    break;
                }
            }
            _ = tokio::time::sleep(Duration::from_millis(100)) => {
                count += 1;
                println!("{:.1} 秒: 仕事 {count} 回目", start.elapsed().as_secs_f64());
            }
        }
    }
    count
}

#[tokio::main]
async fn main() {
    let start = Instant::now();
    let (stop_tx, stop_rx) = watch::channel(false);
    let handle = tokio::spawn(worker(stop_rx, start));

    tokio::time::sleep(Duration::from_millis(350)).await;
    println!(
        "{:.2} 秒: 止める合図を送ります",
        start.elapsed().as_secs_f64()
    );
    stop_tx.send(true).unwrap();

    let count = handle.await.unwrap();
    println!(
        "{:.2} 秒: 止まりました（仕事は {count} 回）",
        start.elapsed().as_secs_f64()
    );
}
```

```
0.1 秒: 仕事 1 回目
0.2 秒: 仕事 2 回目
0.3 秒: 仕事 3 回目
0.36 秒: 止める合図を送ります
0.36 秒: 止まりました（仕事は 3 回）
```

- 仕事係は、`select!` で「合図が変わった」と「0.1 秒たった」を競争させています。合図を受けると、すぐに（0.36 秒のうちに）止まりました。4回目の仕事の `sleep` は、途中で捨てられています（UNIT 03 の例3-5と同じです）。
- `stop.changed()` を毎回のループで作り直してよいのは、tokio のソースで「`changed` はキャンセル安全です」と明記されているからです。負けても、合図を見逃すことはありません。
- `changed.is_err()` も確かめているのは、送り手 `stop_tx` が片付けられたときに、`changed()` がすぐ `Err` を返し続けるからです。これを確かめないと、ループが空回りし続けます。送り手がいなくなったことも、「止まれ」の合図として扱います。
- この「watch の合図で、仕事係を止める」形は、UNIT 06 でサーバーを止めるときに使う形の、小さな版です。

まとめ

- mpsc の channel は、何人もの送り手から1人の受け手へ値を**持ち主ごと**渡す。送る口が全部片付くと `recv` が `None` を返して受け取りのループが終わるので、`main` の `tx` の `drop` を忘れない。
- 容量の決まった channel は、受け手が遅いと送り手を待たせる（backpressure）。`try_send` なら待たずに断れる。容量を増やしても全体は速くならない。
- 1回の返事は oneshot（依頼に封筒を同封するとアクターが作れる）、全員へのお知らせは broadcast（遅れると `Lagged`）、最新の値や止める合図は watch を使う。

UNIT 05: Mutex・RwLock・Semaphore と .await

この単元のゴール

- 第3巻 UNIT 10 の `std::sync::Mutex` を async の中で使うときの決まり「**鍵を持ったまま `.await` しない**」を守り、`{ }` のブロックか、async でない関数で、鍵を持つ範囲を区切れる
- 鍵（`MutexGuard`）を持ったまま `.await` したときのエラー「future cannot be sent between threads safely」を読み、`drop(guard)` を書いても直らない理由と、本当の直し方を説明できる
- spawn しないとエラーにならず、プログラムが本当に止まってしまう（デッドロック）ことを実際に確かめ、clippy の `await_holding_lock` で見つけられる
- 鍵を持ったまま `.await` する必要があるときに、`tokio::sync::Mutex` を選べる
- `RwLock`（読み書き鍵）で読む人を同時に通し、`Semaphore`（セマフォ）で同時に入れる数に上限を付けられる

たとえ話でつかむ

Mutex は、1人ずつしか入れない倉庫の鍵です（第3巻 UNIT 10 と同じたとえです）。鍵を持っている人だけが倉庫に入り、中の品物（データ）を数えたり並べ替えたりできます。用が済んだら鍵を返し、次の人が入ります。

async の世界では、ここに1つ、気をつけることが加わります。

- `.await` は「待つ間、ほかの仕事に席を譲る」印でした（UNIT 01）。鍵を持ったまま `.await` するのは、**倉庫の鍵をポケットに入れたまま、昼ごはんを食べに出かける**ようなものです。その間、だれも倉庫に入れません。
- 標準ライブラリの `std::sync::Mutex` で鍵を待つ人は、**扉の前に立ったまま、一歩も動きません**（スレッドごと止まる）。その人が、昼ごはんから戻ってくるはずの店員本人だったら、もう二度と鍵は返ってきません。これが**デッドロック**です。
- tokio の `tokio::sync::Mutex` で鍵を待つ人は、**番号札を取って、別の仕事をしに行きます**（`.await` で席を譲る）。鍵が空いたら呼び出してもらえます。

ほかに2つの道具を使います。

- **RwLock は、図書館の閲覧室**です。本を読むだけの人は、何人でも同時に入れます。掲示物を張り替える人（書く人）は、読んでいる人が全員出てから、1人で入ります。
- **Semaphore は、入場整理券**です。券は決まった枚数しかなく、券を持っている人だけが中に入れます。出るときに券を返すと、次の人が入れます。

> **用語:** Mutex（ミューテックス：mutual exclusion＝相互排除。中のデータを、同時に1つのスレッドやタスクだけがさわれるようにする鍵）

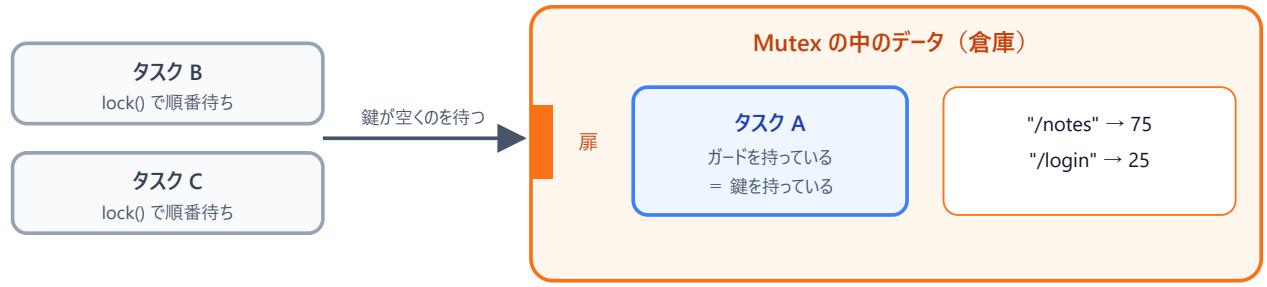
> **用語:** ガード（guard： `lock()` が返す「鍵を持っている証拠」。ガードを通して中のデータをさわり、ガードが片付けられると鍵が返る）

> **用語:** デッドロック（deadlock：お互いに相手が鍵を返すのを待っていて、だれも先へ進めなくなった状態。エラーも出ずに、プログラムが止まったように見える）

> **用語:** Semaphore（セマフォ：決まった枚数の許可証（permit）を配る道具。許可証を持っている数だけが、同時に中に入れる）

図で理解する

まず、鍵とガードの関係をおさらいします。



① lock() で鍵を取る → ② ガードを通して中をさわる → ③ ガードが片付く（} を抜ける）と、鍵が次の人へ渡る

async での決まり: ガードを持ったまま .await しない（鍵を持ったまま出かけない）

図5-16: Mutex の鍵とガード。ガードを持っている1人だけが中のデータをさわれる。ガードが片付くと、待っていた次の人が鍵を取る

次に、**標準の Mutex の鍵を持ったまま `.await` すると何が起きるか**です。スレッドが1本だけのランタイムで、タスク A と B が同じ鍵を使う場合を考えます（「よくあるエラー4」で実際に動かします）。

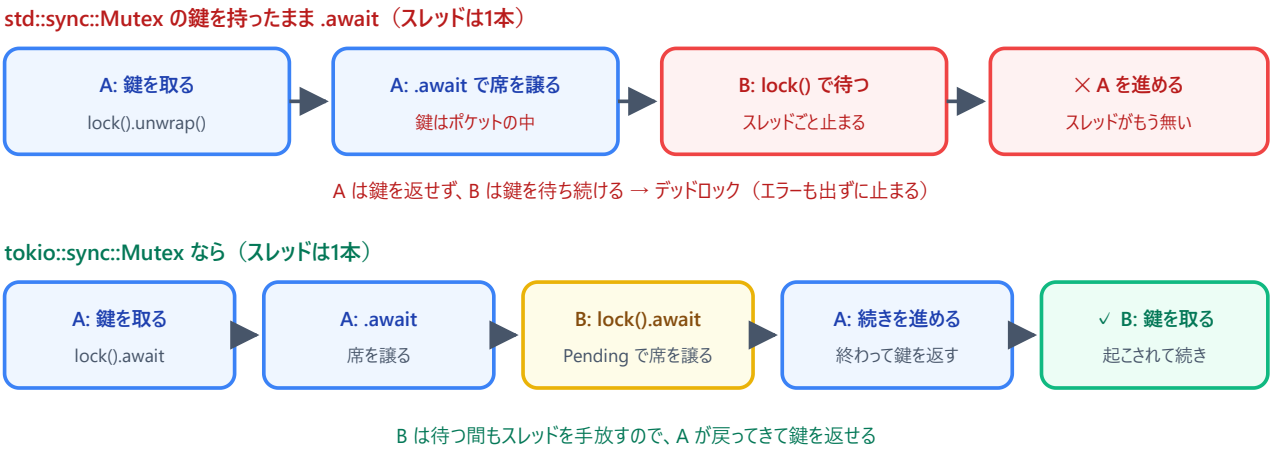


図5-17: 標準の Mutex と tokio の Mutex の違い。標準の Mutex で鍵を待つとスレッドごと止まり、鍵を返すはずのタスクが動けなくなる。tokio の Mutex は待つ間も席を譲る

- 標準の `std::sync::Mutex` の `lock()` は、鍵が空くまで**スレッドそのものを止めて**待ちます。第3巻のように、1つのスレッドで1つの仕事をしていたときは、それで困りませんでした。
- async では、1本のスレッドがたくさんのタスクを切り替えて進めています（UNIT 02）。そのスレッドが止まると、同じスレッドで動くはずだったタスクも全部止まります。鍵を持ったまま席を譲ったタスクも、その中に入っています。
- だから決まりは1つです。**標準の Mutex の鍵は、`.await` の前に必ず返す。**鍵を持ったまま `.await` したいときだけ、`tokio::sync::Mutex` を使います。

まず動かす最小コード

例5-1: 標準の Mutex で数え、鍵を返してから .await する

> **▼ このコードがやること（先に日本語で）**：ページごとのアクセス数を数える表（ `HashMap` ）を、標準の `Mutex` で守って、100個のタスクで共有します。それぞれのタスクは、鍵を取って数を1足し、**鍵を返してから**、時間のかかる処理（DB の代わりの 10 ミリ秒の待ち）を `.await` します。最後に、ページごとの数を表示します。

```rust
use std::collections::HashMap;
use std::sync::{Arc, Mutex};
use std::time::Duration;

// ページごとのアクセス数を数える（標準ライブラリの Mutex で守る）
type Counter = Arc<Mutex<HashMap<String, u64>>>;

async fn handle_request(counter: Counter, path: &str) {
    // 鍵を取って数を足す。{ } を抜けた所でガードが片付けられ、鍵が返る
    {
        let mut map = counter.lock().unwrap();
        *map.entry(path.to_string()).or_insert(0) += 1;
    }
    // 鍵を返してから、時間のかかる処理（DB や外部 API の代わり）を待つ
    tokio::time::sleep(Duration::from_millis(10)).await;
}

#[tokio::main]
async fn main() {
    let counter: Counter = Arc::new(Mutex::new(HashMap::new()));
    let mut handles = Vec::new();
    for i in 0..100 {
        let counter = Arc::clone(&counter);
        let path = if i % 4 == 0 { "/login" } else { "/notes" };
        handles.push(tokio::spawn(async move {
            handle_request(counter, path).await;
        }));
    }
    for handle in handles {
        handle.await.unwrap();
    }
    let map = counter.lock().unwrap();
    let mut paths: Vec<_> = map.iter().collect();
    paths.sort();
    println!("{paths:?}");
}
```

`src/bin/u05_std_mutex.rs` として保存して実行します。

```
cargo run -q --bin u05_std_mutex
```

▼ 実行結果

```
[("/login", 25), ("/notes", 75)]
```

▼ コードを1つずつ分解して解説

解説1: Arc で共有し、Mutex で守る（第3巻 UNIT 10 と同じ形）

- `type Counter = Arc<Mutex<HashMap<String, u64>>>;` は、長い型に `Counter` という別名を付けています（`type` の別名）。
- `Arc` は「何人もで同じものを持つ」ための入れ物、`Mutex` は「同時に1人だけがさわれる」ようにする鍵です。100個のタスクに `Arc::clone` で持ち手を配り、全員が同じ表を数えます。
- `counter.lock().unwrap()` の `unwrap` は、第3巻 UNIT 10 で学んだポイズニング（鍵を持った人が panic した）のときだけ `Err` になる部分です。この例では panic しないので `unwrap` で済ませています。
- 100個のうち、4で割り切れる25個が `/login`、残りの75個が `/notes` です。数え漏れなく、ぴったり合っています。

解説2: { } で囲んで、鍵を持つ範囲をはっきり区切る

- 鍵を使う2行を `{ }` で囲んでいます。`map`（ガード）は `{ }` の中で作った変数なので、`}` を抜けたところで片付けられ、鍵が返ります。
- その**後で** `sleep(...).await` しています。待っている間、鍵はだれも持っていないので、ほかのタスクは自由に数を足せます。
- もし `{ }` で囲まずに、鍵を持ったまま `.await` すると、コンパイルエラーになります（「よくあるエラー1」）。

解説3: async の中でも、標準の Mutex を使ってよい

- 「async の中では tokio の Mutex を使わなければならない」と思われがちですが、そうではありません。tokio のドキュメントにも「よく誤解されているが、async のコードの中で標準ライブラリの Mutex を使うのは問題なく、むしろ好ましいことが多い」と書かれています（tokio 1.53.3 の `sync/mutex.rs`）。
- 標準の Mutex のほうが軽くて速いからです。条件は1つ、**鍵を持ったまま `.await` しないこと**。数を足す・表を書き換えるように、鍵の中の仕事が短く、`.await` を含まないなら、標準の Mutex で十分です。

> **注意:** この例のように、アクセス数をメモリの中の表で数えると、サーバーを再起動したときに0に戻ります。Fly.io などで同じアプリを2台以上動かすと、台ごとに別々の表になり、合計も合いません。消えてはいけないデータや、台をまたいで共有したいデータは、DB に置きます（第7巻）。

もう一歩: 実務に近い形

例5-2: 鍵を使う部分を、async でない関数に分ける

> **▼ このコードがやること（先に日本語で）**: 数を1足して、足した後の数を返す処理を、async でないふつうの関数 `bump` に分けます。async の関数 `handle_request` は、`bump` を呼んでから `.await` で待ち、何回目かを表示します。2回呼んで確かめます。

```
use std::sync::{Arc, Mutex};
use std::time::Duration;

// 鍵を使う部分だけを、async でないふつうの関数に分ける。
// この関数の中には .await が書けないので、「鍵を持ったまま待つ」ことが起きようがない
fn bump(counter: &Mutex<u64>) -> u64 {
    let mut count = counter.lock().unwrap();
    *count += 1;
    *count
}

async fn handle_request(counter: Arc<Mutex<u64>>) {
    let now = bump(&counter);
    tokio::time::sleep(Duration::from_millis(10)).await;
    println!("{now} 回目");
}

#[tokio::main]
async fn main() {
    let counter = Arc::new(Mutex::new(0));
    tokio::spawn(handle_request(Arc::clone(&counter)))
        .await
        .unwrap();
    tokio::spawn(handle_request(Arc::clone(&counter)))
        .await
        .unwrap();
}
```

`src/bin/u05_sync_helper.rs` として保存して実行します。

```
cargo run -q --bin u05_sync_helper
```

▼ 実行結果

```
1 回目
2 回目
```

▼ コードを1つずつ分解して解説

解説1: async でない関数の中には、.await を書けない

- `bump` は `async` の付いていない関数です。UNIT 01 の「よくあるエラー2」（E0728）のとおり、この中には `.await` を書けません。
- だから、`bump` の中で取った鍵は、`bump` が戻るときに必ず返されます。「鍵を持ったまま待つ」ことが、**書こうとしても書けない**形になりました。

解説2: 鍵の中身は、必要な値だけ取り出して持ち出す

- `bump` は、足した後の数 `*count`（`u64` の値）を返しています。ガードそのものではなく、**中の値の写し**を返すので、`handle_request` に戻ったときには鍵はもう返っています。
- 鍵の外で使いたいものは、こうして値として取り出してから鍵を返します。中身が `String` や `Vec` なら、必要な分だけ `clone` して取り出します（第3巻 UNIT 10 の「第5巻への橋渡し」で予告した形です）。
- `{ }` で囲む書き方（例5-1）と、関数に分ける書き方（この例）は、どちらも正解です。鍵を使う処理が数行なら `{ }`、何か所からも使うなら関数にするとよいでしょう。

例5-3: 鍵を持ったまま .await したいときは、tokio の Mutex

> **▼ このコードがやること（先に日本語で）**：「同時に1人しか使えない接続」を `tokio::sync::Mutex` で守り、3つのタスクで共有します。それぞれのタスクは鍵を取り、鍵を持ったまま、送るのに 0.1 秒かかるメッセージの送信を `.await` します。鍵を取った時刻と、送った順番、全体の時間を表示します。

```rust
use std::sync::Arc;
use std::time::{Duration, Instant};
use tokio::sync::Mutex;

// 「1本しかない接続」の代わり。同時に1人しか使えない
struct Connection {
    sent: Vec<String>,
}

impl Connection {
    // 送るのに 100ms かかる（.await を含む）
    async fn send(&mut self, message: String) {
        tokio::time::sleep(Duration::from_millis(100)).await;
        self.sent.push(message);
    }
}

#[tokio::main]
async fn main() {
    let start = Instant::now();
    let connection = Arc::new(Mutex::new(Connection { sent: Vec::new() }));
    let mut handles = Vec::new();
    for n in 1..=3 {
        let connection = Arc::clone(&connection);
        handles.push(tokio::spawn(async move {
            // tokio の Mutex は lock も .await で待つ。待っている間、スレッドは他の仕事をする
            let mut conn = connection.lock().await;
            println!(
                "{:.1} 秒: {n} 番が鍵を取った",
                start.elapsed().as_secs_f64()
            );
            // ガードを持ったまま .await してもよい（そのための Mutex）
            conn.send(format!("メッセージ{n}")).await;
        }));
    }
    for handle in handles {
        handle.await.unwrap();
    }
    println!("送った順: {:?}", connection.lock().await.sent);
    println!("合計 {:.1} 秒", start.elapsed().as_secs_f64());
}
```

`src/bin/u05_tokio_mutex.rs` として保存して実行します。

```
cargo run -q --bin u05_tokio_mutex
```

▼ 実行結果

```
0.0 秒: 1 番が鍵を取った
0.1 秒: 2 番が鍵を取った
0.2 秒: 3 番が鍵を取った
送った順: ["メッセージ1", "メッセージ2", "メッセージ3"]
合計 0.3 秒
```

▼ コードを1つずつ分解して解説

解説1: tokio の Mutex は、lock も .await で待つ

- `tokio::sync::Mutex` の `lock()` は async 関数で、`connection.lock().await` のように `.await` を付けます。鍵が空いていなければ、`Pending` を返して席を譲り、鍵が空いたら起こしてもらえます。待っている間も、スレッドはほかのタスクを進められます。
- 標準の Mutex と違って、戻り値は `Result` ではなく、ガードそのものです。tokio の Mutex にはポイズニングが無いので、`unwrap` が要りません。

解説2: ガードを持ったまま .await してよいのが、tokio の Mutex

- `conn.send(...).await` は、鍵（ガード `conn`）を持ったまま 0.1 秒待っています。tokio の Mutex のガードは、`.await` をまたいでも `Send` なので、`tokio::spawn` に渡してもコンパイルエラーになりません。
- 「接続を使い始めてから送り終わるまで、ほかの人に割り込まれたくない」のように、**`.await` をはさむ一続きの仕事を、まとめて1人でやりたい**ときに使います。

解説3: そのかわり、全体は1人ずつの速さになる

- 3つのタスクは、鍵を 0.1 秒ずつ順番に持ったので、全体で 0.3 秒かかりました。UNIT 02 のように待ちが重なることはありません。鍵を持ったまま待つと、その間はだれも入れないからです。
- tokio の Mutex は、標準の Mutex より重い道具です。tokio のドキュメントも「使える場面では、標準の Mutex を選ぶべき」としています。鍵を持ったまま `.await` する必要が本当にあるときだけ使いましょう。共有の接続のようなものは、UNIT 04 のアクター（係のタスクに依頼する形）で作るほうがすっきりすることも多いです。

例5-4: 読む人は同時に、書く人は1人で（RwLock）

> **▼ このコードがやること（先に日本語で）**：画面の設定（テーマ）を `tokio::sync::RwLock` で守ります。読む人のタスクを3つ作り、それぞれ読み込みの鍵を取って 0.2 秒持ちます。少し遅れて書く人のタスクを作り、書き込みの鍵でテーマを書き換えます。それぞれが鍵を取れた時刻を表示します。

```rust
use std::sync::Arc;
use std::time::{Duration, Instant};
use tokio::sync::RwLock;

#[tokio::main]
async fn main() {
    let start = Instant::now();
    // 読む人は何人でも同時に、書く人は1人だけ
    let settings = Arc::new(RwLock::new(String::from("テーマ: ライト")));
    let mut handles = Vec::new();

    for n in 1..=3 {
        let settings = Arc::clone(&settings);
        handles.push(tokio::spawn(async move {
            let value = settings.read().await;
            println!(
                "{:.1} 秒: 読む人{n}: {}",
                start.elapsed().as_secs_f64(),
                *value
            );
            tokio::time::sleep(Duration::from_millis(200)).await;
        }));
    }
    // 読む人たちが先に鍵を取れるよう、少しだけ遅れて書く人を始める
    tokio::time::sleep(Duration::from_millis(10)).await;
    let writer = {
        let settings = Arc::clone(&settings);
        tokio::spawn(async move {
            let mut value = settings.write().await;
            *value = String::from("テーマ: ダーク");
            println!(
                "{:.1} 秒: 書く人: 書き換えた",
                start.elapsed().as_secs_f64()
            );
        })
    };
    for handle in handles {
        handle.await.unwrap();
    }
    writer.await.unwrap();
    println!("最後の値: {}", settings.read().await);
}
```

`src/bin/u05_rwlock.rs` として保存して実行します。

```
cargo run -q --bin u05_rwlock
```

▼ 実行結果

```
0.0 秒: 読む人1: テーマ: ライト
0.0 秒: 読む人3: テーマ: ライト
0.0 秒: 読む人2: テーマ: ライト
0.2 秒: 書く人: 書き換えた
最後の値: テーマ: ダーク
```

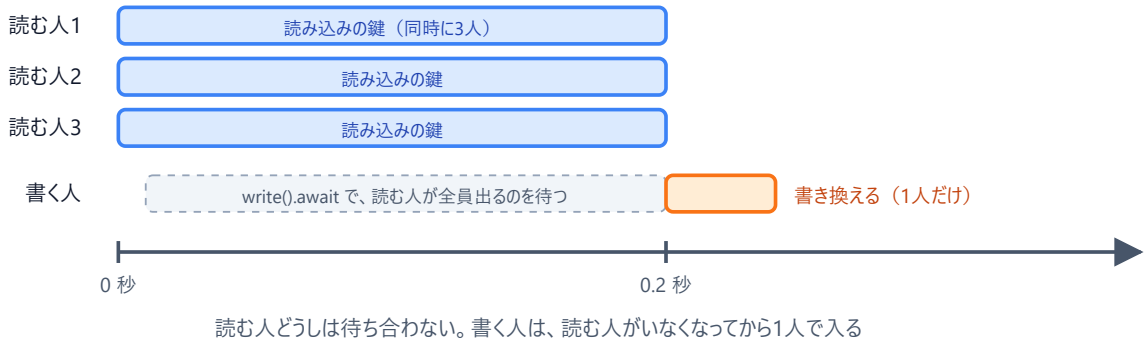


図5-18: RwLock の動き。3人の読む人は同時に鍵を取れる。書く人は、読む人が全員鍵を返す 0.2 秒まで待ってから、1人で書き換える

▼ コードを1つずつ分解して解説

解説1: read() は何人でも、write() は1人だけ

- `settings.read().await` は**読み込みの鍵**です。ほかに読んでいる人がいても、同時に取れます。3人の読む人は、全員 0 秒で鍵を取れました。
- `settings.write().await` は**書き込みの鍵**です。読んでいる人も書いている人も1人もいないときにだけ取れます。書く人は 0.01 秒に来ましたが、読む人が全員 0.2 秒まで鍵を持っていたので、0.2 秒まで待たされました。
- 読み込みの鍵（`value`）を通しては読むことしかできず、書き込みの鍵（`mut value`）を通すと書き換えられます。`*value = ...` で中身を入れ替えています。

解説2: 書く人が待っている間は、新しく来た読む人も待たされる

- tokio の RwLock は「書く人を優先する」作りです（tokio 1.53.3 のドキュメントに「公平（書き込み優先）」とあります）。書く人が待ち始めた後に来た読む人は、書く人が終わるまで待たされます。
- そうしないと、読む人が次々に来る間、書く人が永遠に入れない（飢える）ことがあるからです。

解説3: Mutex とどちらを使うか

- 読む回数がとても多く、書く回数がたまにしかないデータ（設定・一覧の写しなど）なら、RwLock にすると、読む人どうしが待ち合わずに済みます。
- 読むのも書くのも同じくらいなら、仕組みが簡単な Mutex で十分です。標準ライブラリにも `std::sync::RwLock` があり、`.await` をまたがないなら、標準のものを使えます（Mutex と同じ考え方です）。

例5-5: 同時に入れる数を、Semaphore で2つまでにする

> **▼ このコードがやること（先に日本語で）**：許可証が2枚の `Semaphore` を作り、5つのタスクで共有します。それぞれのタスクは許可証を1枚もらってから 0.3 秒の仕事をし、終わったら許可証を返します。開始と終了の時刻を表示して、同時に動いているのが2つまでになっていることを確かめます。

```rust
use std::sync::Arc;
use std::time::{Duration, Instant};
use tokio::sync::Semaphore;

#[tokio::main]
async fn main() {
    let start = Instant::now();
    // 許可証（permit）は 2 枚。同時に動けるのは 2 つまで
    let permits = Arc::new(Semaphore::new(2));
    let mut handles = Vec::new();

    for n in 1..=5 {
        let permits = Arc::clone(&permits);
        handles.push(tokio::spawn(async move {
            // 許可証が空くまで待つ。手に入れた許可証は _permit が片付けられるときに返る
            let _permit = permits.acquire_owned().await.unwrap();
            println!("{:.1} 秒: {n} 番 開始", start.elapsed().as_secs_f64());
            tokio::time::sleep(Duration::from_millis(300)).await;
            println!("{:.1} 秒: {n} 番 終了", start.elapsed().as_secs_f64());
        }));
    }
    for handle in handles {
        handle.await.unwrap();
    }
}
```

`src/bin/u05_semaphore.rs` として保存して実行します。

```
cargo run -q --bin u05_semaphore
```

▼ 実行結果

```
0.0 秒: 1 番 開始
0.0 秒: 2 番 開始
0.3 秒: 1 番 終了
0.3 秒: 3 番 開始
0.3 秒: 2 番 終了
0.3 秒: 5 番 開始
0.6 秒: 3 番 終了
0.6 秒: 4 番 開始
0.6 秒: 5 番 終了
0.9 秒: 4 番 終了
```

▼ コードを1つずつ分解して解説

解説1: acquire_owned で許可証をもらい、片付けで返す

- `Semaphore::new(2)` は、許可証が2枚の Semaphore を作ります。`Arc` で包んで、全部のタスクで共有しています。
- `permits.acquire_owned().await` は、許可証が残っていればすぐに1枚もらい、無ければ、だれかが返すまで待ちます。戻り値の `_permit` が許可証で、**`_permit` が片付けられるときに自動で返されます**（ガードと同じ形です）。返し忘れが起きません。
- `_owned` が付いた版は、`Arc<Semaphore>` から、借り物ではない許可証を作ります。タスクの中で使うときは、この版が扱いやすいです。`unwrap` は、Semaphore が閉じられたとき（`close()`）にだけ `Err` になる部分です。

解説2: 同時に動いているのは、いつも2つまで

- 実行結果を時刻ごとに見ると、0 秒に1・2番、0.3 秒に3・5番、0.6 秒に4番が始まっています。どの時点でも、「開始した」のに「終了していない」ものは2つまでです。5つ全部が終わるまで、約 0.9 秒でした。
- 5番が4番より先に始まっているのは、5番のタスクのほうが先に `acquire_owned` を呼んで、列に並んだからです。tokio の Semaphore は公平で、**許可証を求めた順番**に配ります（tokio 1.53.3 のドキュメントより）。どのタスクが先に列に並ぶかは、実行するたびに変わります。

解説3: UNIT 03 の JoinSet の上限との違い

- UNIT 03 の例3-4では、親が「袋の中の数」を数えて、上限を守っていました。これは、親が1人で全部のタスクを作るときに向いています。
- Semaphore は、**あちこちのタスクが、それぞれ自分で「入ってよいか」を確かめる**ときに向いています。たとえば、たくさんのリクエストの処理が、それぞれ外の API を呼ぶとき、「外の API を同時に呼ぶのは全体で10まで」を、Semaphore を1つ共有するだけで守れます。

最後に、この単元の道具の選び方を図にまとめます。

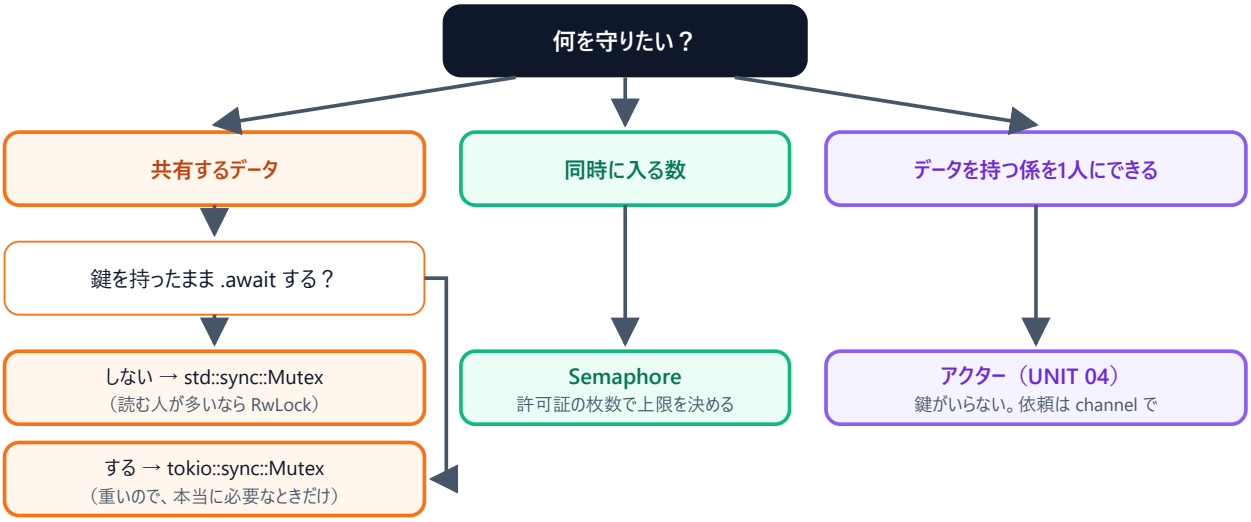


図5-19: 守りたいものによる道具の選び方。データは標準の Mutex が基本で、鍵を持ったまま待つときだけ tokio の Mutex。数の上限は Semaphore、係を1人にできるならアクター

よくあるエラーと読み方

エラー1: 標準の Mutex の鍵を持ったまま .await して spawn する（future cannot be sent between threads safely）

例5-1の `{ }` を外して、鍵を持ったまま `.await` した場合です。

```
use std::collections::HashMap;
use std::sync::{Arc, Mutex};
use std::time::Duration;

type Counter = Arc<Mutex<HashMap<String, u64>>>;

async fn handle_request(counter: Counter, path: &str) {
    let mut map = counter.lock().unwrap();
    *map.entry(path.to_string()).or_insert(0) += 1;
    // 鍵（ガード map）を持ったまま .await してしまう
    tokio::time::sleep(Duration::from_millis(10)).await;
    println!("{path}: {} 回目", map[path]);
}

#[tokio::main]
async fn main() {
    let counter: Counter = Arc::new(Mutex::new(HashMap::new()));
    let handle = tokio::spawn(handle_request(Arc::clone(&counter), "/notes"));
    handle.await.unwrap();
}
```

```
   Compiling async_lab v0.1.0 (...\async_lab)
error: future cannot be sent between threads safely
   --> src\bin\e05_guard_await.rs:18:31
    |
 18 |     let handle = tokio::spawn(handle_request(Arc::clone(&counter), "/notes"));
    |                               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
future returned by `handle_request` is not `Send`
    |
    = help: within `impl Future<Output = ()>`, the trait `Send` is not implemented for
`std::sync::MutexGuard<'_, HashMap<String, u64>>`
note: future is not `Send` as this value is used across an await
   --> src\bin\e05_guard_await.rs:11:51
    |
  8 |     let mut map = counter.lock().unwrap();
    |         ------- has type `std::sync::MutexGuard<'_, HashMap<String, u64>>` which
is not `Send`
...
 11 |     tokio::time::sleep(Duration::from_millis(10)).await;
    |                                                   ^^^^^ await occurs here, with
`mut map` maybe used later
note: required by a bound in `tokio::spawn`
   --> ...\tokio-1.53.3\src\task\spawn.rs:176:21
    |
174 |     pub fn spawn<F>(future: F) -> JoinHandle<F::Output>
    |            ----- required by a bound in this function
175 |     where
176 |         F: Future + Send + 'static,
    |                     ^^^^ required by this bound in `spawn`

error: could not compile `async_lab` (bin "e05_guard_await") due to 1 previous error
```

- **何が起きたか:** UNIT 02 の「よくあるエラー2」（`Rc` を持ったまま `.await`）と、まったく同じ形のエラーです。今度の原因の型は `std::sync::MutexGuard`（標準の Mutex のガード）です。標準の Mutex のガードは `Send` ではありません。鍵は「取ったスレッドで返す」約束になっているので、`.await` の後で別のスレッドに移ってから返されては困るからです。
- `note: future is not Send as this value is used across an await` の下で、8行目で作ったガード `map` が、11行目の `.await` の後（12行目）でも使われていることが示されています。
- **直し方:** 例5-1のように `{ }` で囲んで、ガードを `.await` の前に片付けます。`.await` の後で使いたい値（ここでは何回目か）は、`{ }` の中で取り出しておきます。

エラー2: drop(guard) を書いても、エラーが消えない

「ガードを `.await` の前に片付ければよい」と聞いて、`drop` を足した場合です。

```rust
use std::sync::{Arc, Mutex};
use std::time::Duration;

async fn handle_request(counter: Arc<Mutex<u64>>) {
    let mut count = counter.lock().unwrap();
    *count += 1;
    let now = *count;
    // .await の前に、ガードを明示的に片付ける
    drop(count);
    tokio::time::sleep(Duration::from_millis(10)).await;
    println!("{now} 回目");
}

#[tokio::main]
async fn main() {
    let counter = Arc::new(Mutex::new(0));
    tokio::spawn(handle_request(Arc::clone(&counter))).await.unwrap();
}
```

```
   Compiling async_lab v0.1.0 (...\async_lab)
error: future cannot be sent between threads safely
   --> src\bin\e05_drop_guard.rs:17:18
    |
 17 |     tokio::spawn(handle_request(Arc::clone(&counter))).await.unwrap();
    |                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ future returned by
`handle_request` is not `Send`
    |
    = help: within `impl Future<Output = ()>`, the trait `Send` is not implemented for
`std::sync::MutexGuard<'_, u64>`
note: future is not `Send` as this value is used across an await
   --> src\bin\e05_drop_guard.rs:10:51
    |
  5 |     let mut count = counter.lock().unwrap();
    |         --------- has type `std::sync::MutexGuard<'_, u64>` which is not `Send`
...
 10 |     tokio::time::sleep(Duration::from_millis(10)).await;
    |                                                   ^^^^^ await occurs here, with
`mut count` maybe used later
note: required by a bound in `tokio::spawn`
   --> ...\tokio-1.53.3\src\task\spawn.rs:176:21
    |
174 |     pub fn spawn<F>(future: F) -> JoinHandle<F::Output>
    |            ----- required by a bound in this function
175 |     where
176 |         F: Future + Send + 'static,
    |                     ^^^^ required by this bound in `spawn`

error: could not compile `async_lab` (bin "e05_drop_guard") due to 1 previous error
```

- **何が起きたか:** 9行目で `drop(count)` しているのに、「`count` が `.await` の後で使われるかもしれない（`maybe used later`）」と言われています。Rust 1.99.0 でも、このとおりのエラーになりました。
- コンパイラは、Future の中にしまう変数を決めるとき、`drop` を呼んだかどうかまでは見ずに、**変数の有効範囲（`{ }` のどこで作られ、どこで終わるか）**で判断しています。`count` は関数の最後まで有効な変数なので、`.await` をまたいでいる、と扱われるのです。
- **直し方:** `drop` ではなく、変数の有効範囲そのものを `.await` の前で終わらせます。`{ }` のブロックで囲むか（例5-1）、async でない関数に分けます（例5-2）。実際に直すのは、練習問題1で自分の手でやってみましょう。

エラー3: spawn しないと、コンパイルが通ってしまう（clippy の await_holding_lock）

同じ「鍵を持ったまま `.await`」でも、`tokio::spawn` に渡さずに直接 `.await` すると、`Send` が求められないので、コンパイルが通ってしまいます。

```rust
use std::sync::{Arc, Mutex};
use std::time::Duration;

async fn handle_request(counter: Arc<Mutex<u64>>) {
    let mut count = counter.lock().unwrap();
    *count += 1;
    // 鍵を持ったまま .await する
    tokio::time::sleep(Duration::from_millis(10)).await;
    println!("{} 回目", *count);
}

#[tokio::main]
async fn main() {
    let counter = Arc::new(Mutex::new(0));
    // spawn せずに、main の中で直接 .await する
    handle_request(Arc::clone(&counter)).await;
}
```

実行すると、何事もなかったように動きます。

```
1 回目
```

この見落としを見つけてくれるのが clippy です。`cargo clippy --bin e05_guard_no_spawn` の結果です。

```
    Checking async_lab v0.1.0 (...\async_lab)
warning: this `MutexGuard` is held across an await point
 --> src\bin\e05_guard_no_spawn.rs:5:9
  |
5 |     let mut count = counter.lock().unwrap();
  |         ^^^^^^^^^
  |
  = help: consider using an async-aware `Mutex` type or ensuring the `MutexGuard` is
dropped before calling `await`
note: these are all the await points this lock is held through
 --> src\bin\e05_guard_no_spawn.rs:8:51
  |
8 |     tokio::time::sleep(Duration::from_millis(10)).await;
  |                                                   ^^^^^
  = help: for further information visit https://rust-lang.github.io/rust-clippy/rust-
1.99.0/index.html#await_holding_lock
  = note: `#[warn(clippy::await_holding_lock)]` on by default

warning: `async_lab` (bin "e05_guard_no_spawn") generated 1 warning
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.55s
```

- **何が起きたか**: clippy が「この `MutexGuard` は、`.await` の地点をまたいで持たれています」と警告しています。`help:` は「async 用の Mutex を使うか、`.await` の前にガードを片付けてください」、`note:` は、鍵を持ったままま たいでいる `.await` の場所（8行目）です。
- このプログラムが無事に動いたのは、鍵を使うタスクが1つしかなかったからです。同じ鍵を使うタスクが2つ以上あると、次のエラー4のように、本当に止まることがあります。
- **直し方**: エラー1と同じく、ガードを `.await` の前に片付けます。「spawn していないからエラーにならない」は、安全という意味ではありません。`cargo clippy -- -D warnings`（UNIT 01）で、警告も残さないようにしましょう。

エラー4（エラーが出ない間違い）: 本当に止まる（デッドロック）

スレッドが1本だけのランタイム（`flavor = "current_thread"`、UNIT 02 のヒント）で、タスク A と B が同じ標準の Mutex を使い、どちらも鍵を持ったまま `.await` します。`tokio::join!` で同時に進めるので、`Send` は求められず、コンパイルは通ります（clippy は、エラー3と同じ `await_holding_lock` の警告を出します）。

```
use std::sync::Mutex;
use std::time::Duration;

async fn task(name: &str, counter: &Mutex<u64>) {
    println!("{name}: 鍵を取りに行きます");
    let mut count = counter.lock().unwrap();
    println!("{name}: 鍵を取りました");
    *count += 1;
    // 標準の Mutex の鍵を持ったまま .await して、スレッドを手放してしまう
    tokio::time::sleep(Duration::from_millis(100)).await;
    println!("{name}: 終わりました（{}）", *count);
}

// スレッドが1本だけのランタイムで動かす
#[tokio::main(flavor = "current_thread")]
async fn main() {
    let counter = Mutex::new(0);
    // spawn せずに join! で2つを同時に進めるので、Send は求められずコンパイルが通る
    tokio::join!(task("A", &counter), task("B", &counter));
    println!("全部終わりました");
}
```

このプログラムは終わらないので、Git Bash の `timeout 5`（5 秒たったら止める）を付けて実行しました。

```
A: 鍵を取りに行きます
A: 鍵を取りました
B: 鍵を取りに行きます
```

5 秒たっても続きが表示されず、`timeout` に止められました（終了コード 124 は「時間切れで止めた」という意味です）。

- **何が起きたか**: 図の上の段のとおりです。A が鍵を取り、`.await` で席を譲りました。同じスレッドで B が動き出し、`counter.lock()` で鍵を待ちます。標準の Mutex の `lock` は**スレッドごと止めて**待つので、A を進めるスレッドがもうありません。A は鍵を返せず、B は鍵を待ち続けます。エラーも panic も出ずに、ただ止まりました。
- スレッドが何本もあるランタイムなら、運よく別のスレッドで A が進んで動くこともあります。しかし、同時に来るリクエストが増えると、全部のスレッドが同じように止まり、サーバー全体が応答しなくなります。「たまにしか起きない」ぶん、見つけるのがとても難しい間違いです。
- **直し方**: いちばんよいのは、エラー1と同じく、ガードを `.await` の前に片付けることです。どうしても鍵を持ったまま待つ必要があるなら、tokio の Mutex に変えます。`use tokio::sync::Mutex;` にして、`counter.lock().unwrap()` を `counter.lock().await` に変えたものが次のコードです。

```rust
use std::time::Duration;
use tokio::sync::Mutex;

async fn task(name: &str, counter: &Mutex<u64>) {
    println!("{name}: 鍵を取りに行きます");
    // tokio の Mutex は、鍵が空くのを .await で待つ。待つ間、スレッドはほかの仕事をする
    let mut count = counter.lock().await;
    println!("{name}: 鍵を取りました");
    *count += 1;
    tokio::time::sleep(Duration::from_millis(100)).await;
    println!("{name}: 終わりました（{}）", *count);
}

// スレッドが1本だけのランタイムで動かす
#[tokio::main(flavor = "current_thread")]
async fn main() {
    let counter = Mutex::new(0);
    tokio::join!(task("A", &counter), task("B", &counter));
    println!("全部終わりました");
}
```

`src/bin/u05_fix_deadlock.rs` として実行した結果です。

```
A: 鍵を取りに行きます
A: 鍵を取りました
B: 鍵を取りに行きます
A: 終わりました（1）
B: 鍵を取りました
B: 終わりました（2）
全部終わりました
```

B は鍵を待つ間も席を譲ったので、A が戻ってきて鍵を返し、その後で B が鍵を取れました（図の下の段）。

エラー5（エラーが出ない間違い）：同じタスクの中で、もう一度 lock する

tokio の Mutex でも、**自分が持っている鍵を、自分でもう一度取ろうとする**と、永遠に待ちます。

```
use std::sync::Arc;
use std::time::Duration;
use tokio::sync::Mutex;

#[tokio::main]
async fn main() {
    let notes = Arc::new(Mutex::new(vec![String::from("ノート1")]));

    let first = notes.lock().await;
    println!("1回目の lock: 取れた（{} 件）", first.len());

    // first を返さないまま、同じタスクでもう一度 lock する
    let second = tokio::time::timeout(Duration::from_secs(1), notes.lock()).await;
    match second {
        Ok(_) => println!("2回目の lock: 取れた"),
        Err(_) => println!("2回目の lock: 1 秒待っても取れない（自分が持っている鍵を待っている）"),
    }
}
```

```
1回目の lock: 取れた（1 件）
2回目の lock: 1 秒待っても取れない（自分が持っている鍵を待っている）
```

（`cargo run` の `Compiling ...` などの行は省いています。）

- `tokio::time::timeout(時間, Future)` は、「この時間までに終わらなければ `Err` を返す」関数です。UNIT 03 の練習問題2で `select!` を使って作ったものと同じで、UNIT 06 でくわしく学びます。ここでは、永遠に待たないように1秒で区切っています。
- **何が起きたか:** `first` が鍵を持ったままなので、2回目の `lock` は、`first` が鍵を返すのを待ちます。ところが `first` を持っているのは自分自身なので、返されることはありません。
- **直し方:** 1つのタスクの中では、鍵は1回だけ取り、取ったガードを使い回します。関数 A が鍵を取ったまま、同じ鍵を取る関数 B を呼んでいないか、を確かめましょう。鍵を取る場所を1か所（例5-2のような小さな関数）にまとめておくと、この間違いは起きにくくなります。

RustNote ではここで使う

RustNote のコードには、`Mutex` も `RwLock` も `Semaphore` も出てきません。これは偶然ではなく、設計で避けています。

- **変わるデータは、全部 DB に置いています。**ノートもユーザーもログイン中のセッションも、`notes`・`users`・`sessions` の表にあります。メモリの中で共有して書き換えるデータが無いので、鍵が要りません。同時に書き込んだときの食い違いは、DB の transaction と `version` 列が防ぎます（第7巻 UNIT 05）。
- **共有している `AppState` は、作った後に書き換えません。**UNIT 02 で見たとおり、`AppState` は `Arc<dyn NoteRepository + Send + Sync>` などを持っているだけです。だれも書き換えないものは、`Arc` で配るだけで、鍵なしに何人でも同時に読めます。
- このやり方なら、Fly.io で同じアプリを2台以上動かしても、データが台ごとに分かれることがありません（例5-1の注意）。**鍵をかけたくなったら、まず「そのデータは DB に置くべきではないか？」と考える**のが、Web アプリでの基本です。

唯一、標準ライブラリの「1回だけ作る」道具が使われています。`src/auth.rs` の、ログインに使うダミーのハッシュです。

```
/// 登録されていないメールでも同じくらい時間をかけるための、ダミーのハッシュ。
/// （すぐ返すと「このメールは未登録だ」と応答時間から推測されてしまう）
static DUMMY_HASH: LazyLock<String> = LazyLock::new(|| {
    Argon2::default()
        .hash_password(b"rustnote-dummy-password")
        .expect("ダミーのハッシュを作れません")
        .to_string()
});
```

- `std::sync::LazyLock` は、**最初に使われたときに1回だけ**中のクロージャを実行して値を作り、それ以降は作った値を何人にでも読ませる道具です。2つのリクエストが同時に初めて使っても、作られるのは1回だけです。中では鍵のようなしくみで「1回だけ」を守っていますが、作り終わった後は、読むだけなので待ち合いません。
- 一方で、RustNote には「パスワードのハッシュ（Argon2）を同時に何個まで計算するか」の上限がありません（改善の候補として引き継ぎに記録しています）。Argon2 はわざと重く作られた計算なので、ログインが一度に大量に来ると、CPU を使い切ってしまいます。この上限こそ、例5-5の Semaphore の出番です。重い計算をどこで動かすかと合わせて、UNIT 07 で扱います。

練習問題

問1: 「よくあるエラー2」の `e05_drop_guard.rs` を、`drop` を使わずに直して、`tokio::spawn` に渡せるようにしてください。表示は「1 回目」になります（`src/bin/u05_q1.rs`）。

問2: 同時に使える DB の接続が3本しかないとします。300 ミリ秒かかる問い合わせを6つのタスクから行い、Semaphore で同時に3つまでにしてください。それぞれの問い合わせを始めた時刻と、全体の時間を表示します。全体で何秒になるか予想してから実行しましょう（`src/bin/u05_q2.rs`）。

問3: 次の4つの場面で、この単元の道具（標準の Mutex・tokio の Mutex・RwLock・Semaphore）とアクター（UNIT 04）、DB（第7巻）のどれを使うのがよいか、理由と一緒に答えてください。

1. 1台のサーバーの中だけで、「今、何件のリクエストを処理中か」を数えて、管理画面に表示したい。数を足したり引いたりするだけで、その間に `.await` はしない。

2. 外の地図 API は「同時に5件まで」と決められている。たくさんのリクエストの処理が、それぞれ地図 API を呼ぶ。
3. 起動時に読み込んだ「禁止ワードの一覧」を、全部のリクエストが毎回読む。管理者が、1日に1回くらい一覧を入れ替える。
4. ユーザーごとの「今月の投稿数」を数えて、上限を超えたら投稿を断りたい。サーバーは2台で動いている。

解答と解説

問1の解答:

```
use std::sync::{Arc, Mutex};
use std::time::Duration;

async fn handle_request(counter: Arc<Mutex<u64>>) {
    // 鍵を使う部分を { } で囲み、ガードが .await より前に片付くことを形で示す
    let now = {
        let mut count = counter.lock().unwrap();
        *count += 1;
        *count
    };
    tokio::time::sleep(Duration::from_millis(10)).await;
    println!("{now} 回目");
}

#[tokio::main]
async fn main() {
    let counter = Arc::new(Mutex::new(0));
    tokio::spawn(handle_request(Arc::clone(&counter)))
        .await
        .unwrap();
}
```

```
1 回目
```

- `{ }` のブロックの中でガード `count` を作り、ブロックの最後の式 `*count` で、足した後の数をブロックの値として外へ出しています（第2巻 UNIT 03 の「ブロックも式」）。`count` の有効範囲は `}` で終わるので、`.await` をまたぎません。コンパイラにも、それが形で伝わります。
- 例5-2のように、`fn bump(counter: &Mutex<u64>) -> u64` という async でない関数に分けても正解です。

問2の解答:

```rust
use std::sync::Arc;
use std::time::{Duration, Instant};
use tokio::sync::Semaphore;

// DB への問い合わせの代わり。300ms かかる
async fn query(n: u32, start: Instant) {
    println!(
        "{:.1} 秒: {n} 番 問い合わせ開始",
        start.elapsed().as_secs_f64()
    );
    tokio::time::sleep(Duration::from_millis(300)).await;
}

#[tokio::main]
async fn main() {
    let start = Instant::now();
    // 同時に使える DB の接続は 3 本まで
    let connections = Arc::new(Semaphore::new(3));
    let mut handles = Vec::new();
    for n in 1..=6 {
        let connections = Arc::clone(&connections);
        handles.push(tokio::spawn(async move {
            let _permit = connections.acquire_owned().await.unwrap();
            query(n, start).await;
        }));
    }
    for handle in handles {
        handle.await.unwrap();
    }
    println!(
        "全部終わりました（{:.1} 秒）",
        start.elapsed().as_secs_f64()
    );
}
```

```
0.0 秒: 1 番 問い合わせ開始
0.0 秒: 2 番 問い合わせ開始
0.0 秒: 3 番 問い合わせ開始
0.3 秒: 4 番 問い合わせ開始
0.3 秒: 5 番 問い合わせ開始
0.3 秒: 6 番 問い合わせ開始
全部終わりました（0.6 秒）
```

- 許可証を3枚にしたので、最初の3つが 0 秒に、残りの3つが 0.3 秒に始まり、全体で 0.6 秒でした。番号の順番は、実行するたびに入れ替わることがあります（例5-5の解説2）。

- `_permit` は `query(n, start).await` が終わるまで持たれ、タスクの最後で片付けられて許可証が返ります。もし `let _ = connections.acquire_owned()...` と `_` 1文字にすると、その場ですぐに許可証が返ってしまい、上限が効かなくなります（UNIT 03 の例3-2の解説3と同じ注意です）。

問3の解答:

1. **標準の Mutex**（または、数を数えるだけなら標準の atomic 型 `AtomicU64`）。1台の中だけの、`.await` をはさまない短い書き換えなので、軽い標準の Mutex で十分です。再起動で0に戻ってもかまわない「今の様子」の数なので、メモリに置いてよい例です。
2. **Semaphore**。許可証を5枚にした Semaphore を1つ作り、`Arc` で全部のリクエストの処理に配ります。地図 API を呼ぶ前に許可証をもらい、終わったら返します（例5-5）。
3. **RwLock**。全部のリクエストが毎回読み、書き換えは1日に1回なので、読む人どうしが待ち合わない RwLock が向いています。読むときに `.await` をはさまないなら、標準の `std::sync::RwLock` で十分です。
4. **DB**。サーバーが2台あるので、メモリの中で数えると台ごとに別々の数になり、上限を正しく守れません。数は DB の表に置き、「数えて、上限を確かめて、足す」を transaction の中で行います（第7巻 UNIT 05）。メモリの鍵では、台をまたいだ食い違いは防げません。

> **ヒント:** 問3の1で出てきた atomic 型（`std::sync::atomic::AtomicU64` など）は、数を足す・引く・読むだけなら、鍵を使わずに、割り込まれない1回の操作で書き換えられる型です。第3巻 UNIT 10 の表に出てきた `Arc` の数え方も、この仕組みを使っています。

まとめ

- async の中でも、データを守るのは標準の `std::sync::Mutex` が基本。決まりは「鍵を持ったまま `.await` しない」で、`{ }` のブロックか async でない関数で鍵の範囲を区切る。`drop(guard)` では、コンパイラに伝わらない。
- spawn すると「future cannot be sent」のエラーで止めてもらえるが、spawn しないと通ってしまい、スレッドが止まってデッドロックすることがある。clippy の `await_holding_lock` で見つける。鍵を持ったまま待つ必要があるときだけ tokio の Mutex を使う。
- 読む人が多いデータは RwLock、同時に入れる数の上限は Semaphore（許可証は片付けで返る）。消えてはいけないデータや台をまたぐデータは、鍵ではなく DB で守る。

この巻のまとめ

この巻では、たくさんの仕事を同時にさばくサーバーの土台として、async の仕組みと Tokio のタスクから学び始めました。次のチェックリストで、できるようになったことを確かめてください。自信がない項目は、括弧の中の単元に戻って、コードを動かし直しましょう。表示の順番を予想してから動かすのを忘れずに。

async の土台

- ☐ `async fn` を呼ぶと Future（仕事の札）が返るだけで、中身は `.await` したときに初めて動くことを、実行結果で説明できる（UNIT 01）

- ☐ `tokio::time::sleep(...).await` と `std::thread::sleep` の違いを説明し、async の中では前者を使える（UNIT 01）
- ☐ お互いに関係のない待ちを `tokio::join!` で重ね、全体の時間が「足し算」から「いちばん長いもの」になることを示せる。並行と並列の違いを説明できる（UNIT 01）
- ☐ `tokio::try_join!` が最初に起きた失敗をすぐに返し、残りの Future を途中で捨てる（キャンセルする）ことを説明できる（UNIT 01）
- ☐ Future が poll されて `Pending` / `Ready` を返し、Waker で起こされて進む流れを図で説明できる。`.await` をまたぐ変数が Future の中にしまわれることを知っている（UNIT 01）
- ☐ E0752（`async fn main`）・E0728（`async` でない関数の `.await`）・E0308（`.await` の付け忘れ）と、`unused implementer of Future` の警告を読んで直せる。警告を `cargo clippy -- -D warnings` で止められる（UNIT 01）
- ☐ ランタイムの中身（ワーカースレッド・スケジューラー・タイマーと I/O の係）と、`#[tokio::main]` が `Builder` と `block_on` の形に書き換わることを説明できる（UNIT 02）
- ☐ `tokio::spawn` でタスクを作り、引換券（`JoinHandle`）の `.await` で `Result<戻り値, JoinError>` を受け取れる。タスクは spawn した瞬間から動くことを知っている（UNIT 02）
- ☐ `JoinError` の `is_panic()` と `is_cancelled()` で、panic と `abort` を見分けられる。`Result<Result<T, E>, JoinError>` を3つに分けて扱える（UNIT 02）
- ☐ 引換券を捨てたタスクが `main` の終わりで途中で捨てられることを知っていて、最後までやり遂げてほしい仕事は待つようにできる（UNIT 02）
- ☐ `tokio::spawn` に `Send + 'static` が要る理由を説明し、E0373・E0382（ループの中の `move`）・「future cannot be sent between threads safely」を、`move`・`clone`・`Arc` で直せる（UNIT 02）
- ☐ ランタイムの中で `block_on` を呼ぶと `Cannot start a runtime from within a runtime` で panic する理由を説明できる（UNIT 02）
- ☐ `JoinSet` で子のタスクをまとめて作り、`join_next` で終わった順に受け取れる。袋が捨てられると残った子が取り消されることを、`Drop` の表示で確かめられる（UNIT 03）
- ☐ 「1つ失敗したら `??` で抜けて残りを取り消す」集め方と、「失敗も `Result` のまま全部集める」集め方を書き分けられる（UNIT 03）
- ☐ 袋の中の数を見て、同時に動かすタスクの数に上限を付けられる。上限が「速さ」と「相手への負担」の兼ね合いだと説明できる（UNIT 03）
- ☐ `tokio::select!` で先に終わった枝だけを実行し、負けた Future がその場で捨てられることと、キャンセル安全の意味を説明できる（UNIT 03）
- ☐ ループの中の `select!` で、締め切りや仕事の Future をループの外で作って `pin!` で留め、`&mut` で借りられる。E0271（JoinSet の型の食い違い）・E0277（`PhantomPinned cannot be unpinned`）と、「毎回やり直しになって終わらない」間違いを直せる（UNIT 03）
- ☐ mpsc の channel で、`clone` した送る口から1つの受け取り口へ値を持ち主ごと渡せる。送る口が全部片付くと `recv` が `None` を返すことを使って、受け取りのループを終わらせられる（UNIT 04）

- ☐ 容量の決まった channel で受け手が遅いと送り手が待たされる（backpressure）ことを、時刻の表示で示せる。`try_send` の `Full` で混雑時にすぐ断れる。容量を増やしても全体が速くならない理由を説明できる（UNIT 04）
- ☐ 依頼に oneshot の封筒を同封して、データを1つのタスクだけが持つアクターを作れる（UNIT 04）
- ☐ broadcast（全員に配る・遅れると `Lagged`）と watch（最新の値だけ・止める合図）を目的に合わせて選べる。`borrow` を持ったまま `.await` しない（UNIT 04）
- ☐ E0382（送った値の再利用）・`SendError`（`channel closed`）・`RecvError::Lagged` と、`drop(tx)` の書き忘れで受け取り係が終わらない間違いを直せる（UNIT 04）
- ☐ async の中でも標準の `std::sync::Mutex` を使い、「鍵を持ったまま `.await` しない」を `{ }` のブロックか async でない関数で守れる（UNIT 05）
- ☐ `MutexGuard` を持ったまま `.await` したときの「future cannot be sent between threads safely」を読み、`drop(guard)` では直らない理由（変数の有効範囲で判断される）を説明できる（UNIT 05）
- ☐ spawn しないとコンパイルが通り、1本のスレッドではデッドロックすることを知っていて、clippy の `await_holding_lock` で見つけられる。同じタスクで2回 `lock` しない（UNIT 05）
- ☐ 鍵を持ったまま `.await` する必要があるときだけ `tokio::sync::Mutex` を選べる。RwLock で読む人を同時に通し、Semaphore の許可証で同時に入れる数を決められる（UNIT 05）
- ☐ 消えてはいけないデータや、複数のサーバーで共有するデータは、メモリの鍵ではなく DB で守る理由を説明できる（UNIT 05）

> **お知らせ:** この版には UNIT 01〜05 を収録しています。UNIT 06（timeout・キャンセル・graceful shutdown）・UNIT 07（spawn_blocking）・UNIT 08（HTTP クライアント・再試行）・UNIT 09（tracing）・UNIT 10（バックグラウンドジョブ）は準備中で、次の更新から順に追加します。それまでの間、重い計算を逃がす `spawn_blocking` が RustNote でどう使われているかは、`Rust/rustnote/src/auth.rs` で確かめられます。

次の第6巻では、この巻で学んだタスクの上で動く Web フレームワーク Topcoat を使って、HTTP のリクエストと答えのしくみを学びます。

用語ミニ辞典

この巻で出てきた主な用語です。

用語	読み・意味	出てくる単元
async（非同期処理）	エイシンク。待ち時間の間に、ほかの仕事を進められるように書いた処理	巻頭・UNIT 01
Tokio	トーキョー。Future を動かすランタイムを提供するクレート	巻頭・UNIT 02
Future	フューチャー。「まだ終わっていない仕事」を表す値。`.await` か poll されるまで動かない	UNIT 01
`.await`	Future が終わるまで待つ印。待つ間は、ほかの仕事に席を譲る	UNIT 01
ランタイム（executor）	Future を poll して動かす係。実行係	UNIT 01・02
poll	ポール。Future に「進められるところまで進めて」と頼むこと	UNIT 01
Pending / Ready	poll の答え。「まだです」と「できました（値つき）」	UNIT 01
Waker	ウェイカー。準備ができた Future を、もう一度 poll するよう知らせる起こし係	UNIT 01
状態機械	「今どこまで進んだか」を状態として持ち、少しずつ進むしくみ。async fn はこれに作り変えられる	UNIT 01
pin	ピン。Future をメモリの中で動かないように留めること	UNIT 01
並行（concurrency）	コンカレンシー。複数の仕事を切り替えながら少しずつ進めること	UNIT 01
並列（parallelism）	パラレリズム。複数の CPU で、本当に同時に動かすこと	UNIT 01・02
`tokio::join!`	複数の Future を同時に進め、全部の結果をタプルで返す	UNIT 01
`tokio::try_join!`	`Result` を返す Future 用の `join!`。最初の失敗をすぐに返す	UNIT 01
キャンセル	Future が途中で drop され、二度と poll されなくなること	UNIT 01・02
タスク	ランタイムの上で動く、1つの独立した仕事。`tokio::spawn` で作る	UNIT 02
ワーカースレッド	ランタイムがタスクを進めるために用意したスレッド	UNIT 02
スケジューラー	待ち行列のタスクを、ワーカースレッドに割り振る係	UNIT 02
`tokio::spawn`	Future をタスクとして動かし始め、引換券を返す関数	UNIT 02
JoinHandle	ジョインハンドル。タスクの引換券。`.await` で結果を受け取る	UNIT 02
JoinError	タスクが panic したか、取り消されたことを表すエラー	UNIT 02
abort	アボート。タスクを取り消すこと。次の `.await` の区切りで止まる	UNIT 02
切り離されたタスク	引換券を捨てて、誰も結果を待っていないタスク（detached task）	UNIT 02

用語	読み・意味	出てくる単元
`block_on`	Future が終わるまで、今のスレッドで待ち続ける関数。async の世界の入口で使う	UNIT 02
`Send`	別のスレッドへ渡してよい、という型の印（第3巻 UNIT 10）	UNIT 02
`'static`	借り物を持たず、いつまでも使える、という印。`tokio::spawn` に渡す仕事に求められる	UNIT 02
`async move`	外の変数を借りずに、async ブロックの中へ移す書き方	UNIT 02
coroutine	コルーチン。コンパイラの中での、async ブロックの呼び名。エラーメッセージに出てくる	UNIT 02
JoinSet	ジョインセット。子のタスクの袋。`join_next` で終わった順に受け取り、捨てると残りを取り消す	UNIT 03
`join_next` / `join_all`	JoinSet から結果を1つずつ／全部まとめて受け取るメソッド。どちらも終わった順	UNIT 03
構造化された並行処理	子のタスクが親より長生きしないように、親が子の始まりと終わりを管理する書き方	UNIT 03
`tokio::select!`	複数の Future を同時に進め、最初に終わった枝だけを実行するマクロ	UNIT 03
枝	`select!` の中の「パターン = Future => 式」の1つ1つ	UNIT 03
キャンセル安全	途中で捨てられても、データを失ったり中途半端な状態を残したりしない性質（cancel safety）	UNIT 03
`Unpin`	poll の途中で場所を動かしても平気な型に付く印。`Sleep` などは `Unpin` ではない	UNIT 03
`pin!`	Future を動かないように留めるマクロ。留めると `&mut` で借りて何度も poll できる	UNIT 01・03
channel	チャネル。タスクの間で値を渡す通り道。送る口と受け取る口の1組	UNIT 04
mpsc	送り手は何人でも、受け手は1人の channel（multi-producer, single-consumer）	UNIT 04
bounded / unbounded	箱に入る数の上限がある／無い channel。この巻では上限つきを使う	UNIT 04
backpressure	バックプレッシャー。受け手が追いつかないとき、送り手を待たせたり断ったりするしくみ	UNIT 04
`try_send`	待たずに送る。箱がいっぱいなら、すぐに `Full` のエラーを返す	UNIT 04
oneshot	値を1回だけ届ける channel。依頼への返事に使う	UNIT 04

用語	読み・意味	出てくる単元
アクター	自分だけのデータを持ち、メッセージで届いた依頼を1つずつ処理するタスク	UNIT 04
broadcast	送った1件を、受け手の全員に届ける channel。遅れた受け手は読み飛ばす	UNIT 04
`Lagged`	broadcast で、受け手が追いつけずに読み飛ばした件数を知らせるエラー	UNIT 04
watch	最新の値だけを共有する channel。設定の変更や「止めて」の合図に使う	UNIT 04
Mutex	ミューテックス。中のデータを同時に1人だけがさわれるようにする鍵（標準と tokio の2種類）	UNIT 05
ガード（MutexGuard）	`lock()` が返す「鍵を持っている証拠」。片付けられると鍵が返る	UNIT 05
デッドロック	お互いに鍵が返るのを待って、だれも先へ進めなくなった状態	UNIT 05
await_holding_lock	標準の Mutex のガードを持ったまま `.await` していることを知らせる clippy の警告	UNIT 05
RwLock	読み書き鍵。読む人は何人でも同時に、書く人は1人だけ	UNIT 05
Semaphore	セマフォ。決まった枚数の許可証で、同時に入れる数を決める道具	UNIT 05
permit	許可証。Semaphore からもらい、片付けられるときに返る	UNIT 05
`LazyLock`	最初に使われたときに1回だけ値を作る、標準ライブラリの道具	UNIT 05
atomic 型	`AtomicU64` など。鍵を使わずに、割り込まれない1回の操作で数を書き換えられる型	UNIT 05